# SkillSphere: An AI-Powered Personalized Education and Learning Platform

Talha Rizwan (BCS223076)

Muhammad Danish (BCS223073)

Tehzeen Abbas (BCS223077)

**Supervisor:** Sir Salman Ahmed


FALL-2025

**Department of Computer Science**

**Capital University of Science and Technology, Islamabad**

# *Table of Contents*

## *List of Figures*

## *List of Tables*

# *Chapter 1: Introduction*

## 1.1. Project Introduction

In today's world, traditional education systems often fail to equip students with the practical, in-demand skills required in the modern job market. University programs and online courses mostly follow a fixed structure where the same content is delivered to every student, regardless of their pace or learning ability. This "one-size-fits-all" approach limits creativity, weakens engagement, and leaves learners struggling to apply theoretical knowledge in real scenarios.

To address this issue, we propose an **AI-powered tutoring system** that acts as an intelligent, interactive teacher — available anytime, anywhere. The system focuses on **skill-based learning**, helping students grow from **basic to advanced levels** in a structured yet adaptive manner. It removes the need for pre-recorded videos or fixed lessons by allowing the **Instructor to upload only the course name and outline**. The **AI tutor automatically designs and delivers lectures**, creates examples, generates quizzes, and evaluates assignments — all without manual content creation.

The tutor communicates using **both voice and text**, switching intelligently between speaking, writing on a whiteboard, or showing visuals based on the complexity of the topic. It can pause when the student asks a question, provide an appropriate answer, and then resume the lecture — closely mimicking real teacher behavior.

The system includes **two main learning modes**:

1. **Course Mode** – Students can select structured, skill-based courses that guide them from basic to advanced concepts in programming, design, communication, and other professional domains.
2. **AI Chat Mode** – Students can freely interact with the AI tutor to clarify any topic, ask spontaneous questions, or discuss new concepts beyond the predefined course.

By integrating powerful **AI and NLP APIs** (for text, speech, and interaction), the system offers a lightweight yet intelligent solution that personalizes each session in real time . It encourages continuous learning, self-paced progress, and real skill development — bridging the gap between traditional education and the evolving digital world.

## 1.2. Existing Examples / Solutions

There are a number of applications that provide AI-based or digital learning functionalities. In the following section, some of the existing popular platforms are described.

- **Khan Academy (Khanmigo):** A globally recognized platform that offers video lessons, text explanations, and quizzes across a variety of subjects. While it is comprehensive and free, its major limitation is that it provides **static lessons without AI tutor interaction** or adaptive learning features.

- **QANDA**: A mobile application focused mainly on **math problem solving via image recognition**. Students can upload a photo of a math question and receive step-by-step solutions. However, it is **subject-specific (mainly math)** and does not provide full course structures or voice/text tutoring.

- **Doubtnut**: A tutoring application that provides **step explanations for science and math problems**. It allows limited voice interaction but lacks **adaptive course flow** and continuous personalized lessons.

- **Socratic (by Google)**: An AI-guided problem-solving application where students can input or scan problems to receive AI-based solutions. While useful, it is restricted to **single-problem assistance**, with **no whiteboard, no structured courses, and limited interactive tutoring**.

- **ChatGPT (OpenAI):** A powerful conversational AI capable of providing detailed explanations and engaging in open-ended learning discussions. However, ChatGPT operates in a general-purpose environment and lacks structured course flow, or dedicated educational tools such as quizzes, whiteboard visuals, and real-time voice-based adaptive teaching.

Table 1.1 presents the comparison of features between Khan Academy, QANDA, Doubtnut, Socratic, and the proposed system **SkillSphere**.

*Table 1:Existing Solutions Features Comparison*

| Sr. No. | Characteristics | Khanmigo | QANDA | Doubtnut | Socratic | SkillSphere (Proposed System) |
|---|---|---|---|---|---|---|
| 1 | Voice Interaction | | | Partial | Partial | ✓ (Full student-tutor dialogue) |
| 2 | Text-based Interaction | ✓ | ✓ | ✓ | ✓ | ✓ |
| 3 | Adaptive Learning | Static | Static | Static | Static | ✓ (Personalized progression) |
| 4 | Whiteboard / Visuals | Auto-draw/Illustrate | | | | ✓ (AI whiteboard support) |
| 5 | Pause / Resume (student interrupts) | | | | | ✓ |
| 6 | Assignments & Quizzes | ✓ | Limited | Limited | Limited | ✓ (AI generated, adaptive) |
| 7 | Gamification / Progress Levels | ✓ (Badges) | | | | ✓ (Badges, streaks, milestones) |
| 8 | Free Course Access | ✓ | ✓ | | ✓ | ✓ |
| 9 | Certification | | | | | ✓ (With watermark) |

## 1.3. Business Scope

The demand for AI-powered and skill-based education platforms is rapidly increasing as students and professionals look for smarter, more interactive ways to learn.

Our proposed system, **SkillSphere**, fills this gap by offering a **fully AI-driven tutoring experience** where the tutor not only teaches but also creates its own lectures, examples, and quizzes based on the course outline. The platform focuses on **skill-based learning**, allowing users to build expertise from **basic to advanced levels** across various domains such as programming, design, and communication.

Unlike existing e-learning apps, SkillSphere combines **voice-based interaction**, **AI-generated teaching**, and **real-time responsiveness**. Students can either follow structured courses or engage in open learning mode for broader exploration. With the integration of local language support and 24/7 availability, it ensures accessibility for a wide range of learners.

Given the growing adoption of AI tools in education, this project has strong commercial potential. It can attract **students, educational institutes, and training centers** looking to implement AI tutors for cost-effective, scalable, and personalized learning. With very few competitors offering AI that both **creates and delivers content autonomously**, SkillSphere holds a unique position in the market and can expand rapidly through subscription models, certification fees, and institutional partnerships.

## 1.4. Useful Tools and Technologies

The following tools and technologies will support the development of **SkillSphere**:

- **Frontend Development**
  - **React.js**: to build a responsive and interactive web platform.
  - **Bootstrap:** for modern, mobile-friendly UI design.
- **Backend Development**
  - **Node.js with Express**: for backend logic and REST APIs.
  - **Python (Flask)**: to integrate AI models.
- **Database**
  - **MySQL**: for storing user profiles, progress data, and resources.

- **Artificial Intelligence & NLP**
    - **Chat / Explanation APIs: OpenAI GPT API** for text-based tutoring, Q&A, and explanations; Google Gemini API for multimodal support (text, images, and documents).
- **Mobile Development**
    - **React Native:** to build cross-platform Android and iOS apps.

This technology stack ensures the system will be **scalable, efficient, and accessible** across web and mobile platforms.

## 1.5. Project Work Break Down

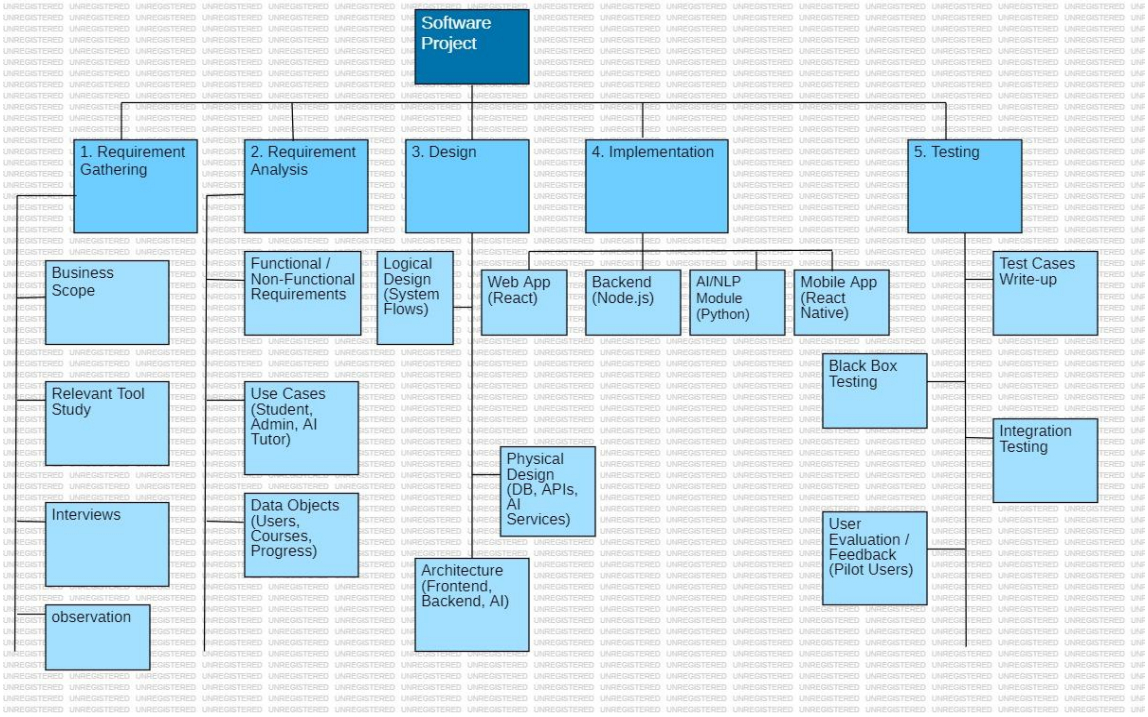



*Figure 1: Project Breakdown*

## 1.6. Project Timeline

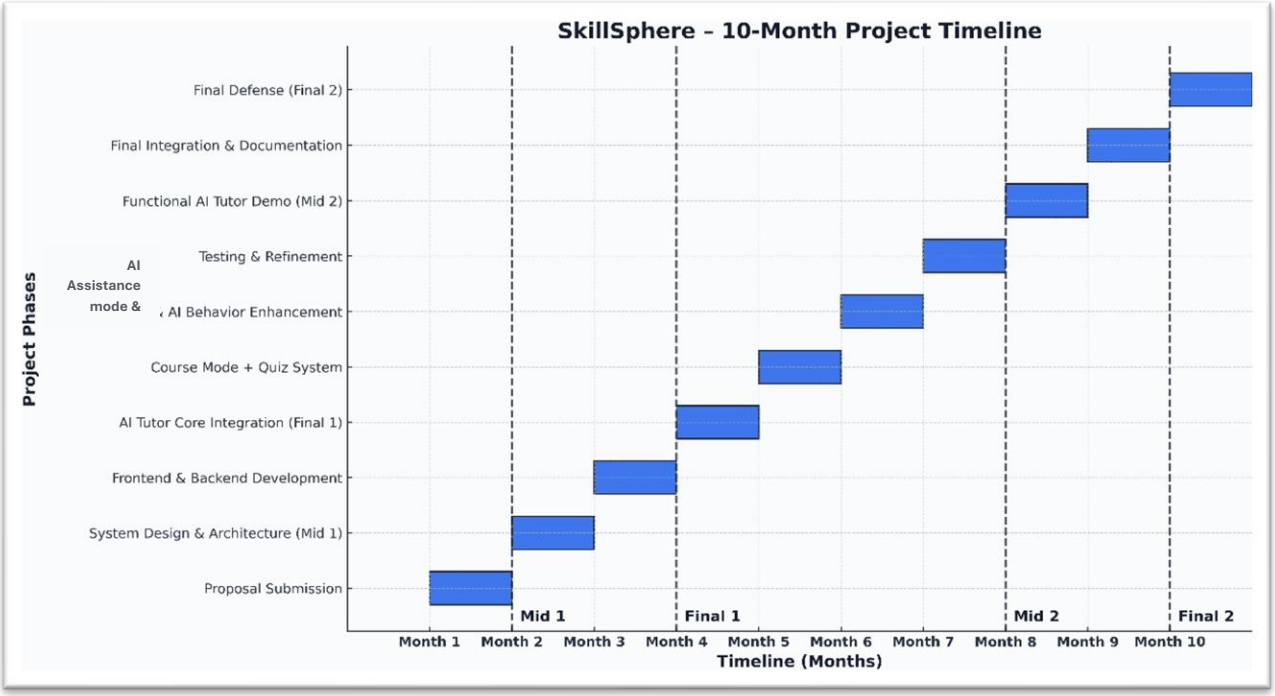



*Figure 2: Project Time Line*

### 1.6.1. Module-wise Breakdown

- **Week 1–2: Requirement Analysis, Gathering & Research**
  - Study similar applications (Socratic, Khanmigo).
  - Finalize functional and non-functional requirements.
  - Shortlist datasets and APIs (Speech-to-Text, Text-to-Speech, LLM).
- **Week 3–4: System Design**
  - Create architecture diagram (Frontend + Backend + AI Integration).
  - Design database schema (Users, Courses, Progress).
  - Prepare UI/UX mockups (Figma/Canva).
- **Week 5–6: Core AI Tutor (Text + Voice Q/A)**
  - **Integrate Chat/Explanation API** (OpenAI GPT API / Google Gemini API).
- **Week 7–8: Whiteboard & Visual Explanation**
  - Add whiteboard feature (Tkinter canvas / React whiteboard).
  - Enable AI to show diagrams or text-based explanations on the whiteboard.
- **Week 9: Instructor Module (Basic)**
  - Instructor login functionality.
  - Instructor can upload course outlines.

- **Week 10–11: Student Module**
  - Student registration/login.
  - Track course progress (levels + basic badges).
- **Week 12: Integration & Testing**
  - Integrate all modules.
  - Test student–AI interaction.
- **Week 13: Bug Fixing & Improvements**
  - Add voice interruption (AI pauses when student asks a question).
  - Enhance UI.
- **Week 14: Final Demo & Report**
  - Prepare prototype presentation.
  - Submit documentation and Part 1 FYP report.
- **Deliverable after 14 weeks (Part 1):**
  - AI tutor that explains in **voice + text**.
  - Student can interrupt with questions, and AI responds.
  - Whiteboard for simple explanations.
  - Instructor can upload course outlines.
  - Student progress tracking available.

# *Chapter 2: Requirement Specification and Analysis*

Requirements analysis is a process of determining user expectations for a new or modified product. These features, called requirements, must be quantifiable, relevant and detailed. In software engineering, such requirements are often called functional specifications. In Chapter 2 we will enlist the functional and non-functional requirements and model functional requirements in the form of use case model.

## 2.1. Functional Requirements

A functional requirement defines a function of a system or its component. Functional requirements may be calculations, technical details, data manipulation and processing and other specific functionality that define what a system is supposed to accomplish.

*Table 2: Instructor Functional Requirements*

| S. No. | Functional Requirement | Type | Status |
|---|---|---|---|
| 1 | Instructor logs in using email login code or password. | Core | Completed |
| 2 | Instructor changes password using email sign-in code if the password is forgotten. | Core | Completed |
| 3 | Instructor creates new Instructor and Expert accounts and assigns login credentials. | Core | Completed |
| 4 | Instructor creates new skill categories for course organization. | Core | Completed |
| 5 | Instructor creates a new course including name, description, level, language, skill category, and duration. | Core | Completed |
| 6 | Instructor adds course topics and outline through a structured form; topics unlock sequentially for students. | Core | Completed |
| 7 | Instructor uploads supporting course material (PDFs, notes, examples). | Core | Completed |
| 8 | Instructor submits the course draft for AI-based lecture generation. | Core | Completed |
| 9 | Instructor receives notification when AI content is generated and reviews the lecture. | Intermediate | Completed |
| 10 | Instructor publishes the course to make it accessible to students. | Core | Completed |
| 11 | Instructor updates the course outline or uploads new material. | Intermediate | Completed |
| 12 | Instructor deletes/hides a course to make it unavailable for students. | Intermediate | Completed |
| 13 | Instructor views registered students' profiles, progress, and performance. | Core | Completed |
| 14 | Instructor blocks a student violating platform policies. | Intermediate | Completed |
| 15 | Instructor unblocks a previously blocked student. | Intermediate | Completed |

| | | | |
|---|---|---|---|
| 16 | Instructor manages certificate templates and course completion criteria. | Core | Completed |
| 17 | Instructor receives feedback from course experts for improvements. | Intermediate | Completed |

*Table 3: Expert Functional Requirements*

| S. No. | Functional Requirement | Type | Status |
|---|---|---|---|
| 1 | Expert logs in using email login code or password. | Core | Completed |
| 2 | Expert resets password through email login code. | Core | Completed |
| 3 | Expert views all course materials and outlines uploaded by Instructor. | Core | Completed |
| 4 | Expert submits feedback on course quality, clarity, and improvements. | Intermediate | Completed |

*Table 4: Student functional Requirements*

| S. No. | Functional Requirement | Type | Status |
|---|---|---|---|
| 1 | Student signs up using email, password, and basic details (Name, Age, Qualification). | Core | Completed |
| 2 | Student logs in using email login code or password. | Core | Completed |
| 3 | Student resets password using email login code verification. | Core | Completed |
| 4 | Student browses skills and selects desired skill categories. | Core | Completed |
| 5 | Student views available published courses under selected skill. | Core | Completed |
| 6 | Student views course outline with locked/unlocked/completed topics. | Core | Completed |
| 7 | Student starts the first unlocked topic. | Core | Completed |
| 8 | Student interacts during lecture using interrupt button (text or voice-based questions). | Core | Completed |
| 9 | System resumes lecture after answering student's question. | Intermediate | Completed |
| 10 | Student completes lecture for a topic. | Core | Completed |
| 11 | Student purchases and downloads AI-generated flashcards. | Intermediate | Completed |
| 12 | Student attempts AI-generated quizzes. | Core | Completed |
| 13 | Student views quiz analysis and AI feedback. | Intermediate | Completed |
| 14 | Student views overall course progress and stats. | Core | Completed |

| | | | |
|---|---|---|---|
| 15 | Student resumes learning from previously saved point. | Core | Completed |
| 16 | Student applies for a certificate after completing a course. | Intermediate | Completed |
| 17 | Student makes payment for certification. | Intermediate | Pending |
| 18 | Student receives automatically generated certificate in PDF. | Core | Completed |
| 19 | Student creates To-Do items or study reminders. | Intermediate | Completed |
| 20 | Student receives notifications for quizzes, deadlines, lectures. | Intermediate | Completed |
| 21 | Student enters AI Assistant mode for additional support. | Core | Completed |
| 22 | Student asks open-ended questions in AI Assistant mode and receives answers. | Core | Completed |

*Table 5: AI Functional Requirements*

| S. No. | Functional Requirement | Type | Status |
|---|---|---|---|
| 1 | AI receives course name, description, outline, and materials at runtime. | Core | Completed |
| 2 | AI generates structured lecture content for each topic. | Core | Completed |
| 3 | AI designs topic-wise quiz questions automatically. | Core | Completed |
| 4 | AI delivers lecture through voice, text, and whiteboard diagrams. | Core | Completed |
| 5 | AI pauses speech when student presses “Interrupt”, answers, and resumes. | Core | Completed |
| 6 | AI evaluates quizzes and provides performance grading. | Core | Completed |
| 7 | AI assists students in free chat mode using voice, text, and visuals. | Intermediate | Completed |
| 8 | AI generates flashcards of each topic and quiz. | Intermediate | Completed |

## 2.2. Non-Functional Requirements

A non-functional requirement is a requirement that specifies criteria that define the quality, performance, and constraints of the system, rather than describing specific system behaviors. These requirements ensure the system operates efficiently, securely, and reliably.

*Table 6: Non-Functional Requirements*

| S. No. | Non-Functional Requirement | Category |
|---|---|---|
| 1 | The system should verify the email, OTP, and login information accurately. | Security |
| 2 | The system should load each screen within 3 seconds under normal network conditions. | Performance |
| 3 | The system should protect all passwords using hashing techniques. | Security |
| 4 | The system should maintain and retrieve course progress, quizzes, and user data correctly. | Reliability |
| 5 | The system should provide an easy-to-navigate UI/UX for all users. | Usability |
| 6 | AI-generated lectures should play smoothly without interruptions. | Performance |
| 7 | The system should support up to 5,000 concurrent users. | Scalability |
| 8 | The system shall offer both light and dark modes. | Usability |
| 9 | All communication should be encrypted using HTTPS. | Security |
| 10 | System database must be backed up every 24 hours. | Reliability |

## 2.3. Selected Functional Requirements

Following is the list of the requirements selected for the current iteration

*Table 7: Selected Functional Requirements*

| S. No. | Functional Requirement | Status |
|---|---|---|
| **1** | Student signs up using email, password, and personal details. | Completed |
| **2** | Student verifies email using OTP. | Completed |
| **3** | Student logs in using verified email and password. | Completed |
| **4** | Student browses available skill categories. | Completed |
| **5** | Student selects a skill and views all published courses under it. | Completed |
| **6** | Student views the course outline (locked/unlocked topics). | Completed |
| **7** | Student starts the first unlocked topic. | Completed |
| **8** | Student interacts with the AI tutor using voice/text-based questions. | Completed |
| **9** | AI pauses the lecture, answers the question, and resumes automatically. | Completed |
| **10** | Student completes the topic and proceeds to the next unlocked topic. | Completed |
| **11** | Student attempts AI-generated quizzes (MCQs). | Completed |
| **12** | Student views quiz results and AI-generated feedback. | Completed |
| **13** | Student downloads AI-generated flashcards. | Completed |
| **14** | Student views overall course progress in the dashboard. | Completed |
| **15** | Student applies for a certificate after course completion. | Completed |
| **16** | Student pays for the certificate through a secure payment method. | Completed |
| **17** | Student receives the certificate in downloadable PDF format. | Completed |
| **18** | Student enters AI Assistant mode to ask additional open-ended questions. | Completed |
| **19** | Instructor logs in to manage system features. | Completed |
| **20** | Instructor creates skill categories. | Completed |
| **21** | Instructor creates a new course and adds description, level, and language. | Completed |

| | | |
|---|---|---|
| **22** | Instructor adds topics and structured outlines. | Completed |
| **23** | Instructor uploads course materials (PDFs, slides, images). | Completed |
| **24** | Instructor submits course to AI for lecture generation. | Completed |
| **25** | Instructor reviews and publishes the AI-generated course. | Completed |
| **26** | Instructor updates or deletes an existing course. | Completed |
| **27** | Expert logs in to review course materials. | Completed |
| **28** | Expert provides feedback on the course. | Completed |
| **29** | Instructor receives expert feedback for improvements. | Completed |

## 2.4. System Use Case Modeling

A use case is a list of actions or event steps, typically defining the interactions between a role (known in the Unified Modeling Language as an actor) and a system, to achieve a goal. The actor can be a human or other external system. Customer's use cases are shown in the following the figure.

### 2.4.1. Instructor Use Case

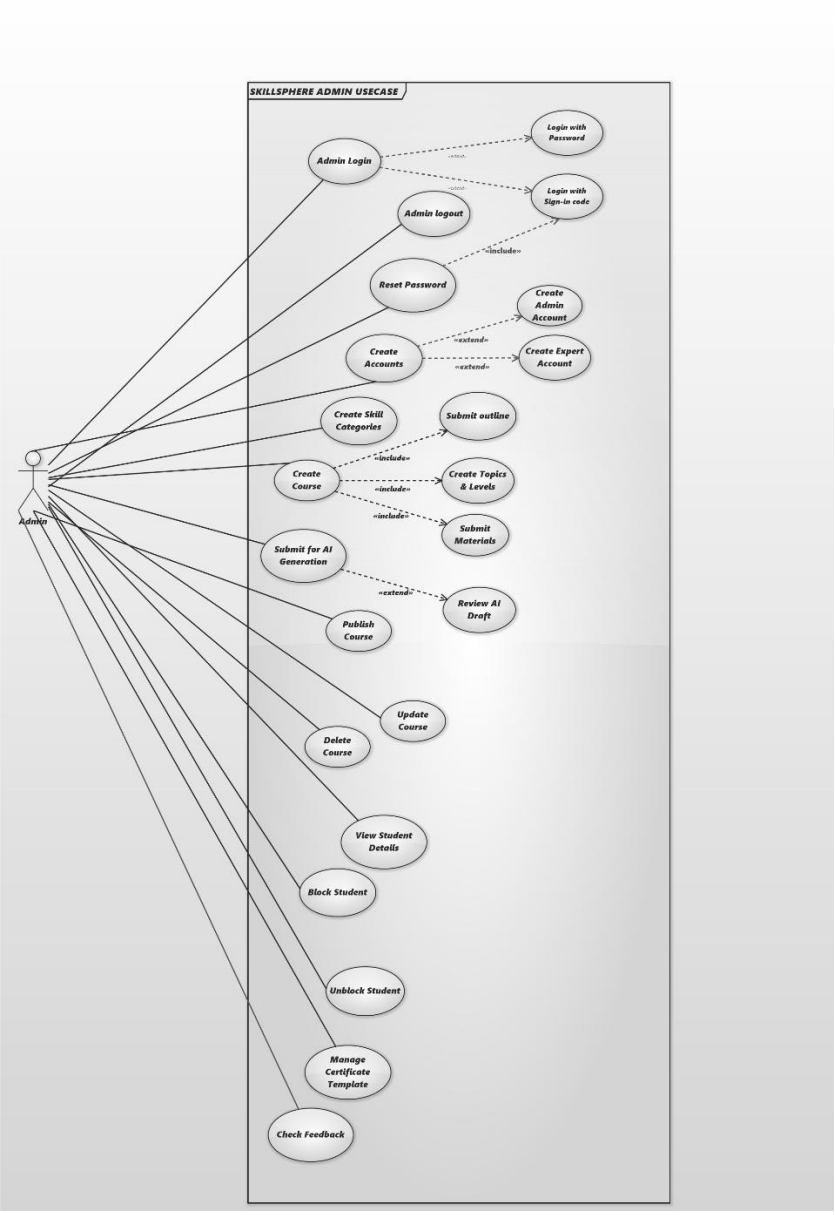



*Figure 3: Instructor Use Case Diagram*

*Table 8: Instructor Login/Signup Usecase*

| **Use Case ID** | **UC-A1** | |
|---|---|---|
| **Field** | **Value** | |
| Use Case Name | Instructor Login / Logout | |
| Created By | SkillSphere Team | |
| Last Updated By | SkillSphere Team | |
| Date Created | 2025-11-28 | |
| Last Revision Date | 2025-11-28 | |
| Actors | Instructor | |
| Description | Instructor authenticates using email/password or email OTP and can logout to invalidate session. | |
| Trigger | Instructor opens Instructor Portal and selects Login / Logout. | |
| Preconditions | Instructor account exists and is verified; network available. | |
| Post conditions | Active Instructor session created or session tokens revoked on logout. | |
| Normal Flow | Instructor<br>1. Instructor enters credentials or requests OTP<br>2. Instructor submits OTP (if used)<br>3. Instructor selects Logout. | System<br>1. System validates credentials or sends OTP.<br>2. System issues access + refresh tokens and allows access.<br>3. System invalidates refresh token and ends session. |
| Alternative Flows | A1: Forgot password → reset via email OTP. A2: Unverified account → system prompts verification. | |
| Exceptions | 1. Invalid credentials → show error. 2. Auth service down → display maintenance message. | |

*Table 9:Create Instructor / Expert Account Use case*

| **Use Case ID** | **UC-A2** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Create Instructor / Expert Account |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Instructor |
| Description | Instructor creates new Instructor or Course Expert accounts and sends an invite/verification. |
| Trigger | Instructor clicks “Create Account” in Instructor panel. |
| Preconditions | Instructor authenticated with appropriate privileges. |
| Post conditions | New user record created and invitation/verification sent. |
| Normal Flow | Instructor — System<br>1. Instructor fills user form and submits. — System validates, creates user record and sends invite email.<br>2. New user completes verification and sets password. — System activates the account. |
| Alternative Flows | A1: Email exists → show option to resend invite or link existing account. |
| Exceptions | 1. Email delivery failure. 2. DB write failure. |

*Table 10: Create Skill Category use case*

| **Use Case ID** | **UC-A3** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Create Skill Category |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Instructor |
| Description | Instructor creates hierarchical skill categories used to organize courses. |
| Trigger | Instructor selects “Create Skill Category”. |
| Preconditions | Instructor authenticated. |
| Post conditions | Category persisted and visible in course creation lists. |
| Normal Flow | Instructor — System<br>1. Instructor enters category name/description and submits. — System validates and persists category.<br>2. System shows success; category appears in UI lists. |
| Alternative Flows | A1: Duplicate name → prompt to rename or merge. |
| Exceptions | 1. DB save error. |

*Table 11: Create Course & Add Topics use case*

| **Use Case ID** | **UC-A4** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Create Course & Add Topics |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Instructor |
| Description | Instructor creates a course (meta) and ordered topic outline; first topic auto-unlocked. |
| Trigger | Instructor clicks “Create Course” or uploads outline file. |
| Preconditions | Instructor authenticated; skill categories exist. |
| Post conditions | Course and topic records created; topic[0] set unlocked. |
| Normal Flow | Instructor — System<br>1. Instructor fills metadata and outline or uploads CSV/JSON. — System validates input.<br>2. Instructor submits. — System creates course & topic records, sets first topic unlocked.<br>3. System displays course draft. |
| Alternative Flows | A1: Malformed upload → show validation errors. |
| Exceptions | 1. Partial save → rollback. |

*Table 12: Upload Supporting Materials use case*

| **Use Case ID** | **UC-A5** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Upload Supporting Materials |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Instructor |
| Description | Instructor uploads files (PDF, slides, images) and associates them with course topics. |
| Trigger | Instructor selects “Upload Material” for a course/topic. |
| Preconditions | Course exists; storage configured. |
| Post conditions | Files stored and linked to topic records. |
| Normal Flow | Instructor — System<br>1. Instructor selects files and uploads. — System validates type/size, stores files.<br>2. System links files to topic and confirms upload. |
| Alternative Flows | A1: Large file → chunked upload with progress. |
| Exceptions | 1. Unsupported format. 2. Storage quota exceeded. |

*Table 13: Submit Course to AI & Review Generated Content use case*

| **Use Case ID** | **UC-A6** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Submit Course to AI & Review Generated Content |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Instructor, System (AI Worker) |
| Description | Instructor triggers AI generation for topics; reviews drafts and approves for publishing. |
| Trigger | Instructor clicks “Generate Lectures (AI)”. |
| Preconditions | Outline and materials uploaded; AI integrations configured. |
| Post conditions | topic_content_draft saved; Instructor notified; content approved or revised. |
| Normal Flow | Instructor — System<br>1. Instructor requests AI generation. — System enqueues per-topic jobs.<br>2. Worker calls AI APIs and saves drafts. — System notifies Instructor on completion.<br>3. Instructor reviews and approves or requests regen. — System marks content ready when approved. |
| Alternative Flows | A1: AI failure → show fallback cached text and allow manual edit. |
| Exceptions | 1. AI API timeouts/rate-limits. |

*Table 14: Publish / Update / Delete Course use case*

| **Use Case ID** | **UC-A7** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Publish / Update / Delete Course |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Instructor |
| Description | Instructor publishes course to students, updates metadata or hides/deletes a course. |
| Trigger | Instructor selects Publish / Edit / Delete on a course. |
| Preconditions | Course draft exists; Instructor authenticated. |
| Post conditions | Course status updated; changes visible to students. |
| Normal Flow | Instructor — System<br>1. Instructor clicks Publish → System sets course.status = published and notifies students.<br>2. Instructor edits → System saves new draft/version.<br>3. Instructor deletes/hides → System archives or sets status hidden. |
| Alternative Flows | A1: Partial publish of selected topics. |
| Exceptions | 1. Publishing while AI generation incomplete → block or warn. |

*Table 15: Manage Certificates & Criteria use case*

| **Use Case ID** | **UC-A8** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Manage Certificates & Criteria |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Instructor |
| Description | Instructor configures certificate templates, pass thresholds and payment requirements. |
| Trigger | Instructor opens certificate settings for a course. |
| Preconditions | Course exists. |
| Post conditions | Certificate template saved and used for generation. |
| Normal Flow | Instructor — System<br>1. Instructor configures template, watermark, pass threshold. — System saves settings.<br>2. On completion+payment, System generates certificate PDF per template. |
| Alternative Flows | A1: Manual override per student. |
| Exceptions | 1. Missing template assets. |

*Table 16: View Students, Block / Unblock, Receive Expert Feedback use case*

| **Use Case ID** | **UC-A9** |
|---|---|
| **Field** | **Value** |
| Use Case Name | View Students, Block / Unblock, Receive Expert Feedback |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Instructor |
| Description | Instructor views student profiles/progress, blocks/unblocks students and reviews expert feedback. |
| Trigger | Instructor opens student profile or feedback notification. |
| Preconditions | Instructor authenticated; student/expert accounts exist. |
| Post conditions | Student status updated; feedback logged. |
| Normal Flow | Instructor — System<br>1. Instructor views student dashboard → System fetches progress & logs.<br>2. Instructor selects Block/Unblock → System updates status and notifies user.<br>3. Instructor reviews expert feedback → System shows feedback entries. |
| Alternative Flows | A1: Automated block suggestions require Instructor confirmation. |
| Exceptions | 1. Privacy deletion request conflicts. |

## 2.4.2. Expert Use Case

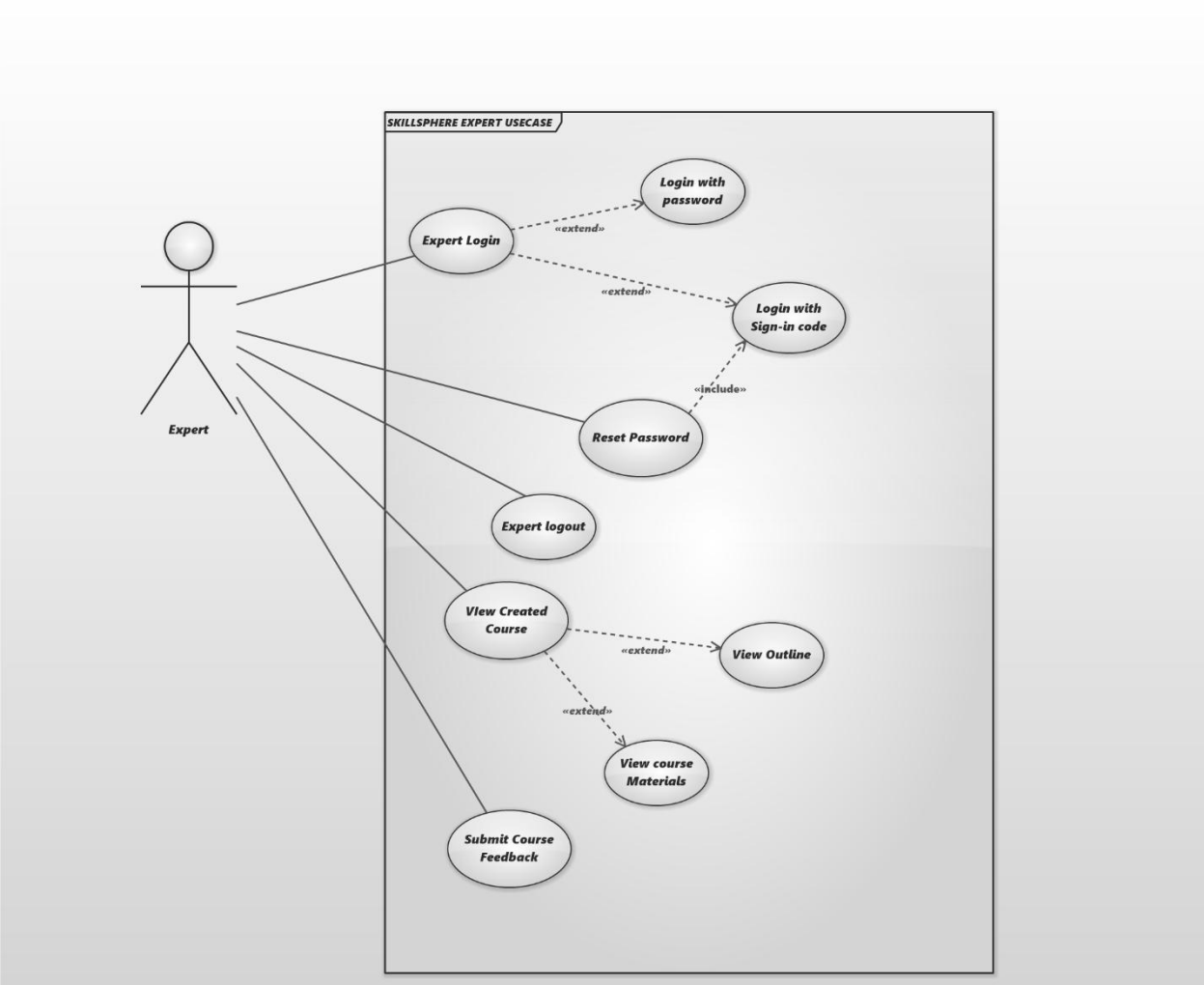



*Figure 4: Expert Use Case Diagram*

*Table 17: Expert Login / Password Reset use case*

| **Use Case ID** | **UC-E1** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Expert Login / Password Reset |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Course Expert |
| Description | Expert authenticates and can reset password using email OTP. |
| Trigger | Expert opens Expert Portal and logs in or selects Reset Password. |
| Preconditions | Expert account exists and verified. |
| Post conditions | Expert session active or password reset completed. |
| Normal Flow | Expert — System<br>1. Expert submits credentials or requests OTP. — System validates and issues tokens.<br>2. For reset, System sends OTP and allows password change. |
| Alternative Flows | A1: Locked account → contact Instructor. |
| Exceptions | 1. OTP delivery failure. |

*Table 18: View Course Materials & Outlines use case*

| **Use Case ID** | **UC-E2** |
|---|---|
| **Field** | **Value** |
| Use Case Name | View Course Materials & Outlines |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Course Expert |
| Description | Expert reviews course metadata, outlines and uploaded materials to prepare feedback. |
| Trigger | Expert selects a course to review. |
| Preconditions | Expert authenticated and has permission to view the course. |
| Post conditions | Expert reads materials and prepares feedback. |
| Normal Flow | Expert — System<br>1. Expert selects course → System retrieves materials and drafts.<br>2. Expert reviews and annotates items for feedback. |
| Alternative Flows | A1: Missing access → request permission. |
| Exceptions | 1. File retrieval error. |

*Table 19: Submit Feedback use case*

| **Use Case ID** | **UC-E3** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Submit Feedback |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Course Expert |
| Description | Expert submits structured feedback (ratings, comments, suggested edits). |
| Trigger | Expert clicks “Submit Feedback”. |
| Preconditions | Expert authenticated and viewing course. |
| Post conditions | Feedback persisted and Instructor notified. |
| Normal Flow | Expert — System<br>1. Expert fills feedback form and submits. — System saves feedback and notifies Instructor.<br>2. Instructor reviews feedback later. |
| Alternative Flows | A1: Attach annotated documents. |
| Exceptions | 1. Storage error. |

### 2.4.3. Student Use Case

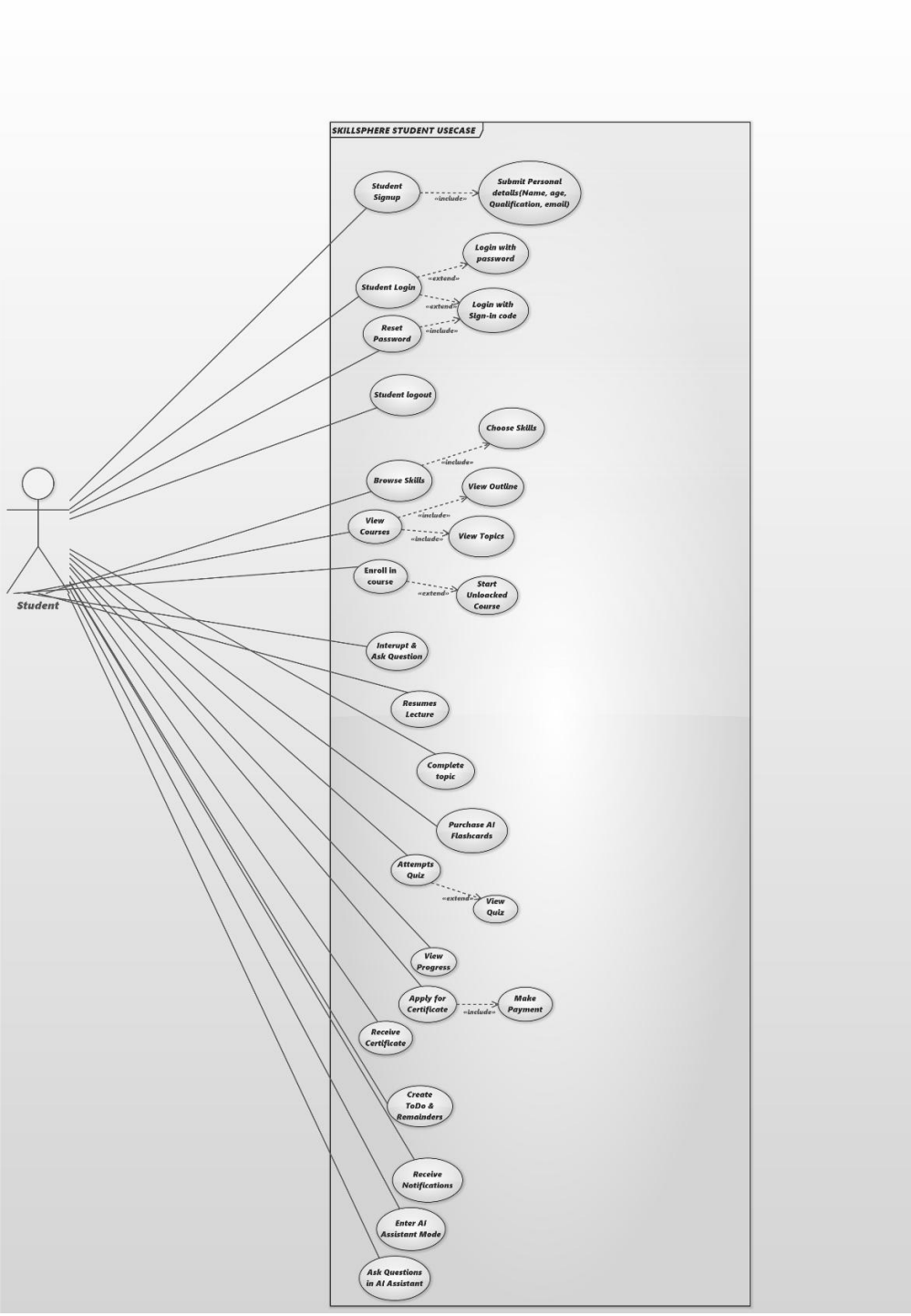



*Figure 5: Student Use Case Diagram*

*Table 20: Student Signup & Email Verification (OTP) use case*

| **Use Case ID** | **UC-S1** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Student Signup & Email Verification (OTP) |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Student |
| Description | New student registers with details and verifies email via OTP. |
| Trigger | Student clicks Sign Up and submits form. |
| Preconditions | Unique email; network available. |
| Post conditions | Account activated and usable. |
| Normal Flow | Student — System<br>1. Student submits registration form. — System creates account in pending state and sends OTP.<br>2. Student enters OTP. — System verifies and activates account. |
| Alternative Flows | A1: OAuth signup → skip OTP if provider verified. |
| Exceptions | 1. OTP expired or not received. |

*Table 21: Student Login & Password Reset use case*

| **Use Case ID** | **UC-S2** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Student Login & Password Reset |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Student |
| Description | Student logs in or resets password via email OTP. |
| Trigger | Student submits login or requests password reset. |
| Preconditions | Account active. |
| Post conditions | Student session created or password updated. |
| Normal Flow | Student — System<br>1. Student submits credentials → System validates and issues tokens.<br>2. For reset, System sends reset OTP and allows new password. |
| Alternative Flows | A1: Social login via OAuth. |
| Exceptions | 1. Invalid credentials. |

*Table 22: Browse Skill Categories & View Courses use case*

| **Use Case ID** | **UC-S3** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Browse Skill Categories & View Courses |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Student |
| Description | Student browses skills and views published courses under a selected skill. |
| Trigger | Student opens the course catalog. |
| Preconditions | Student logged in (or guest with limited access). |
| Post conditions | Student views course list and can enroll/bookmark. |
| Normal Flow | Student — System<br>1. Student opens Browse → System lists skill categories.<br>2. Student selects a skill → System lists published courses under it.<br>3. Student selects a course → System shows course details and outline. |
| Alternative Flows | A1: Search/Filter by level or language. |
| Exceptions | 1. No courses found in selected skill. |

*Table 23: View Outline; Start Topic; Resume use case*

| **Use Case ID** | **UC-S4** |
|---|---|
| **Field** | **Value** |
| Use Case Name | |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Student |
| Description | Student views topic list (locked/unlocked/completed), starts first unlocked topic and resumes from saved point. |
| Trigger | Student opens course or clicks Resume. |
| Preconditions | Course published and student enrolled. |
| Post conditions | Lecture playback starts and resume_point recorded. |
| Normal Flow | Student — System<br>1. Student opens course → System shows outline with statuses.<br>2. Student clicks Start/Resume → System loads lecture and resume_point if present.<br>3. Student begins playback. |
| Alternative Flows | A1: Student selects previous topic (if unlocked). |
| Exceptions | 1. Progress data out-of-sync. |

*Table 24: Interrupt Lecture → Pause → Answer → Resume use case*

| **Use Case ID** | **UC-S5** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Interrupt Lecture → Pause → Answer → Resume |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Student, AI Tutor, System |
| Description | Student interrupts an ongoing lecture (voice/text); system pauses playback, sends context+question to AI, plays answer, and resumes lecture. |
| Trigger | Student presses Interrupt or VAD detects question. |
| Preconditions | Lecture playing; VAD/STT/TTS and LLM available. |
| Post conditions | Resume point restored; Q/A logged. |
| Normal Flow | Student — System / AI — System<br>1. Student speaks or taps Interrupt → System captures audio/text. — System runs STT and validates intent.<br>2. System pauses TTS playback and saves resume_point. — System constructs prompt with topic context and calls LLM.<br>3. AI returns answer → System plays TTS and/or displays visuals.<br>4. System resumes lecture from saved point. |
| Alternative Flows | A1: Low STT confidence → prompt user to repeat. A2: AI timeout → show cached text answer and resume. |
| Exceptions | 1. False trigger from noise. 2. AI service outage. |

*Table 25: Complete Topic → Take Quiz → View Results & Unlock Next use case*

| **Use Case ID** | **UC-S6** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Complete Topic → Take Quiz → View Results & Unlock Next |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Student, AI Tutor, System |
| Description | Student completes topic, takes AI-generated quiz; system auto-grades and unlocks next topic on pass. |
| Trigger | Student clicks Finish or reaches end of topic. |
| Preconditions | Quiz exists for the topic. |
| Post conditions | Quiz results stored and next topic unlocked if pass. |
| Normal Flow | Student — System / AI — System<br>1. Student finishes topic → System fetches quiz.<br>2. Student completes quiz → System grades MCQs and uses LLM/rubric for short answers.<br>3. System records score and if score ≥ pass_threshold → marks topic completed and unlocks next topic; notifies student. |
| Alternative Flows | A1: Failure → provide feedback and allow retry. |
| Exceptions | 1. Grading inconsistency flagged for review. |

*Table 26: Purchase & Download Flashcards; View Dashboard use case*

| **Use Case ID** | **UC-S7** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Purchase & Download Flashcards; View Dashboard |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Student, System, Payment Gateway |
| Description | Student purchases AI-generated flashcards and downloads them; views overall progress on dashboard. |
| Trigger | Student clicks Buy Flashcards or opens Dashboard. |
| Preconditions | Payment gateway configured; flashcards generated. |
| Post conditions | Payment recorded; flashcards access granted and downloadable. |
| Normal Flow | Student — System — Payment Gateway<br>1. Student initiates purchase → System creates payment session.<br>2. Payment gateway processes; on success → System grants access and provides download link.<br>3. Student opens Dashboard → System returns progress stats. |
| Alternative Flows | A1: Payment fails → notify user and retry. |
| Exceptions | 1. Gateway outage. |

*Table 27: Apply & Pay for Certificate; Receive PDF use case*

| **Use Case ID** | **UC-S8** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Apply & Pay for Certificate; Receive PDF |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Student, System, Payment Gateway |
| Description | Student applies for course certificate, pays if required, and downloads PDF certificate. |
| Trigger | Student selects Apply for Certificate. |
| Preconditions | Course completion verified and pass criteria met. |
| Post conditions | Certificate generated and available for download. |
| Normal Flow | Student — System — Payment Gateway<br>1. Student applies → System verifies completion and pass.<br>2. If paid, System creates payment session; on success → System generates PDF certificate and stores it.<br>3. Student downloads certificate. |
| Alternative Flows | A1: Free certificate → immediate generation. |
| Exceptions | 1. Payment/webhook failure. |

*Table 28: AI Assistant Mode (Open Chat) & Save Notes use case*

| **Use Case ID** | **UC-S9** |
|---|---|
| **Field** | **Value** |
| Use Case Name | AI Assistant Mode (Open Chat) & Save Note |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Student, AI Tutor |
| Description | Student asks open-ended questions in chat mode; AI responds and student may save answer as a note. |
| Trigger | Student opens Assistant and submits question. |
| Preconditions | AI services available; student authenticated. |
| Post conditions | Answer delivered and optionally saved to student notes. |
| Normal Flow | Student — System — AI<br>1. Student enters question (text/voice) → System runs STT if voice.<br>2. System builds prompt (with optional course context) and calls LLM. — AI returns answer.<br>3. System displays/plays answer.<br>4. Student optionally saves answer → System stores note. |
| Alternative Flows | A1: LLM timeout → show cached guidance. |
| Exceptions | 1. Sensitive content blocked. |

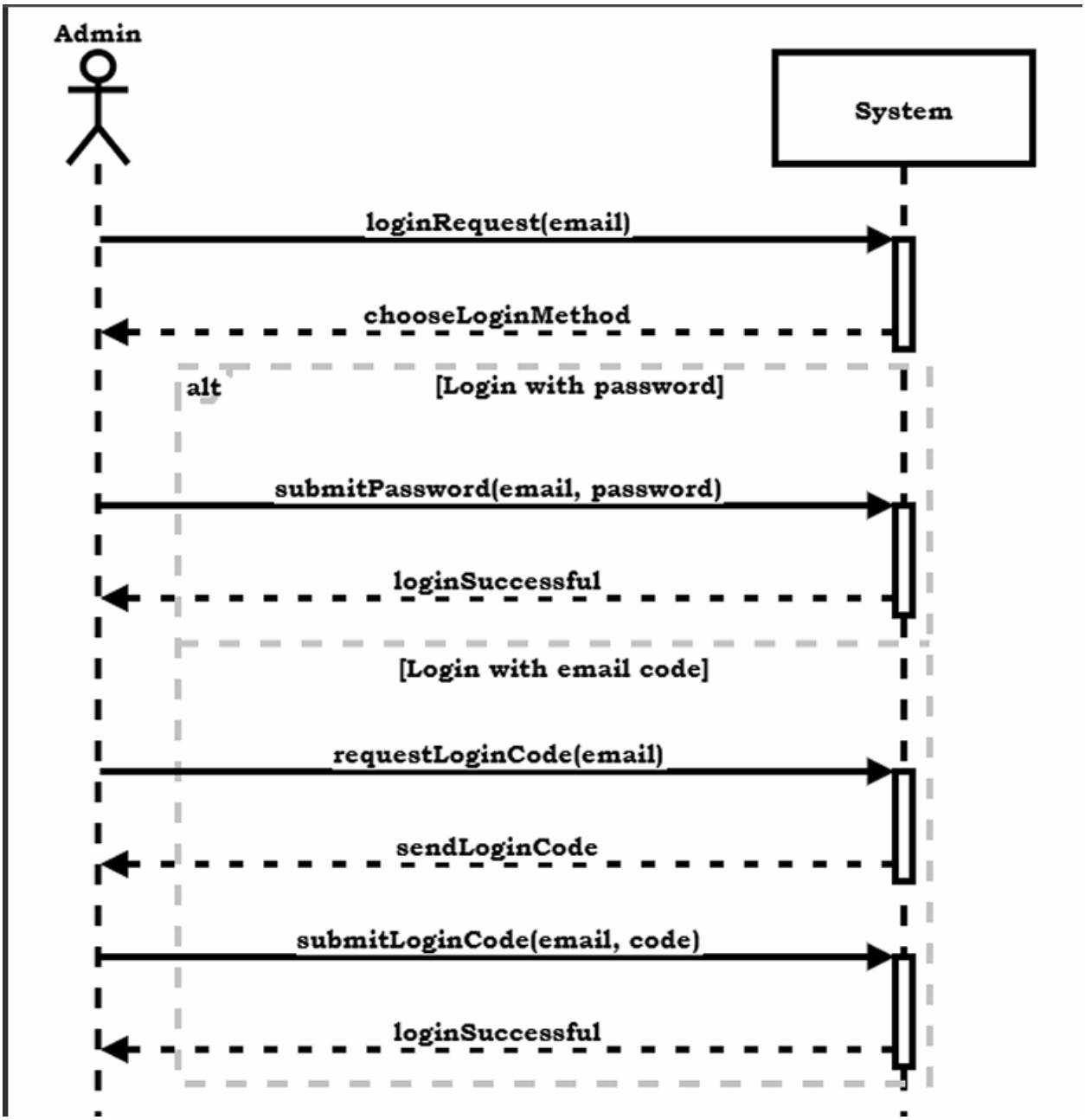
*Table 29: Create To-Do & Receive Notifications use case*

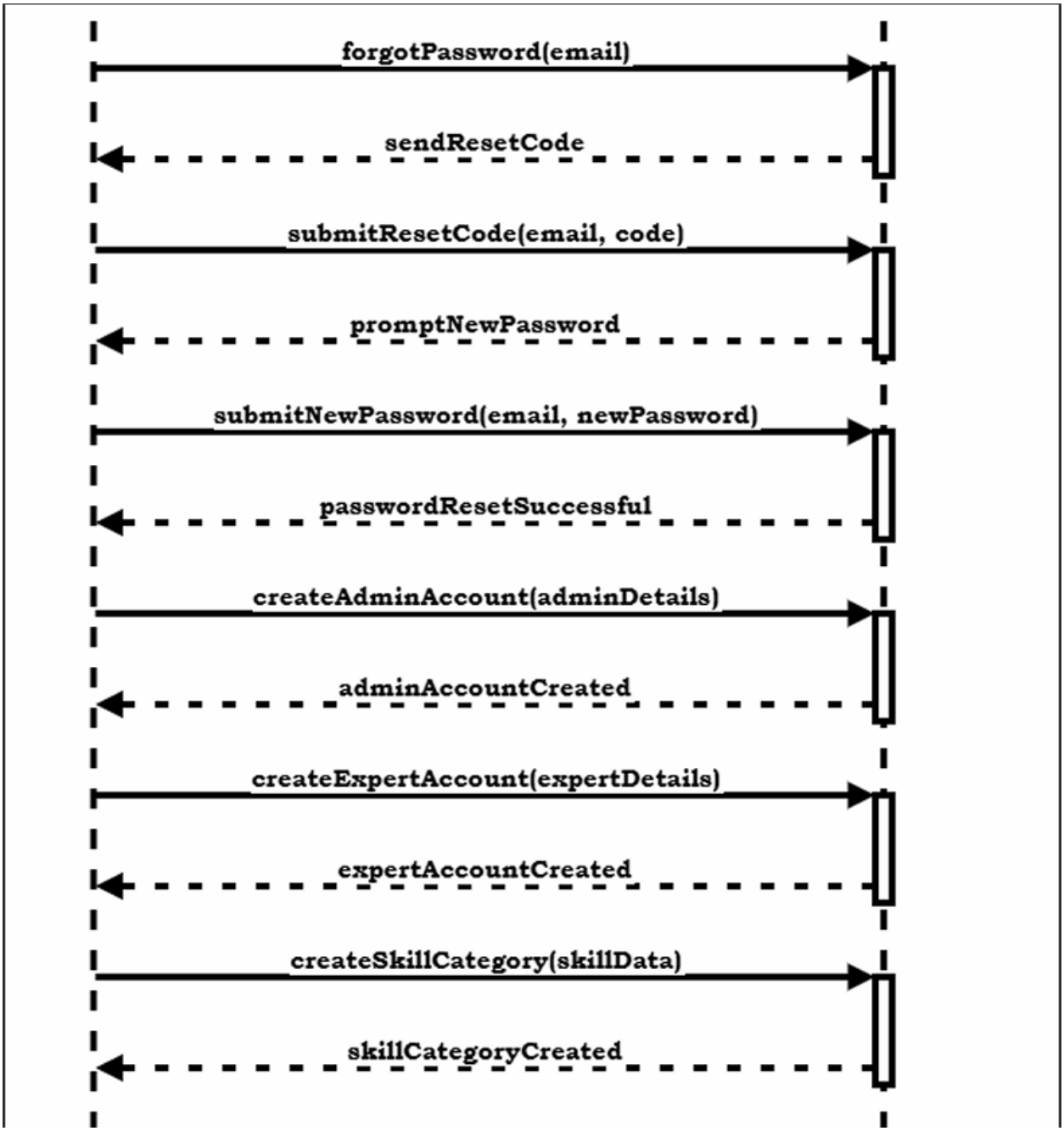
| **Use Case ID** | **UC-S10** |
|---|---|
| **Field** | **Value** |
| Use Case Name | Create To-Do & Receive Notifications |
| Created By | SkillSphere Team |
| Last Updated By | SkillSphere Team |
| Date Created | 2025-11-28 |
| Last Revision Date | 2025-11-28 |
| Actors | Student, System |
| Description | Student creates study tasks and receives scheduled reminders via push/email. |
| Trigger | Student creates task or system schedules reminder. |
| Preconditions | Device registered for push or email configured. |
| Post conditions | Notification delivered and task state updated. |
| Normal Flow | Student — System — Notification Provider<br>1. Student creates to-do with date/time → System stores task and schedules notification.<br>2. At scheduled time → System sends push/email via provider.<br>3. Student marks complete → System updates task. |
| Alternative Flows | A1: Provider failure → fallback to email or retry. |
| Exceptions | 1. Push delivery failure. |

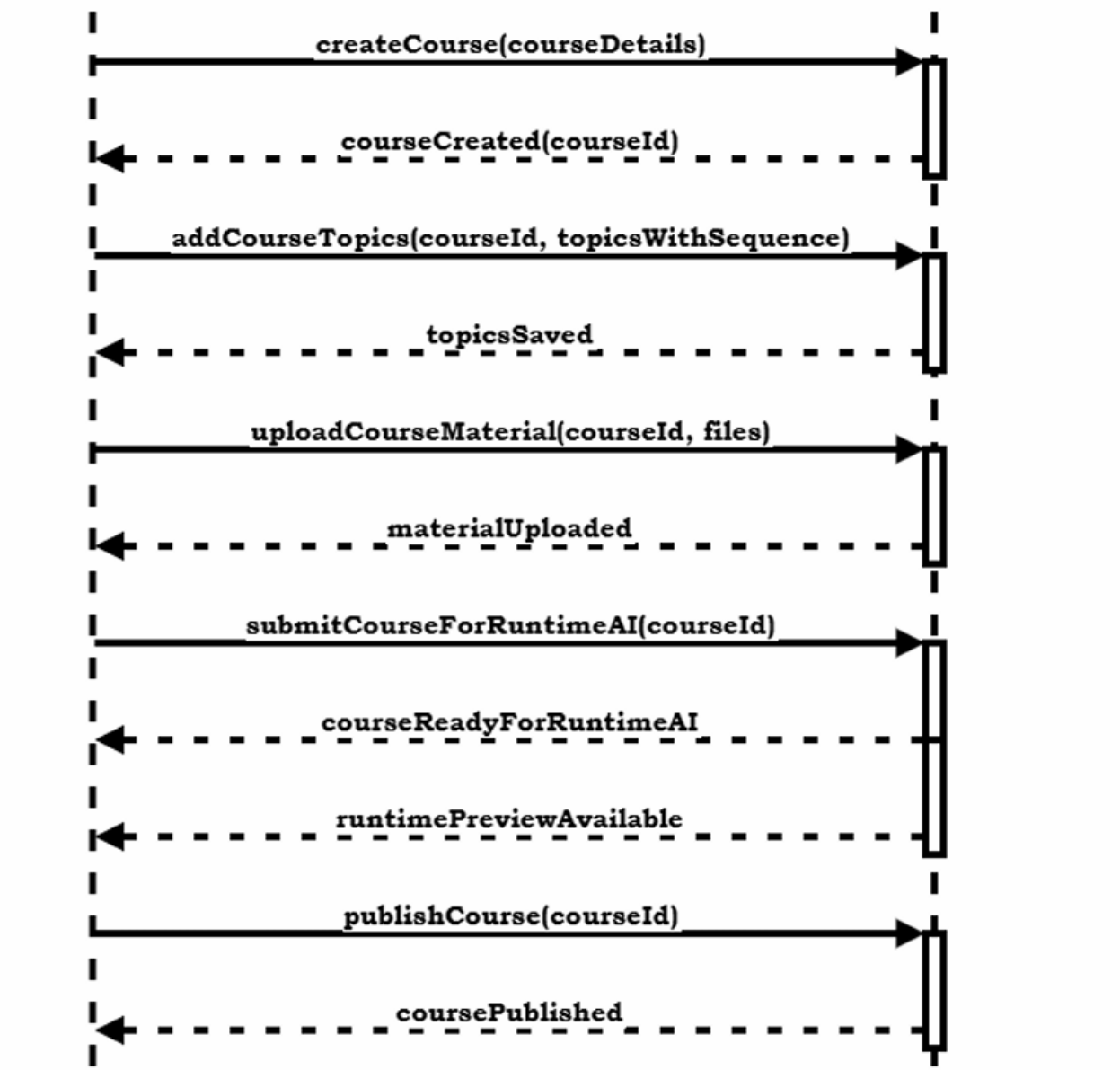
## 2.5. System Sequence Diagram

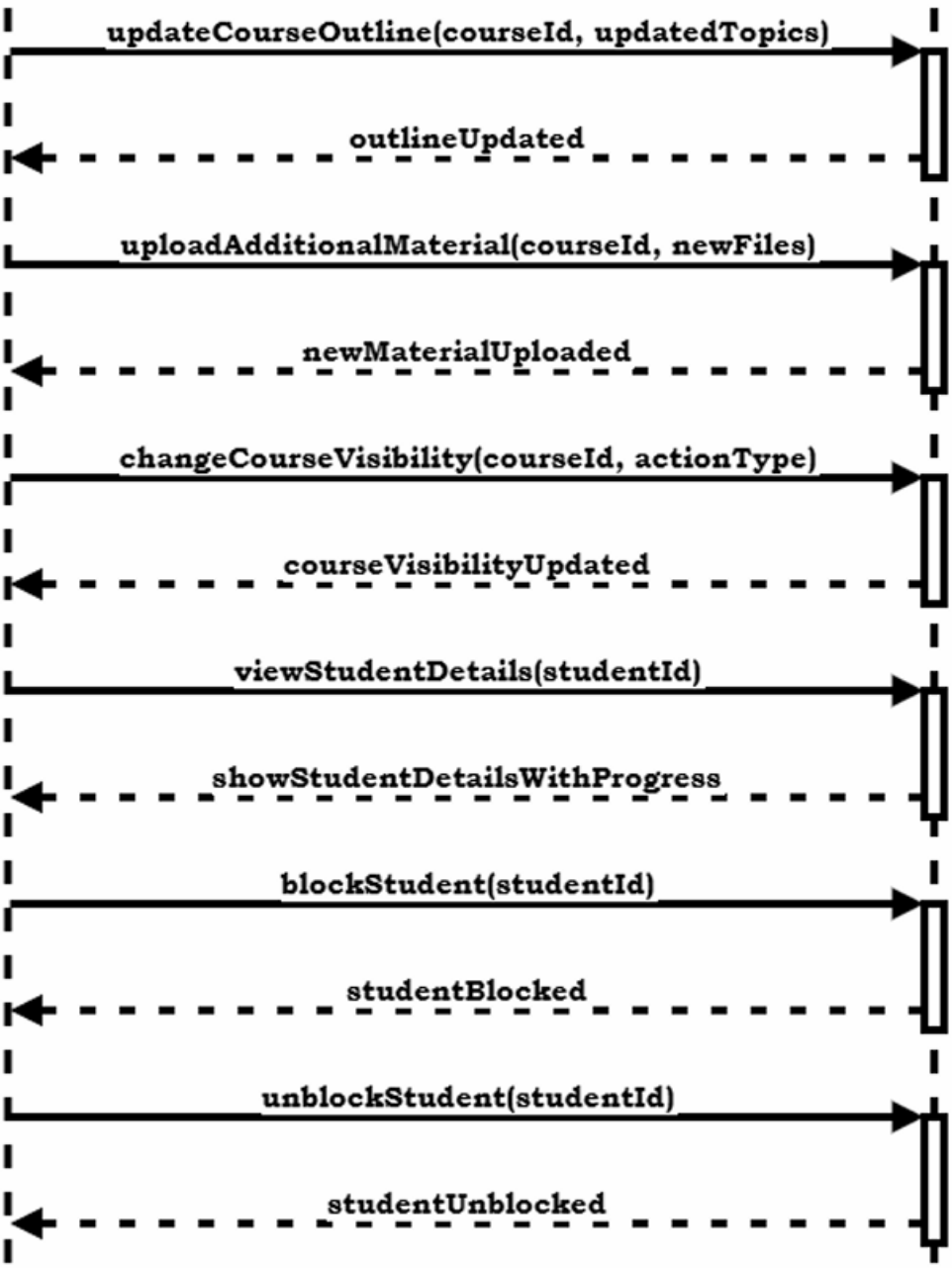
System sequence diagram (SSD) is a sequence diagram that shows, for a particular scenario of a use case, the events that external actors generate their order, and possible inter-system events.

### 2.5.1. Instructor SSD




forgotPassword(email)
sendResetCode
submitResetCode(email, code)
promptNewPassword
submitNewPassword(email, newPassword)
passwordResetSuccessful
createAdminAccount(adminDetails)
adminAccountCreated
createExpertAccount(expertDetails)
expertAccountCreated
createSkillCategory(skillData)
skillCategoryCreated


createCourse(courseDetails)
courseCreated(courseId)
addCourseTopics(courseId, topicsWithSequence)
topicsSaved
uploadCourseMaterial(courseId, files)
materialUploaded
submitCourseForRuntimeAI(courseId)
courseReadyForRuntimeAI
runtimePreviewAvailable
publishCourse(courseId)
coursePublished


updateCourseOutline(courseId, updatedTopics)
outlineUpdated
uploadAdditionalMaterial(courseId, newFiles)
newMaterialUploaded
changeCourseVisibility(courseId, actionType)
courseVisibilityUpdated
viewStudentDetails(studentId)
showStudentDetailsWithProgress
blockStudent(studentId)
studentBlocked
unblockStudent(studentId)
studentUnblocked

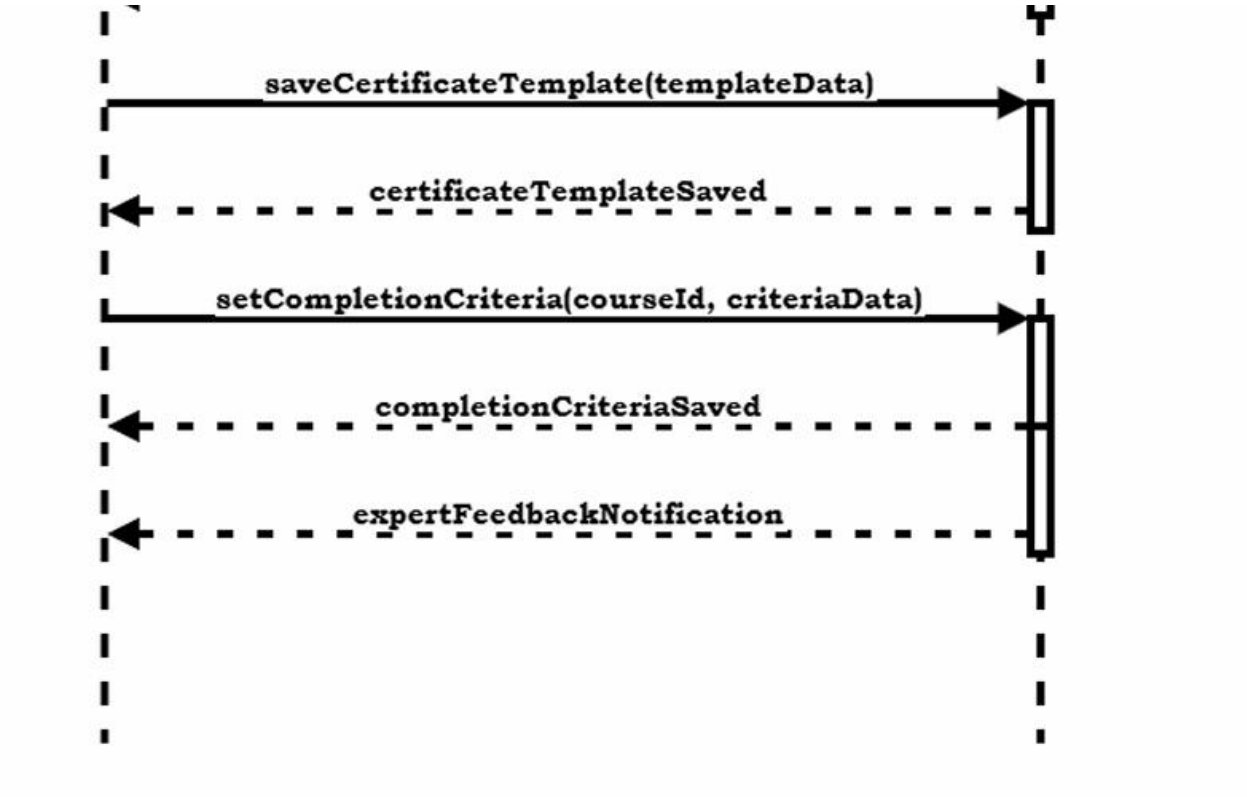
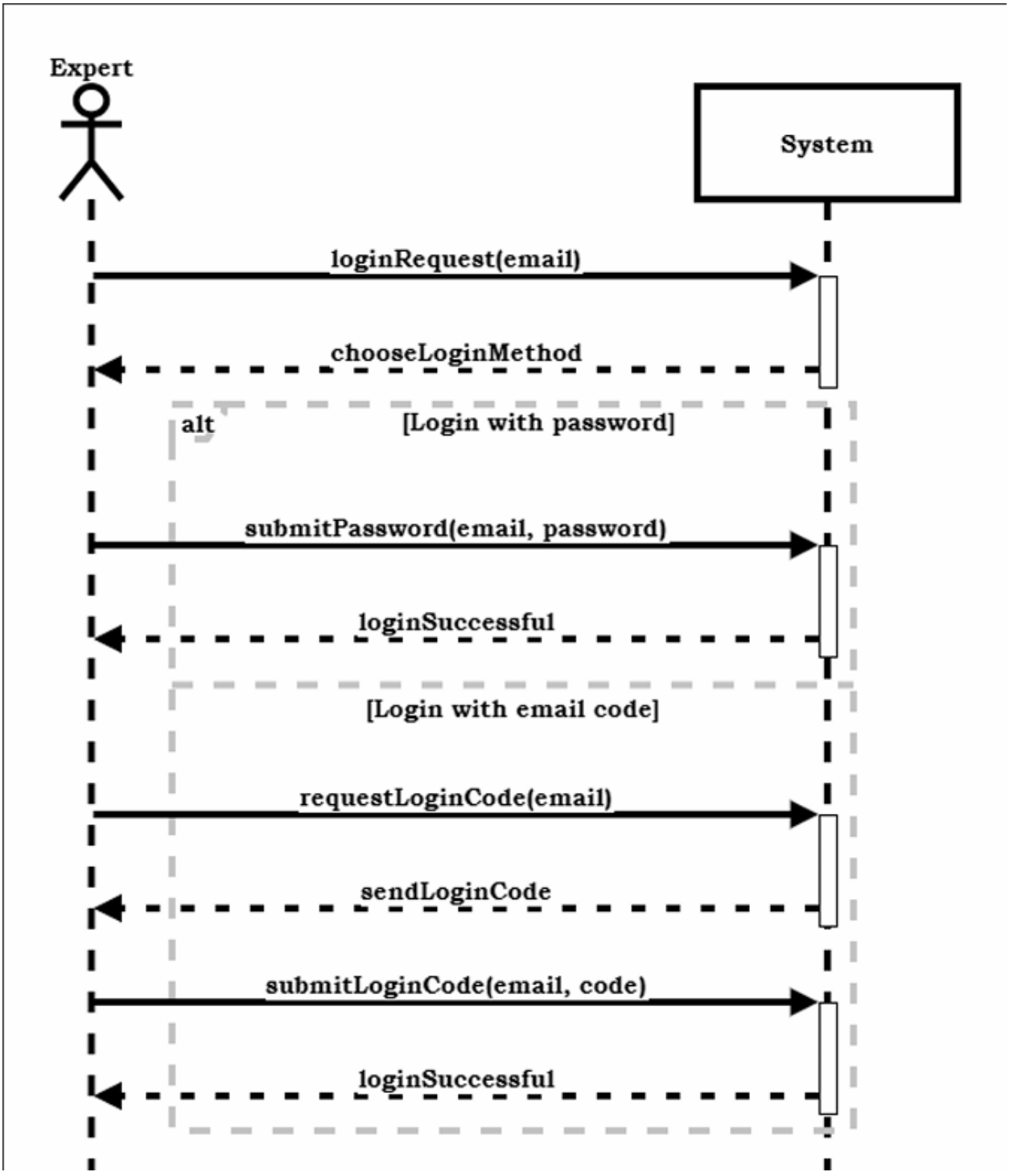



*Figure 6: Instructor SSD Diagram*

### 2.5.2. Expert SSD



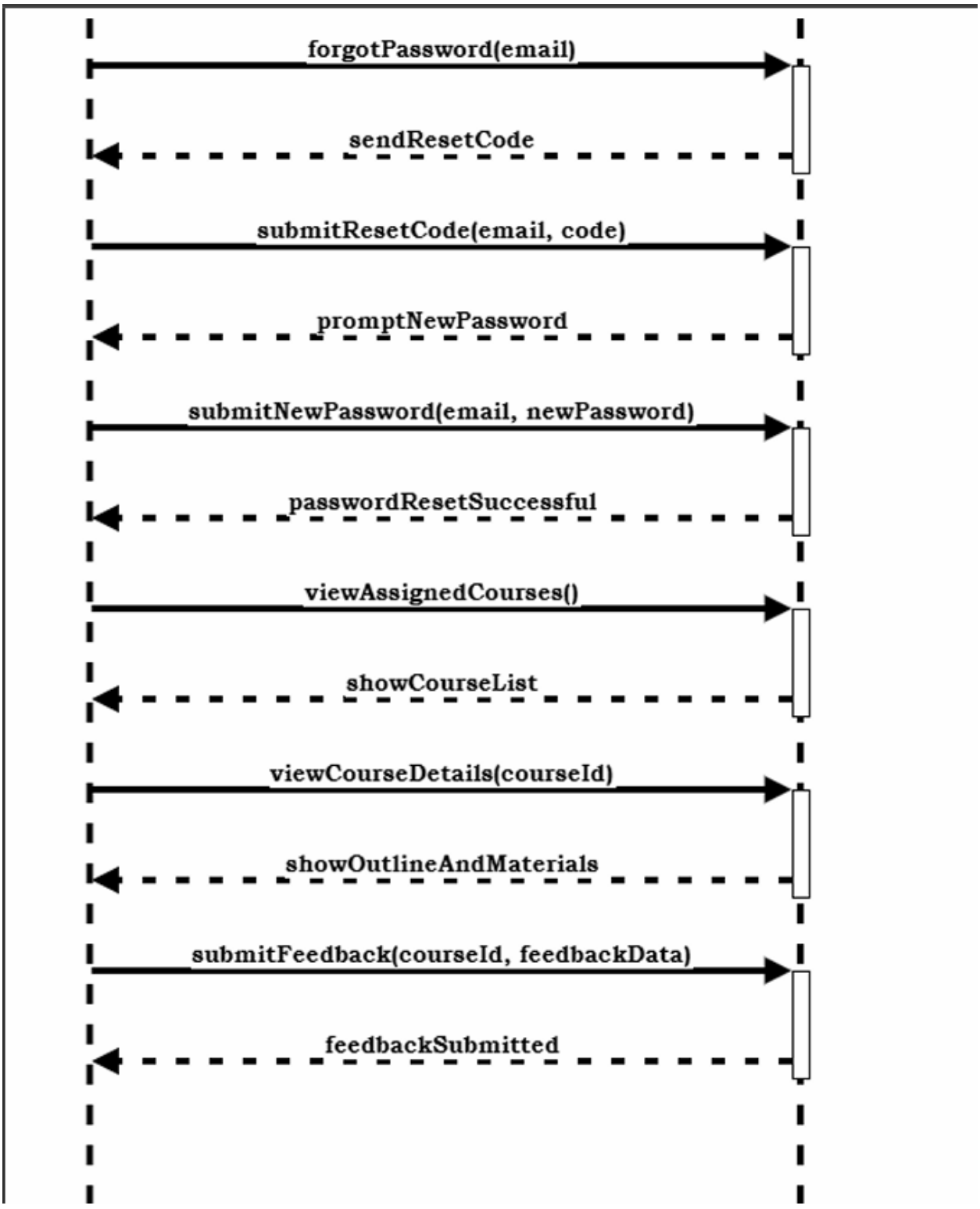
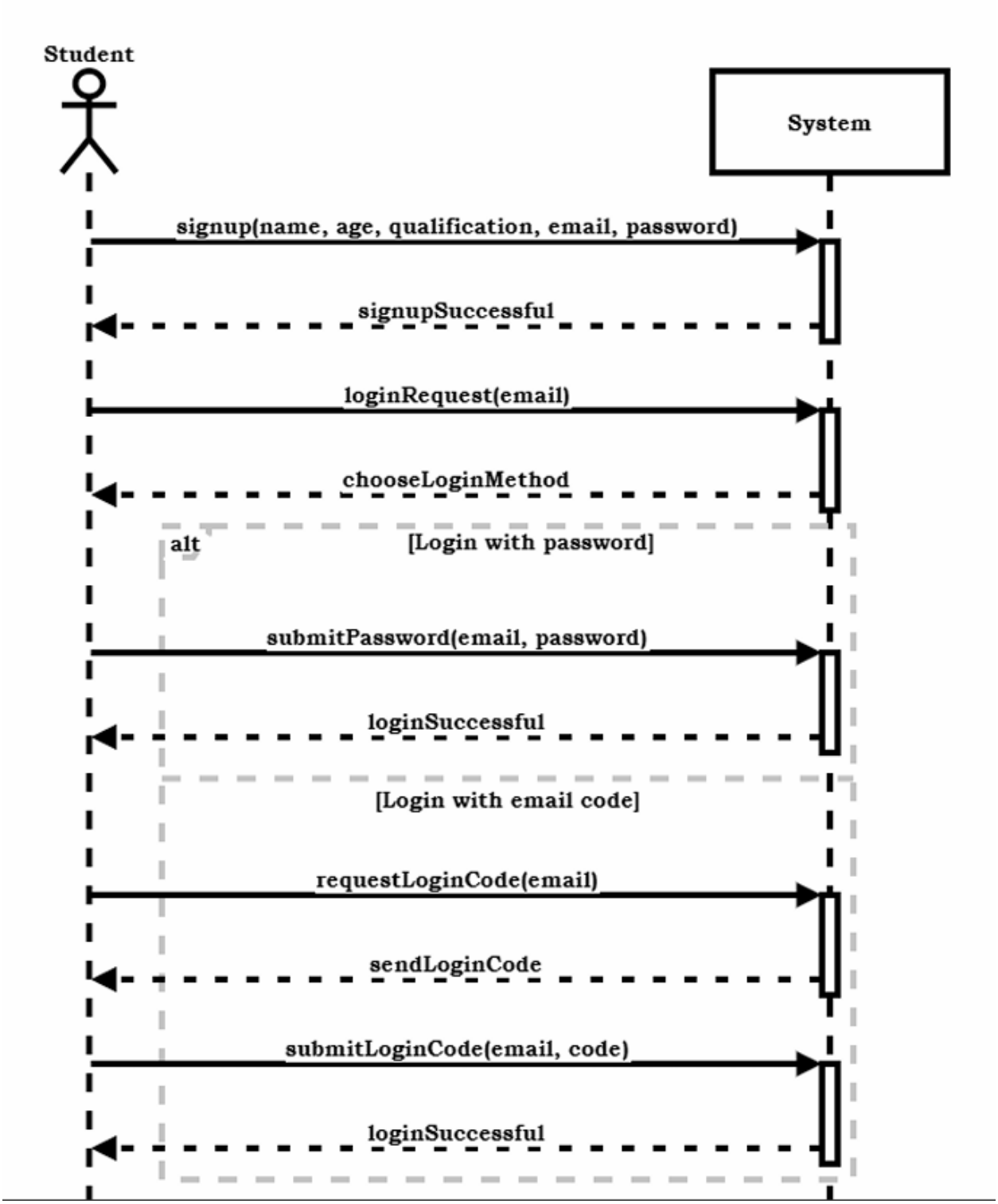
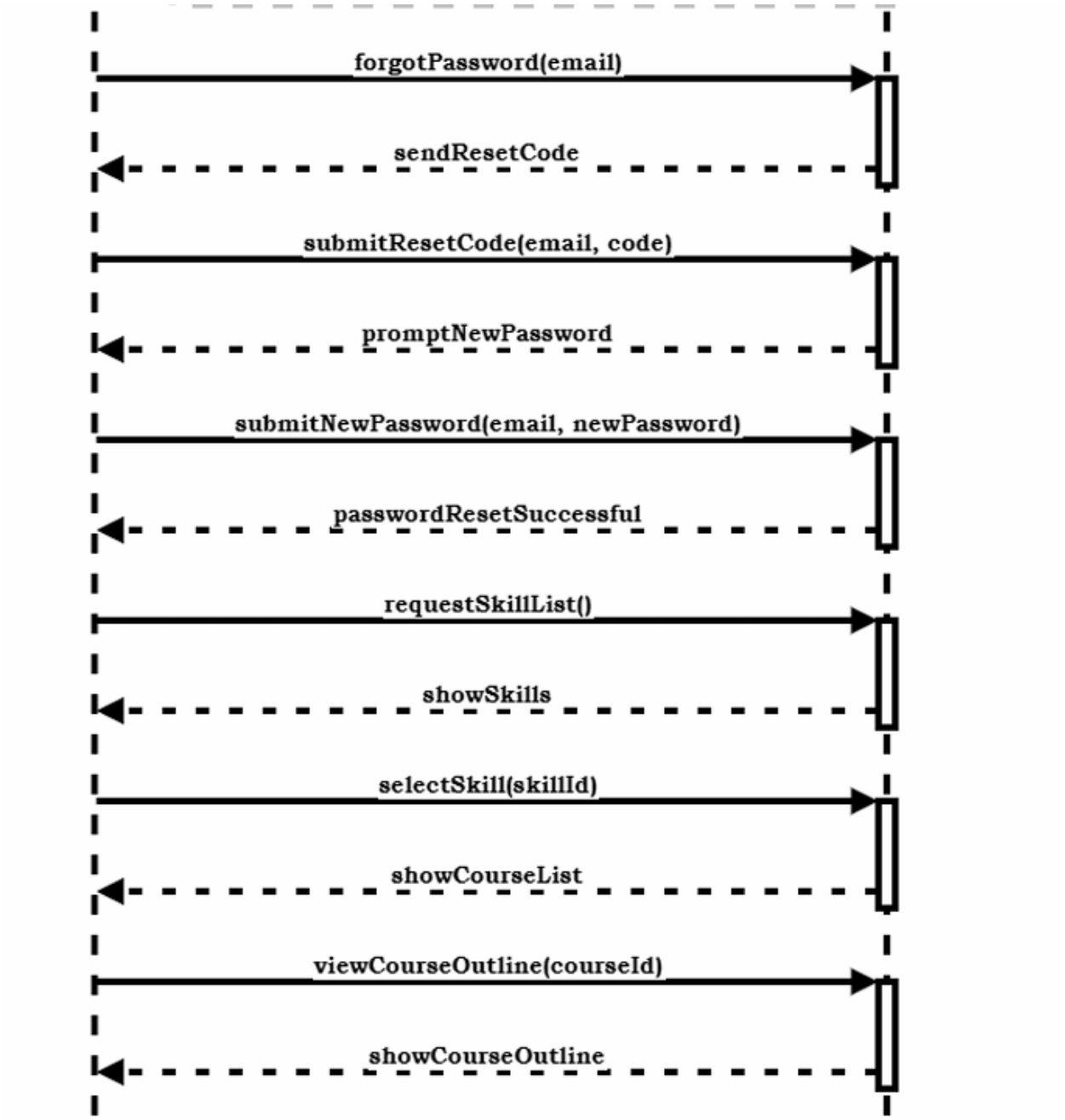
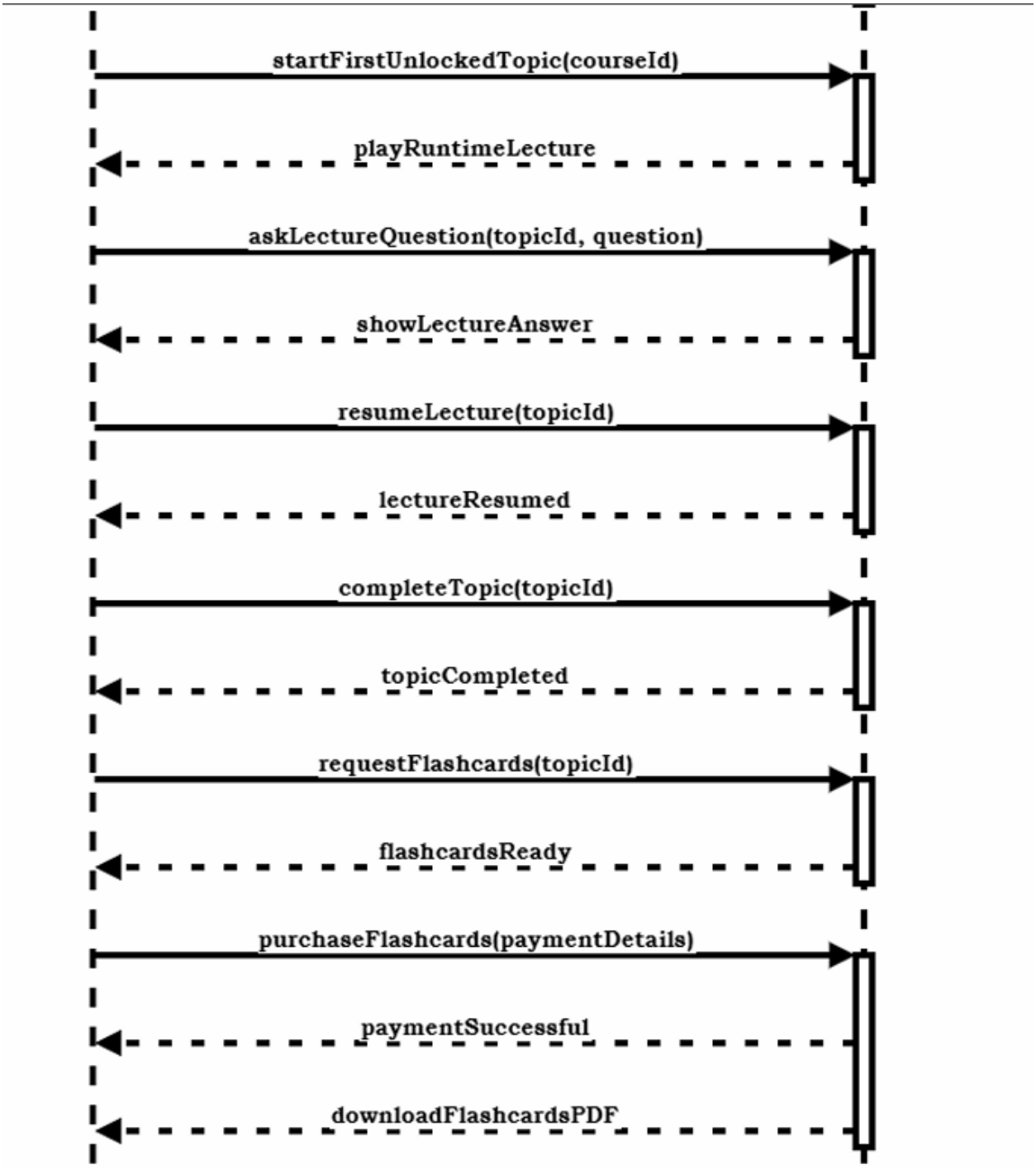


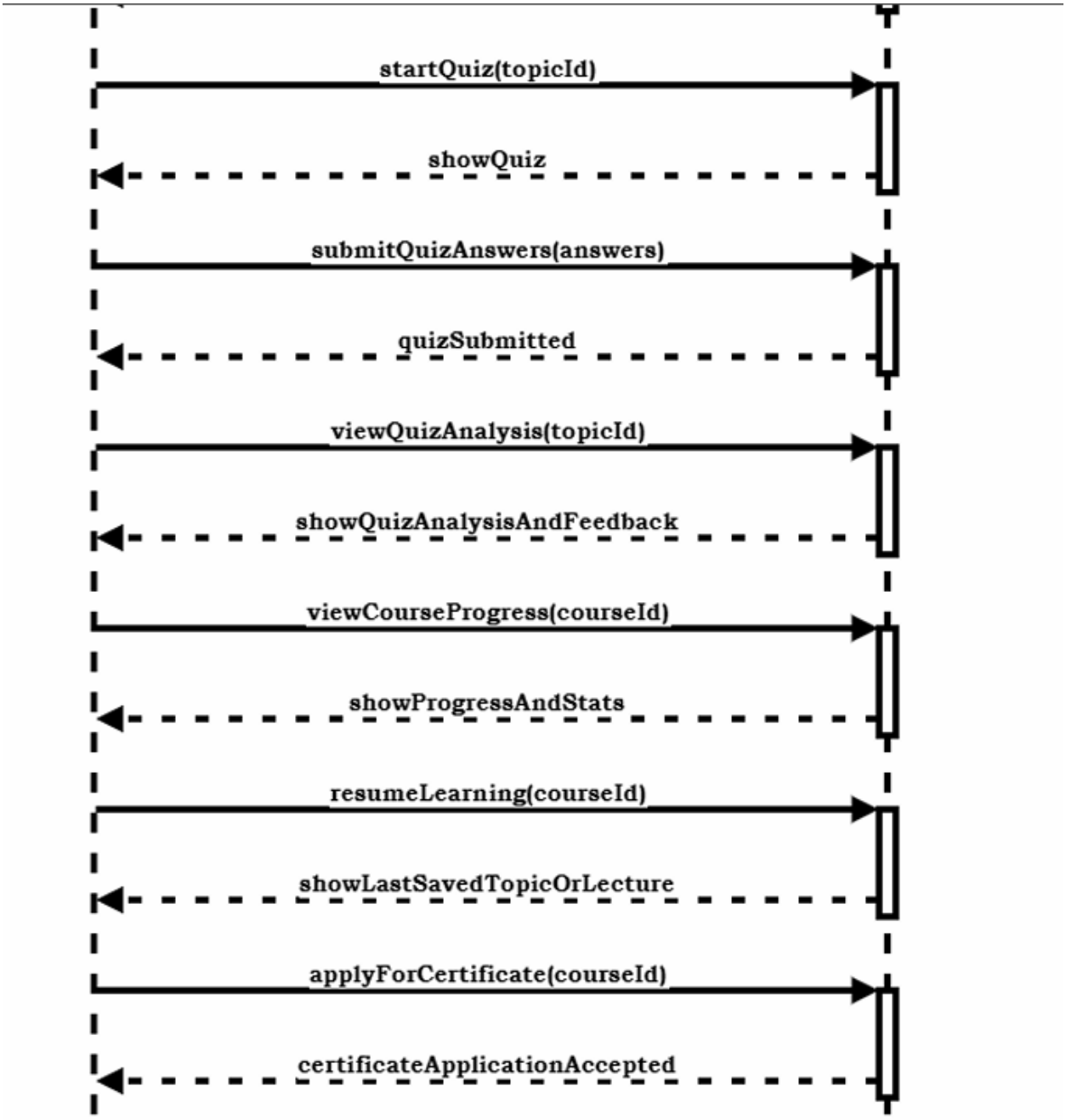

*Figure 7: Expert SSD Diagram*

### 2.5.3. Student SSD




forgotPassword(email)
sendResetCode
submitResetCode(email, code)
promptNewPassword
submitNewPassword(email, newPassword)
passwordResetSuccessful
requestSkillList()
showSkills
selectSkill(skillId)
showCourseList
viewCourseOutline(courseId)
showCourseOutline


startFirstUnlockedTopic(courseId)
playRuntimeLecture
askLectureQuestion(topicId, question)
showLectureAnswer
resumeLecture(topicId)
lectureResumed
completeTopic(topicId)
topicCompleted
requestFlashcards(topicId)
flashcardsReady
purchaseFlashcards(paymentDetails)
paymentSuccessful
downloadFlashcardsPDF


startQuiz(topicId)
showQuiz
submitQuizAnswers(answers)
quizSubmitted
viewQuizAnalysis(topicId)
showQuizAnalysisAndFeedback
viewCourseProgress(courseId)
showProgressAndStats
resumeLearning(courseId)
showLastSavedTopicOrLecture
applyForCertificate(courseId)
certificateApplicationAccepted

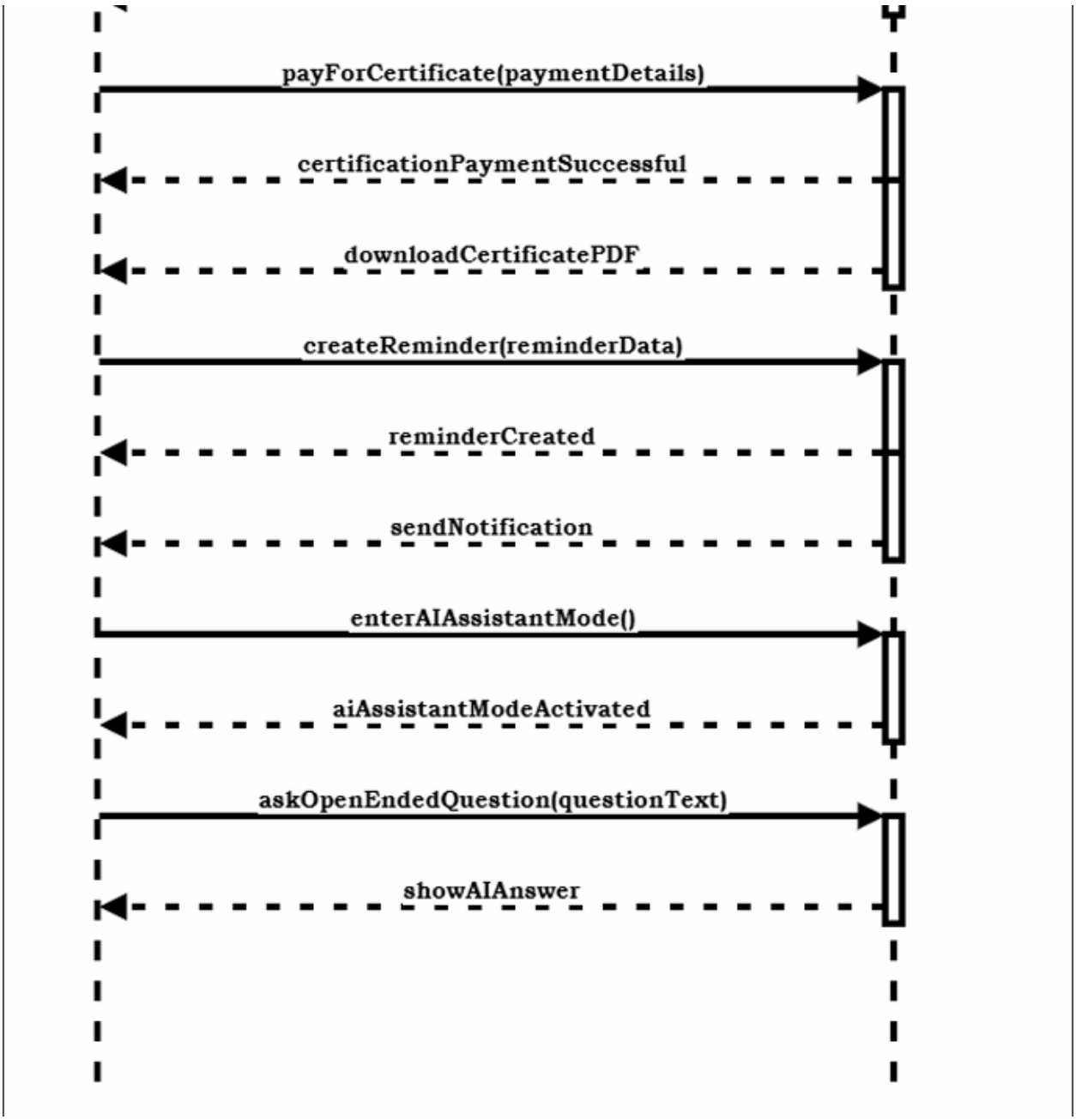



*Figure 8: Student SSD Diagram*

## 2.6. Domain Model

The basic concepts of the domain are represented by key entities such as **Users, Courses, Skill Categories, Topics, Materials, Enrollments, Quizzes, Flashcards, Topic Progress, Payments, Certificates, Feedback, Notifications, AI Chats, and Reminders**, as shown in the following domain model. These interconnected entities describe how users interact with learning content, track progress, complete assessments, receive system-generated support, and manage certifications within the platform.

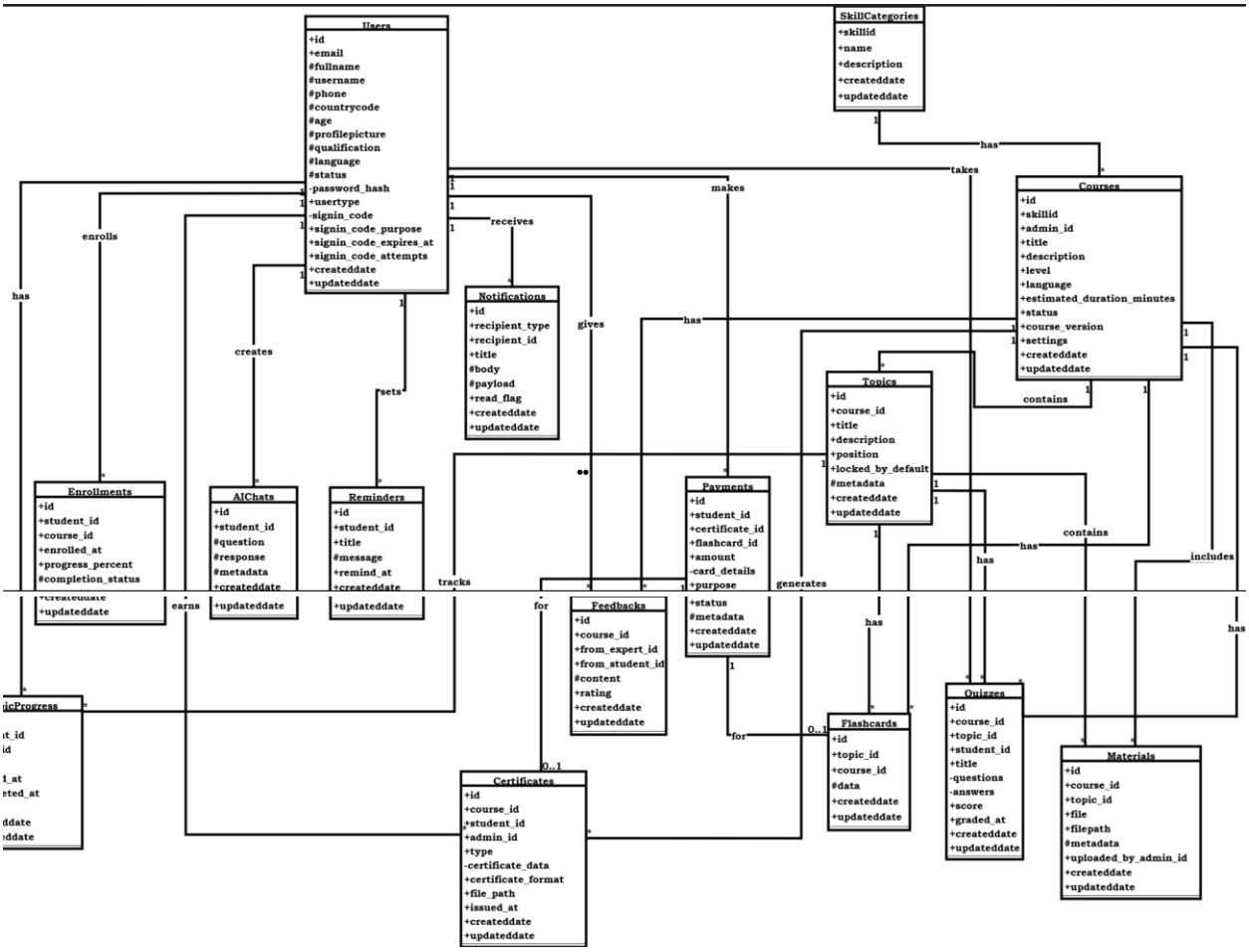



*Figure 9: Domain Model*

# *Chapter 3: System Design*

## 3.1. Layer Definition

*Table 30: Layer Definition*

| Layers | Description |
|---|---|
| Presentation Layer | This layer will be used for the interaction with the user through a graphical user interface. |
| Business Logic Layer | This layer contains the business logic. All the constraints and majority of the functions reside under this layer. |
| Database Layer | This layer contains the database of the application being developed. |

### 3.1.1. Presentation Layer:

Occupies the top level and displays information related to services available on a website. This tier communicates with other tiers by sending results to the browser and other tiers in the network.

### 3.1.2. Business Logic Layer:

Application Layer also called the middle tier, logic tier, business logic or logic tier, this tier is pulled from the presentation tier. It controls application functionality by performing detailed processing.

### 3.1.3. Database Layer:

Database layer includes database servers where information is stored and retrieved. Data in this tier is kept independent of application servers or business logic.

## 3.2. Software Architecture

Software architecture is described as the organization or structure of a system, where the system represents a collection of components that accomplish a specific function or set of functions. Below is the architecture diagram of the system:

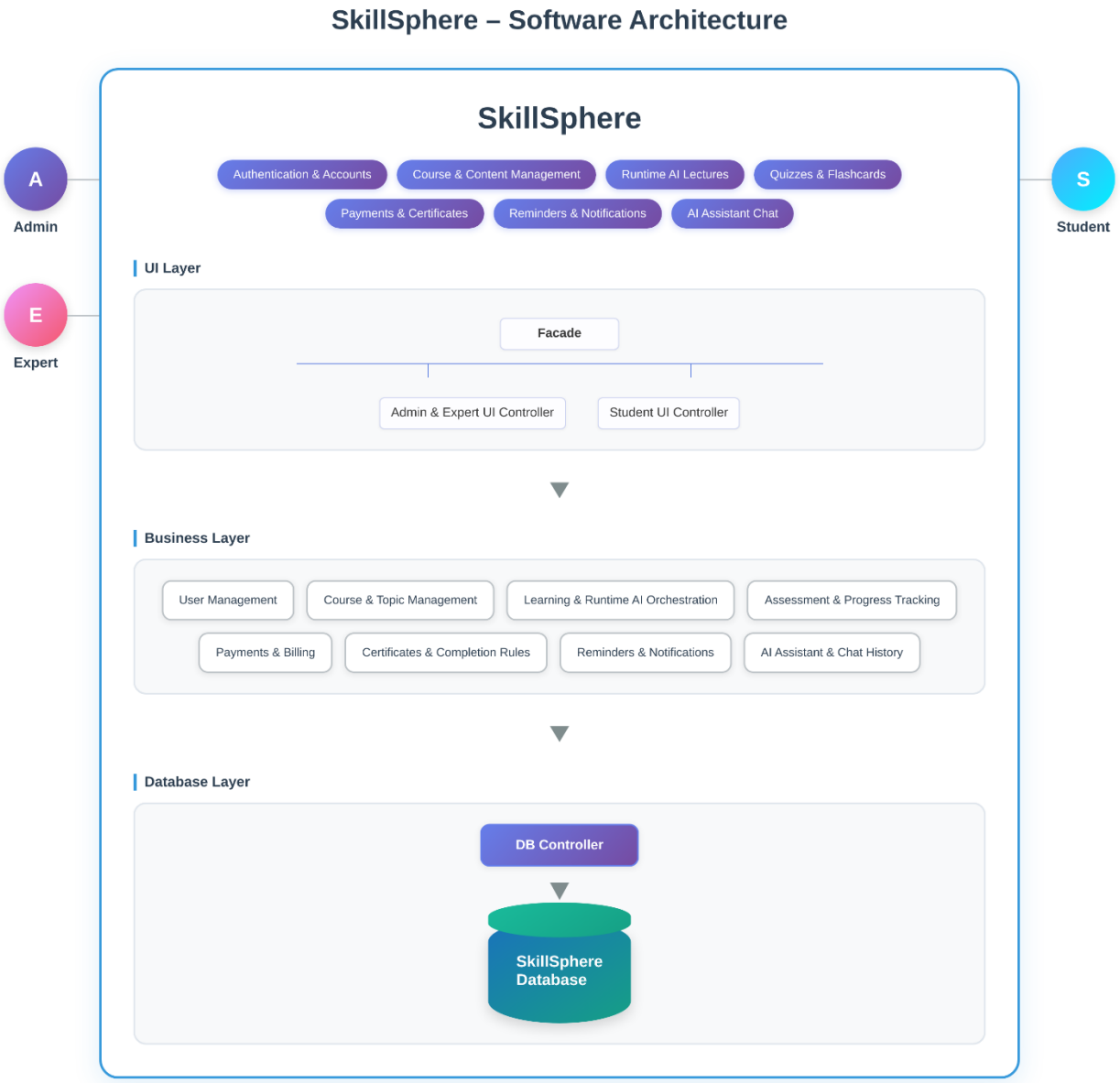



*Figure 10: Software Architecture*

## 3.3. Class Diagram

Class diagrams illustrate the attributes, operations, and relationships of the classes within a system, along with the constraints that govern their behavior. They are fundamental to modeling object-oriented systems because they provide a clear blueprint of the system's structure and are the only UML diagrams that can be directly mapped to object-oriented programming languages.

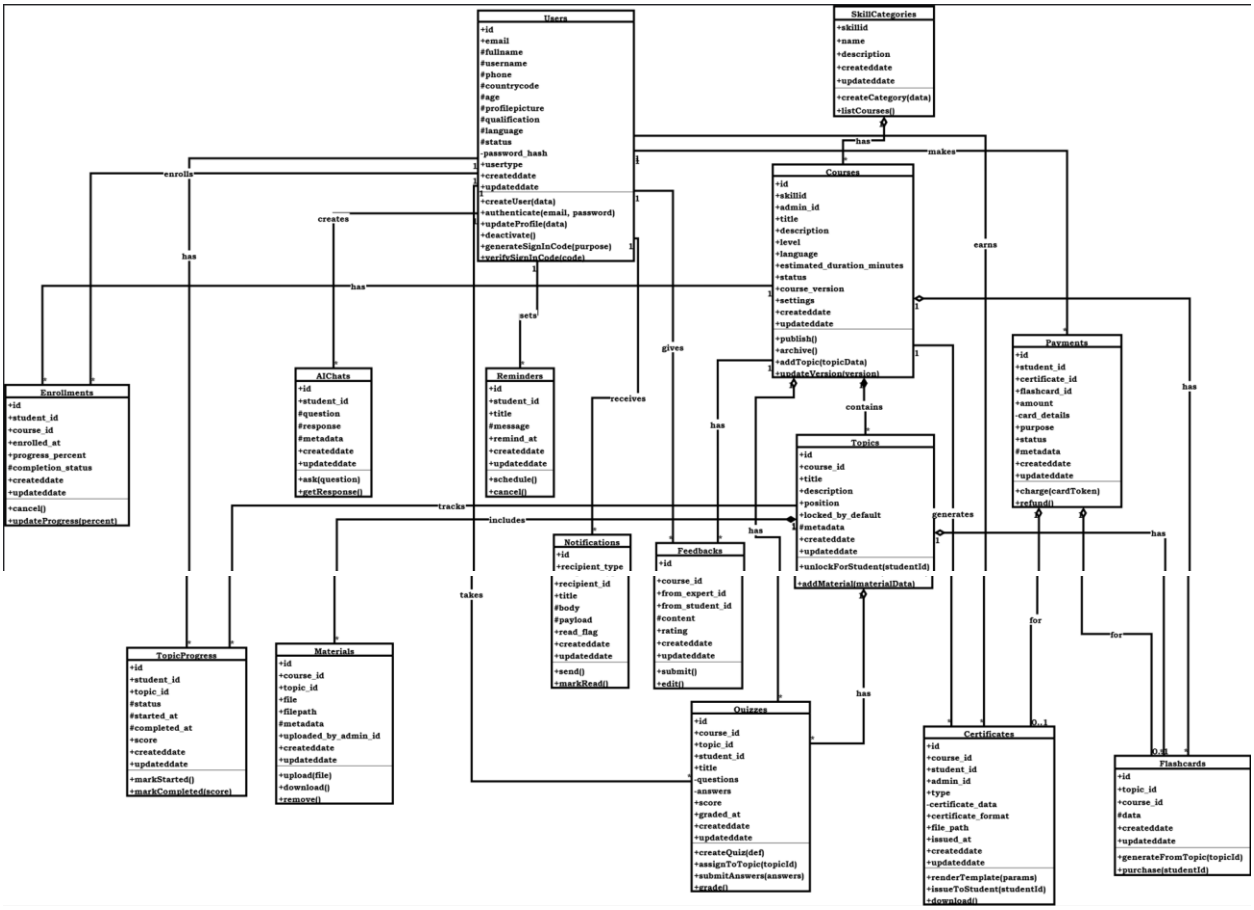



*Figure 11:UML Class Diagram*

### Users

The *Users* class stores essential information about every user in the system, including email, fullname, username, phone, and various authentication attributes such as OTP, password hash, verification status, and signup codes. Users can sign in, sign out, update their profile, and verify their account. This class plays a central role in the system, maintaining both personal and security-

related data and connecting to other major components such as courses, enrollments, payments, AI chats, and reminders.

### SkillCategories

The *SkillCategories* class represents different skill groups used to categorize courses. It contains attributes such as skillId, name, and description, along with created and updated timestamps. A SkillCategory can be connected to multiple courses and helps organize learning content according to learner interest areas.

### Courses

The *Courses* class stores course-related data including the title, description, level, language, duration, versioning, and Instructor responsible for creating the course. It also keeps ratings and status flags. Courses aggregate related components such as topics, materials, quizzes, enrollments, and certificates. This class represents the core learning unit of the platform.

### Topics

The *Topics* class contains detailed learning units within a course. It stores the topic title, description, its sequence position, status flags, and timestamps. Topics are linked to materials, flashcards, quizzes, and track student progress. They form the structural breakdown of each course.

### Materials

The *Materials* class stores all learning materials associated with a topic or course, including file name, file type, and file path. It also captures metadata like who uploaded the material and when it was created or updated. Materials give learners access to videos, documents, slides, and other instructional content.

### Flashcards

The *Flashcards* class includes topic-based learning flashcards used for practice and revision. It stores flashcard data and timestamps. Flashcards offer quick, focused learning interactions and are directly associated with specific topics to support memory retention.

### Quizzes

The *Quizzes* class represents assessments linked to topics or courses. It stores quiz titles, questions, answers, scores, grading details, and timestamps. Quizzes help track learner understanding and are connected to student progress and course structure.

### TopicProgress

The *TopicProgress* class tracks each student's progress for every topic. It stores status, scores, start and completion timestamps, and other progress indicators. This class ensures detailed monitoring of learning activities at the topic level.

### Enrollments

The *Enrollments* class records which students are enrolled in which courses. It includes enrolled dates, progress percentage, completion status, and timestamps. Enrollments act as the linking point between users and courses, enabling the system to track learning progress.

### Payments

The *Payments* class stores all payment information for course enrollments. It includes studentId, courseId, amount, transaction details, status, and metadata. Payments ensure secure and organized handling of financial activities within the platform.

### Certificates

The *Certificates* class stores certification information issued to students upon completing a course. It contains certificate data, format, file path, verification codes, and timestamps. Certificates validate the student's learning achievements and provide verifiable proof of completion.

### Feedback

The *Feedback* class allows students and experts to provide course-related feedback. It contains content, ratings, and references to the course and users involved. Feedback helps improve course quality and learner satisfaction.

### Notifications

The *Notifications* class stores messages sent to users, including the recipient type, message body, payload, and read status. It helps manage all system-generated alerts and communication.

### AIChats

The *AIChats* class records interactions between the user and the AI system. It stores queries, responses, and timestamps. This class supports personalized learning assistance and automated help services.

### Reminders

The *Reminders* class manages automated reminders for users, such as deadlines or learning prompts. It contains reminder text, time, and status, helping students stay on track with their learning journey.

## 3.4. Sequence Diagram

Sequence diagrams visually model the flow of logic within a system, allowing you to clearly document and validate the interactions between components. They are widely used in both analysis and design phases to understand, refine, and communicate system behavior.

### 3.4.1. Instructor SD

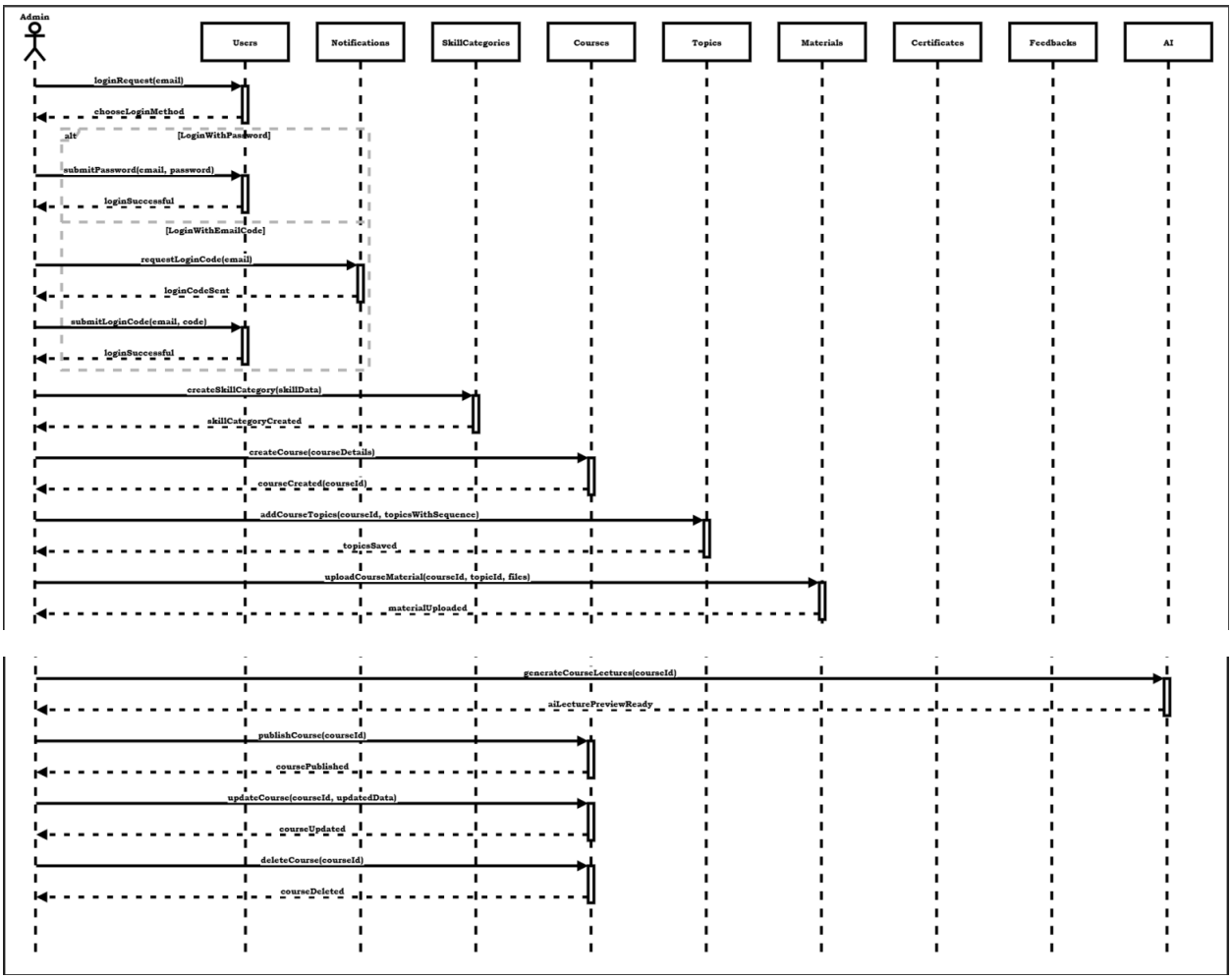



*Figure 12:Instructor SD Diagram*

### 3.4.2. Expert SD

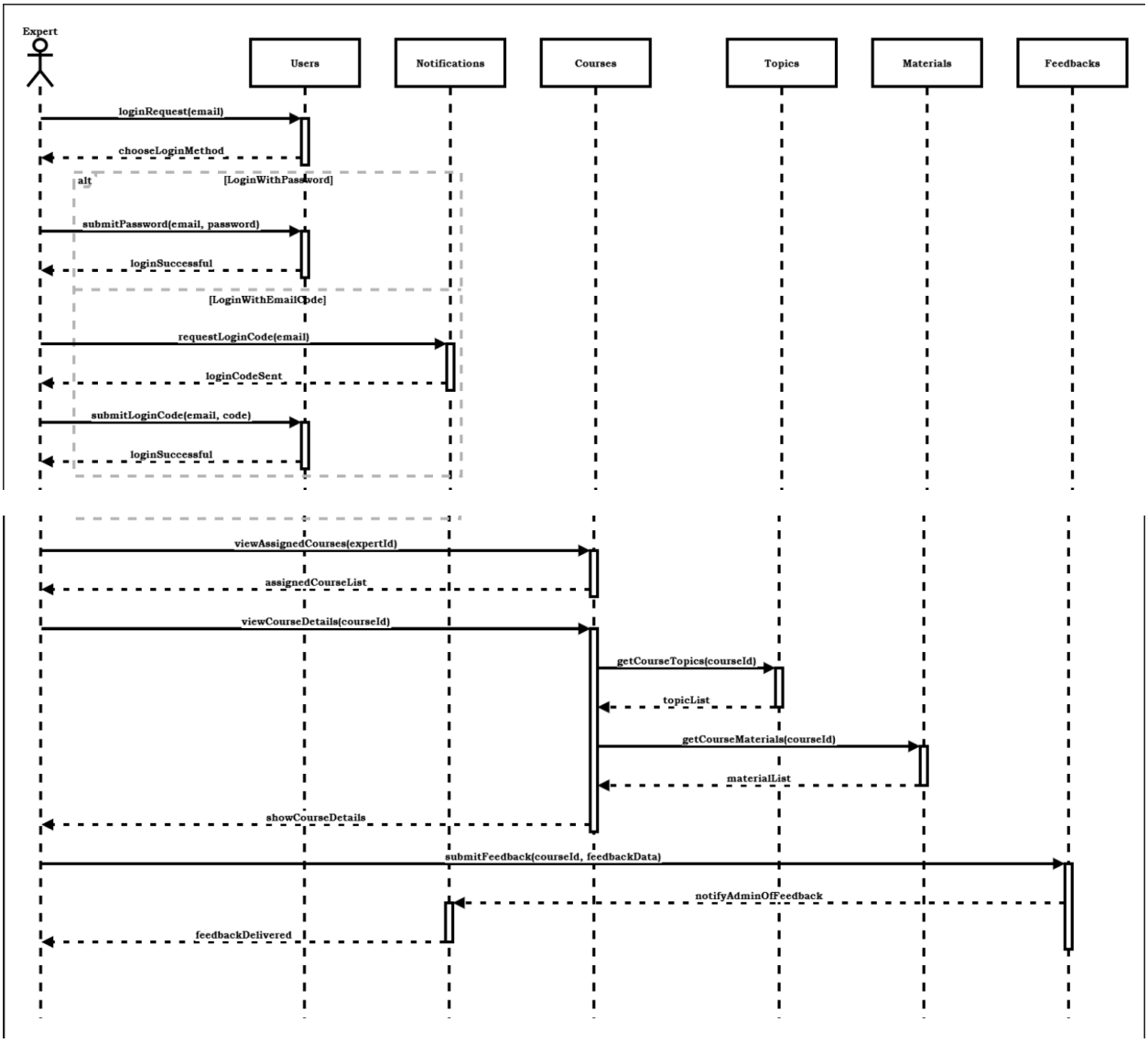



*Figure 13: Expert SD Diagram*

### 3.4.3. Student SD

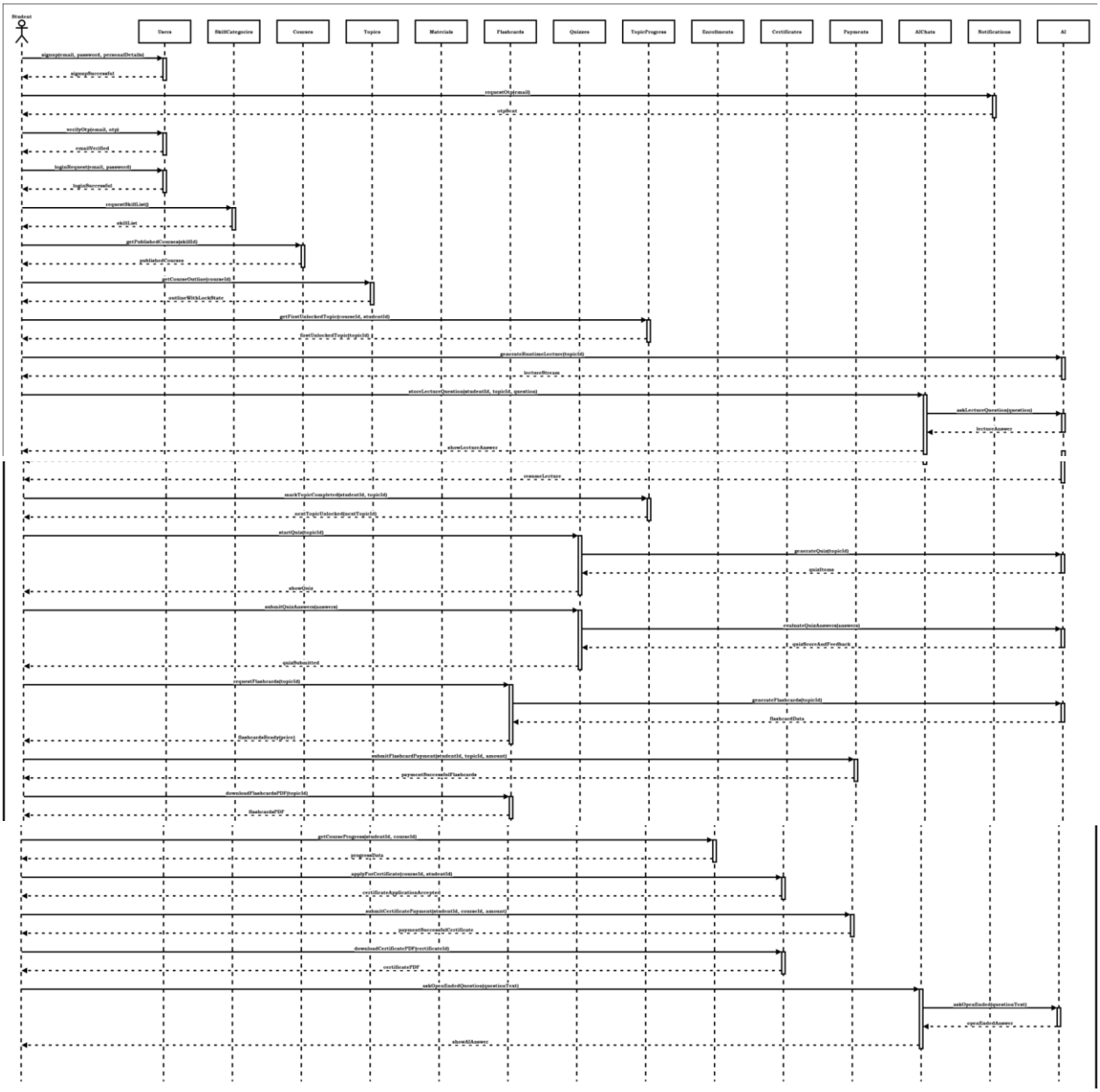
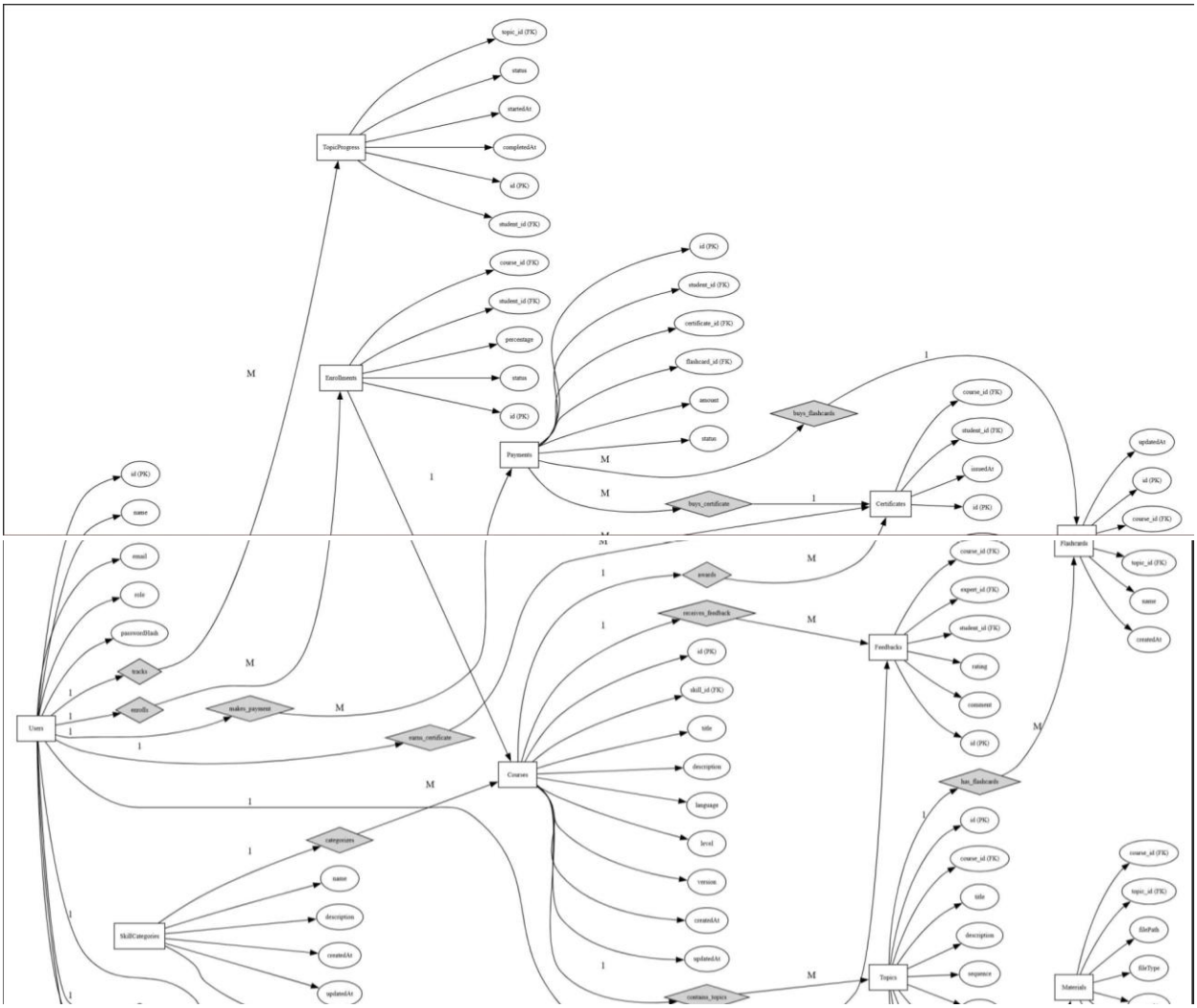



*Figure 14: Student SD Diagram*

## 3.5. Entity Relationship (ER) Diagram

The entity-relationship (ER) model is a graphical approach used to represent the logical relationships among entities within a database system. It describes the interrelated objects of interest in a specific domain and defines how these objects interact with one another. An ER model is composed of entity types, which categorize the key elements of the domain, and the relationships that connect instances of these entities. This model serves as a foundational tool for designing clear, structured, and efficient databases.



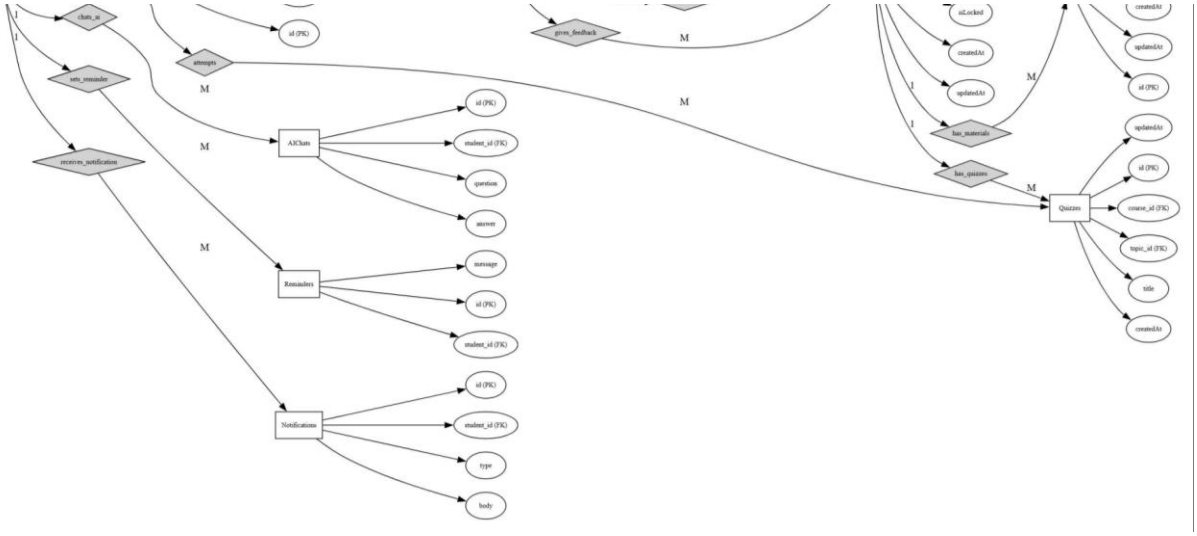



*Figure 15: ER Diagram*

## 3.6. Database Schema

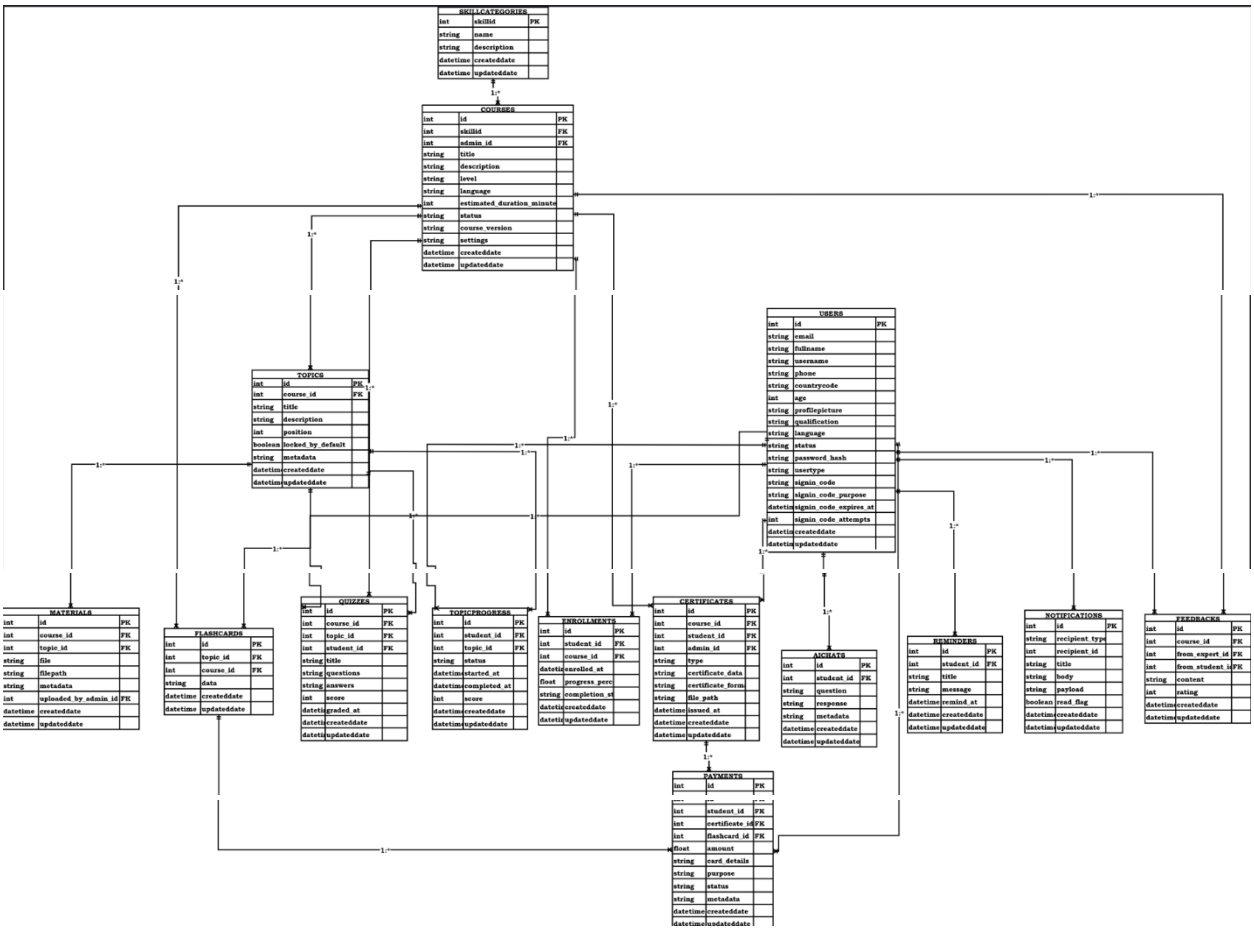



*Figure 16: Database Schema Diagram*

## 3.7 User Interface Design

User Interface (UI) Design focuses on anticipating what users might need to do and ensuring that the interface has elements that are easy to access, understand, and use to facilitate those actions. UI brings together concepts from interaction design, visual design, and information architecture. SkillSphere implements a responsive design using React Native that adapts seamlessly across mobile, tablet, and desktop platforms.

## Landing Screen:

The Landing Screen is the first screen users encounter when opening the SkillSphere application. This screen introduces the platform with the SkillSphere brand logo displayed prominently at the top along with a welcome message highlighting the learning opportunities available. Users can navigate to the Sign-Up screen by clicking the "Get Started" button, or existing users can click the "Login" button to access their accounts. There is also an option to explore available courses before creating an account.

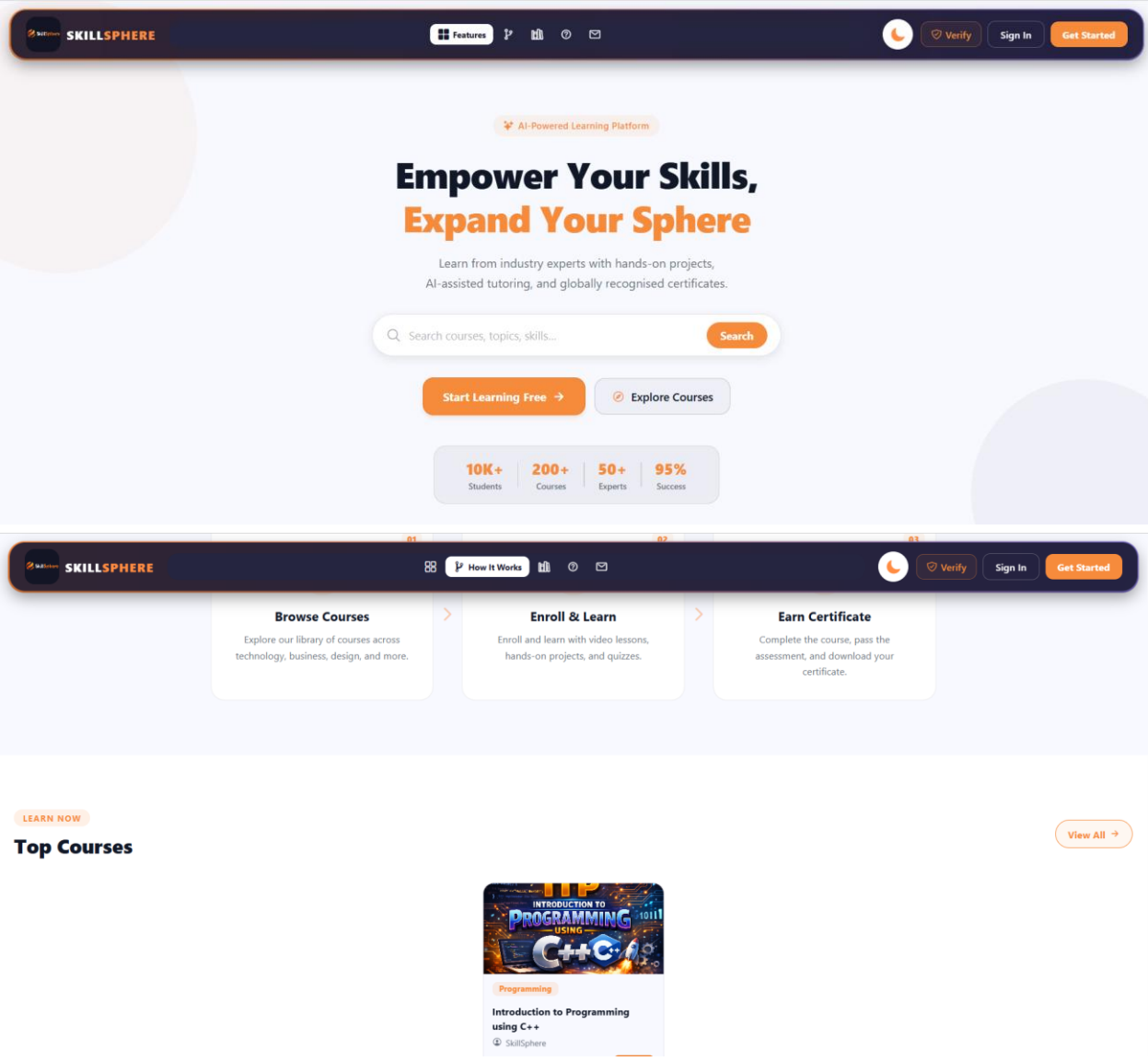



*Figure 17: Landing Screen GUI*

## Phone and Web Mode:

SkillSphere is built using React Native with web support, enabling the application to run seamlessly on both mobile devices and web browsers. The Phone mode provides a touch-optimized interface with mobile-friendly navigation, collapsible menus, and gesture-based interactions suitable for Android smartphones. The Web mode offers an expanded desktop layout with a persistent sidebar navigation, larger content areas, and hover-based interactions optimized for mouse and keyboard usage. The application automatically detects the platform and adjusts the layout, spacing, and component sizes to deliver an optimal user experience on each device.

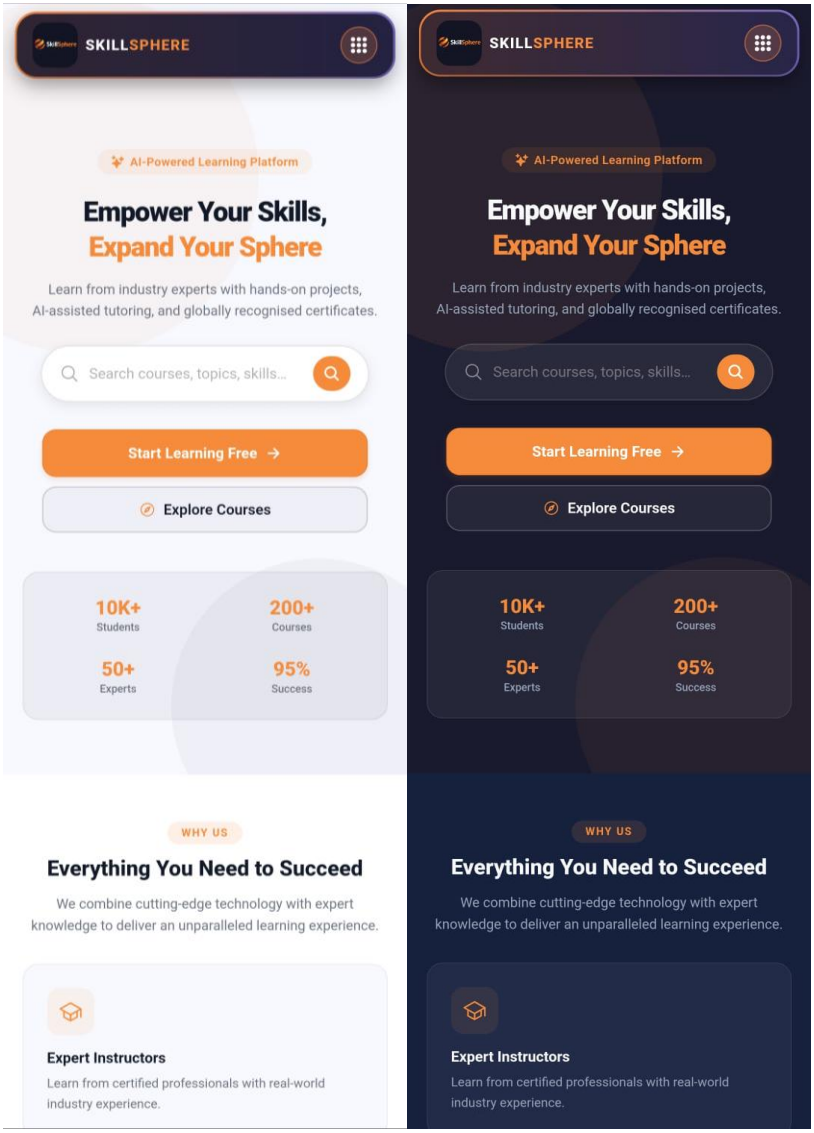



*Figure 18: Phone Mode GUI*

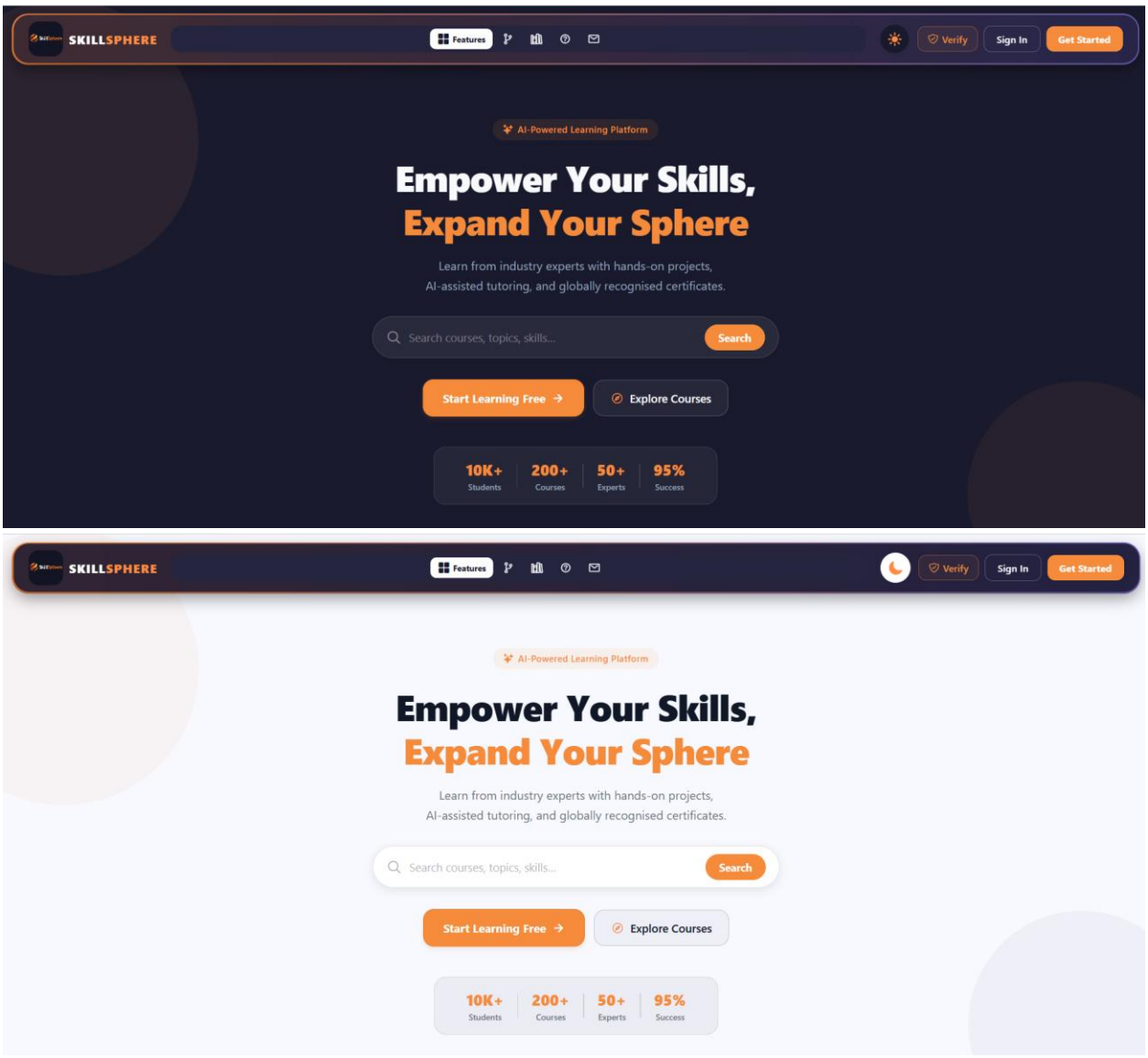



*Figure 19: Web Mode GUI*

## Login Screen:

The Login Screen allows returning users to securely access their accounts. Users must enter their registered email address in the "Email" field and their password in the "Password" field which includes a show/hide toggle for convenience. The screen supports two authentication modes - traditional password login and OTP-based login which can be switched using a toggle button. Users can also sign in using their Google account by clicking the "Sign in with Google" button. Additional links are provided for password recovery and new user registration.

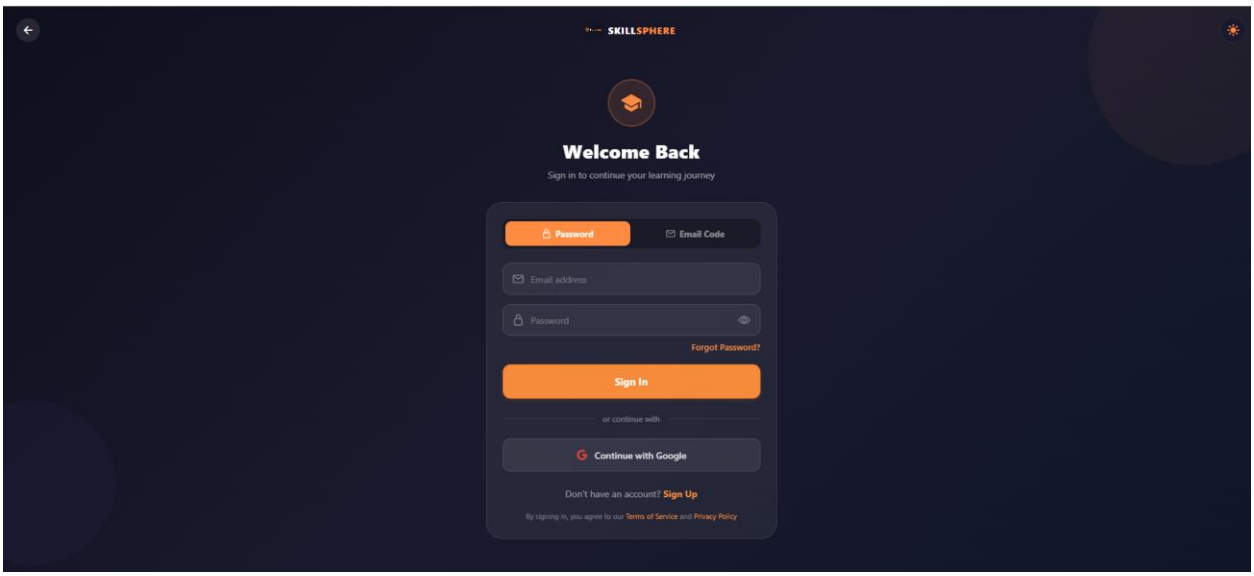



*Figure 20: Login Screen GUI*

## Sign Up Screen:

For first time users, the Sign-Up screen allows them to create a new SkillSphere account through a simple multi-step process. In the first step, users enter their full name and email address, then click "Continue" to proceed. The system sends a 6-digit OTP to the provided email which the user must enter for verification. After successful verification, users create their password, confirm it, and select their role as either Student or Expert. Users can also quickly register using their Google account by clicking the "Sign up with Google" button.

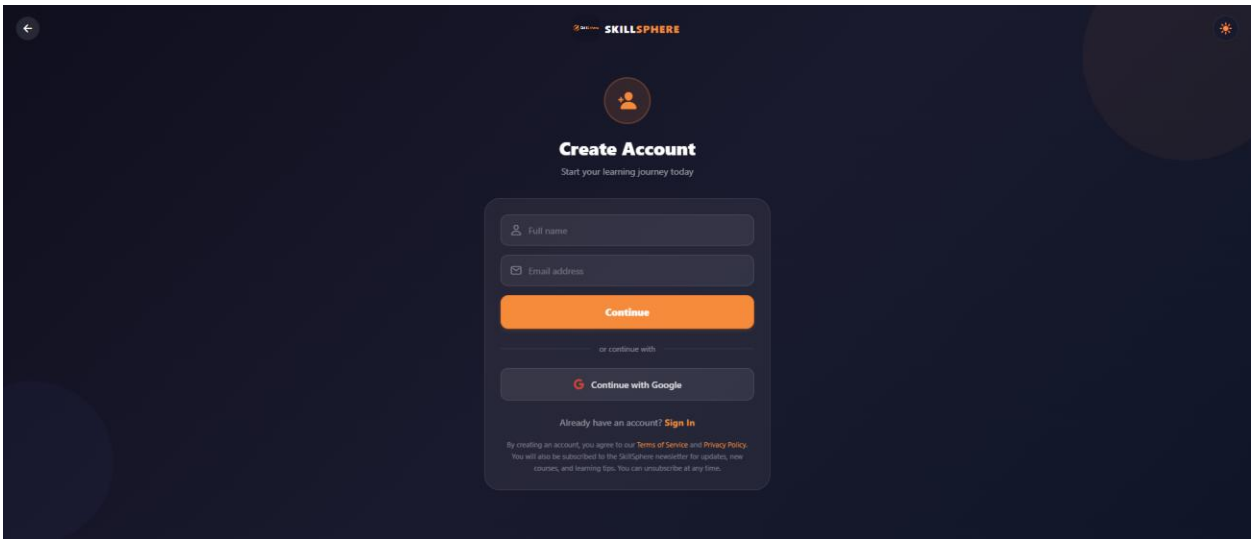



*Figure 21: Sign Up Screen GUI*

## OTP Verification Screen:

The OTP Verification Screen is displayed after users submit their email during signup or when using OTP-based login. The screen presents six individual input fields for entering the 6-digit verification code sent to the user's email address. The input fields feature auto-focus functionality, automatically moving to the next field as each digit is entered for a seamless user experience. A countdown timer displays the remaining time before users can request a new OTP code. Below the input fields, a "Resend OTP" button becomes active after the timer expires, allowing users to request a fresh verification code if needed.

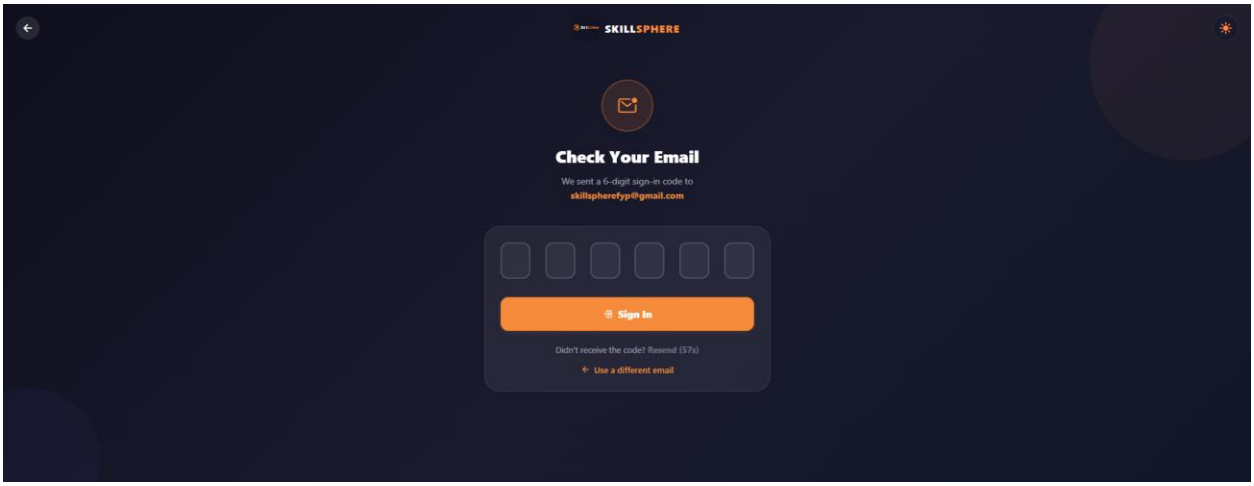



*Figure 22: OTP Verification Screen GUI*

## Student Dashboard:

After successful login, students are directed to the Student Dashboard which serves as the main hub for their learning journey. The dashboard displays a personalized welcome message with the student's name and shows statistics cards indicating total enrolled courses, completed courses, and certificates earned. A progress chart visualizes the student's weekly learning activity. The screen also features recommended courses, recent learning activity timeline, and quick action buttons for browsing courses, accessing enrolled courses, viewing certificates, and using the AI Tutor. A sidebar navigation menu provides access to all platform features including Courses, Categories, Certificates, Todo List, and Settings.

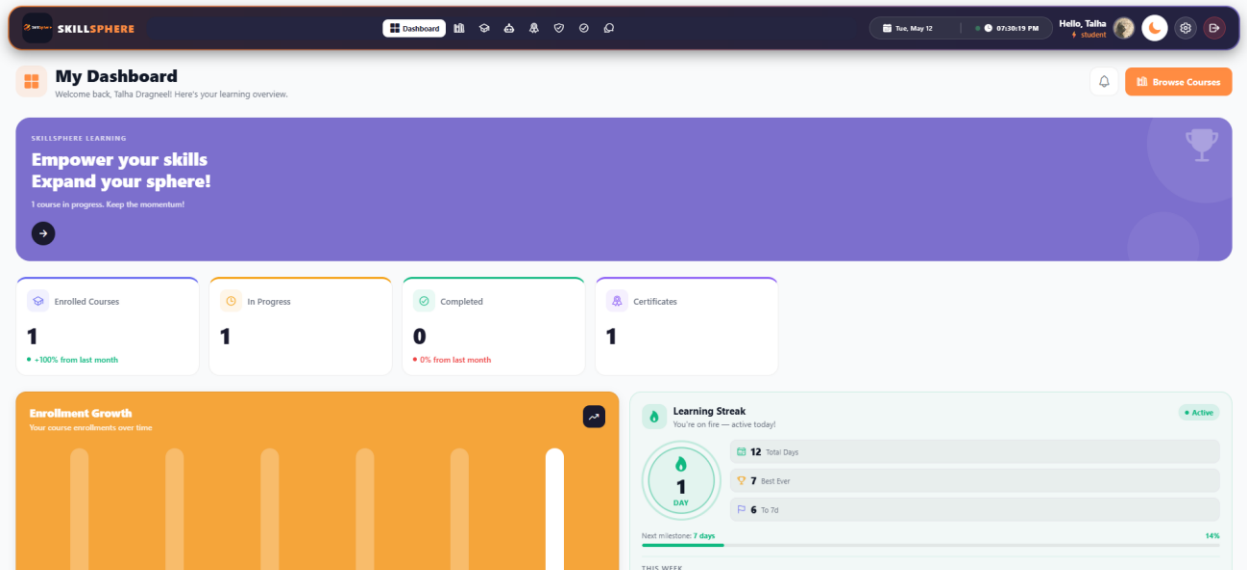



*Figure 23: Student Dashboard GUI*

## Instructor Dashboard:

The Instructor Dashboard provides Instructoristrators with a comprehensive control panel for managing the SkillSphere platform. The screen displays four main statistics cards showing Total Students, Total Courses, Active Enrollments, and Certificates Issued. Below the statistics, bar charts display analytics for user registrations and course enrollments over time. A recent activities panel shows the latest platform updates including new student enrollments and course additions. The sidebar navigation allows Instructoristrators to access course management, student management, category management, certificate issuance, user feedback, and user account management for creating new Instructor or expert accounts.

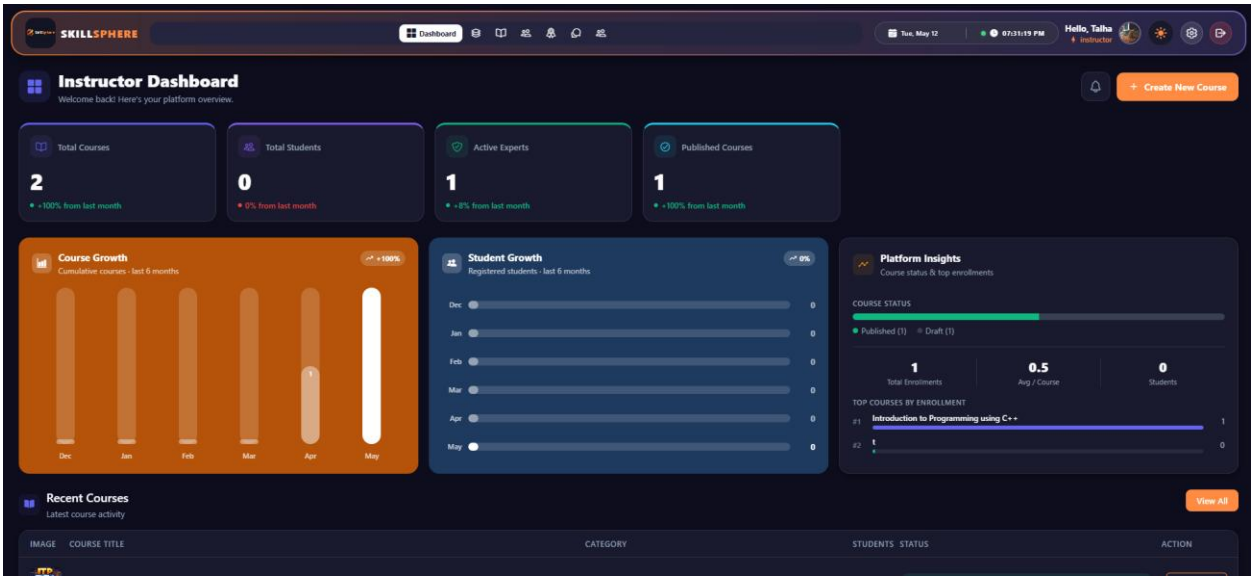


*Figure 24: Instructor Dashboard GUI*

## Expert Dashboard:

The Expert Dashboard is designed for instructors and subject matter experts who create and manage courses on the platform. The dashboard displays key performance metrics including total courses created, number of enrolled students, and average course ratings. A feedback summary section shows recent student reviews and ratings for the expert's courses. The screen includes quick access buttons to create new courses, view existing courses, and respond to student feedback. The sidebar navigation provides access to course management, student enrollments, feedback forms, and account settings.

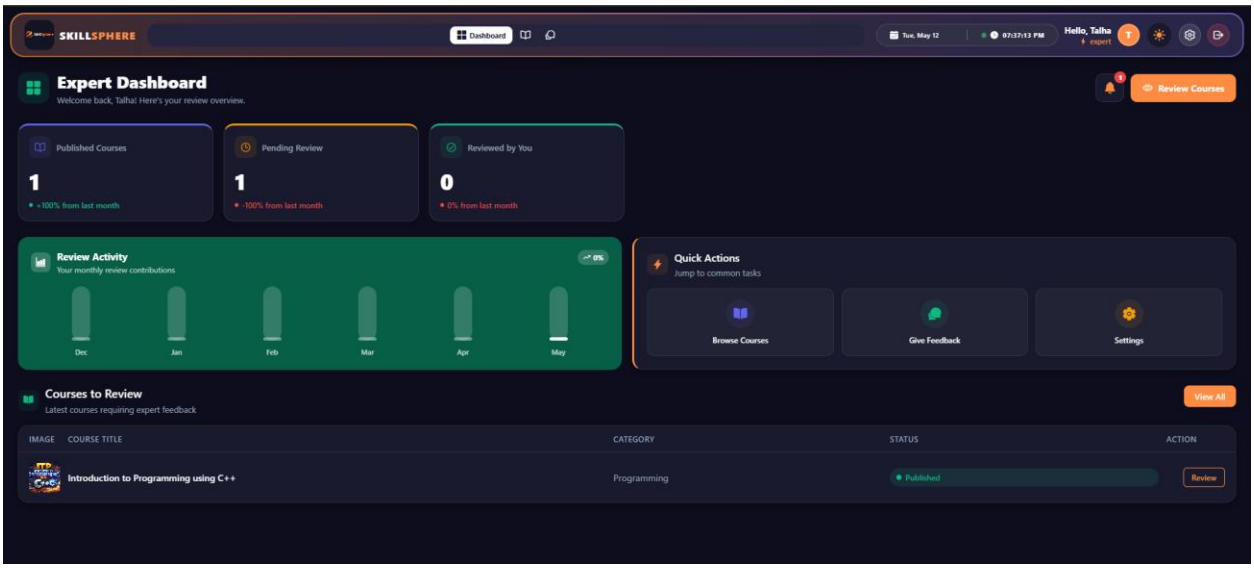



*Figure 25: Expert Dashboard GUI*

## Admin Dashboard:

The AdminDashboard provides the highest level of Instructoristrative control over the entire SkillSphere platform. This dashboard includes all the features of the Instructor Dashboard along with additional system-wide management capabilities. The statistics section displays platform-wide metrics including total Instructors, total experts, total students, and overall course performance. The Admincan create and manage Instructor accounts, expert accounts, and has full access to all platform data and settings. The sidebar navigation includes additional options for managing Instructoristrators, managing experts, viewing system logs, and configuring platform-wide settings.

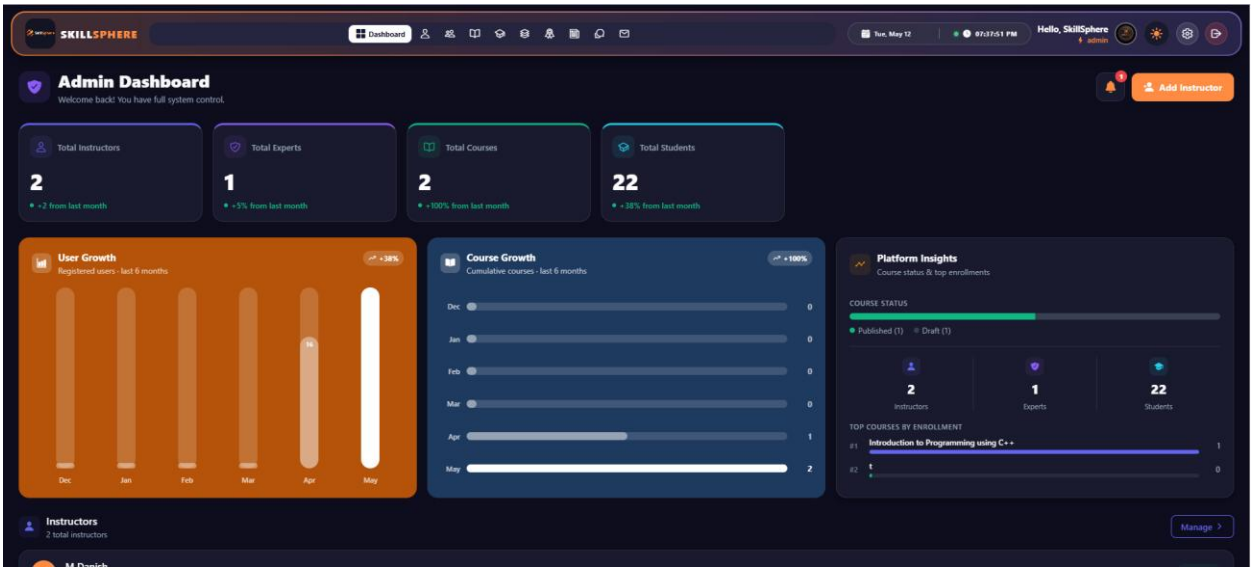



*Figure 26: Admin Dashboard GUI*

## Light and Dark Mode:

SkillSphere provides users with the flexibility to switch between Light and Dark mode themes based on their personal preference and viewing environment. The Light mode features a clean white background with dark text, providing optimal readability in well-lit environments. The Dark mode offers a darker color scheme with lighter text, reducing eye strain during nighttime usage and conserving battery on devices with OLED screens. Users can easily toggle between these modes using the theme switch button available in the application settings or navigation bar.

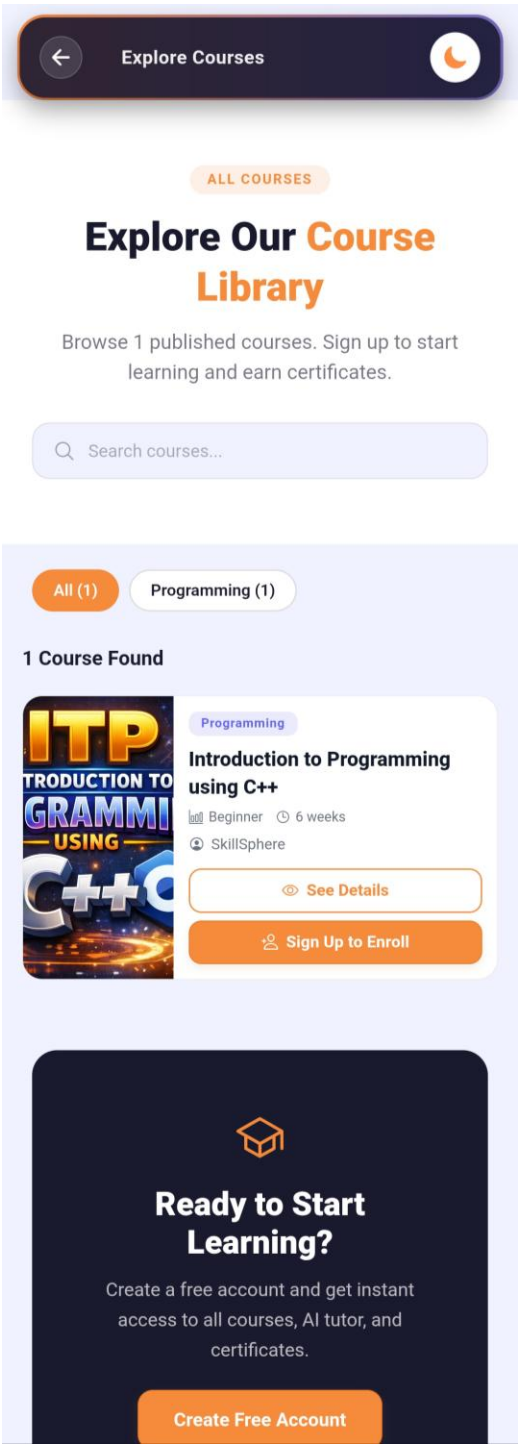



*Figure 27: Light Mode GUI*


*Figure 28: Dark Mode GUI*

# *Chapter 4: Software Development*

This chapter provides the details about the coding standards we adopted during the implementation phase of SkillSphere - an AI-powered learning management system built with React Native and Node.js.

## 4.1. Coding Standards

The adopted coding standards are discussed in the following subsections.

### 4.1.1. Indentation

Two spaces are used as the unit of indentation for all JavaScript, JSX, and JSON files throughout both the backend (Node.js/Express) and the frontend (React Native) codebases. This is consistent with the widely accepted JavaScript community standard and ESLint default configuration. The indentation pattern is consistently followed throughout.

### 4.1.2. Declaration

- One declaration per line is used to enhance the clarity of code. The order and position of declarations is as follows:
- First, all import/require statements are placed at the top of each file, grouped logically: external library imports first (e.g., `react`, `express`), followed by internal module imports (e.g., models, services, utilities).
- Constants and configuration variables are declared after imports.
- Function declarations and exports follow, grouped by functionality.
- In React components, hooks (`useState`, `useEffect`, `useMemo`) are declared at the top of the component function, followed by derived state, event handler functions, and then the JSX return statement.
- In Express controllers, each exported function handles a single API endpoint, maintaining the single-responsibility principle.
- Sequelize model definitions follow a consistent structure: field definitions first, then options (timestamps, tableName), and finally exports.

### 4.1.3. Statement Standards

Each line contains at most one statement. Compound statements (blocks) that contain lists of statements enclosed in braces are indented one more level than the enclosing statement. The opening brace is placed at the end of the line that begins the compound statement. The closing brace begins a new line and is indented to the level of the compound statement.

Arrow functions are preferred for callbacks and short functions. The `async/await` pattern is used consistently instead of raw Promises with `.then()` chains. All asynchronous controller functions are wrapped in `try/catch` blocks for proper error handling. Template literals (backtick strings) are used for string interpolation instead of concatenation.

### 4.1.4. Naming Convention

Naming conventions make programs more understandable by making them easier to read. The following conventions are followed:

- camelCase is used for variables, functions, and method names (e.g., `fetchCourses`, `enrollInCourse`, `thumbnailImage`).
- PascalCase is used for React components, model names, and class names (e.g., `StudentDashboard`, `CourseDetailScreen`, `CertificateTemplate`).
- UPPER_SNAKE_CASE is used for constants and environment variables (e.g., `JWT\_SECRET`, `LOGO\_PATH`, `UPLOADS\_DIR`).
- kebab-case: is used for file names of utility modules and configuration files (e.g., `url-helpers.js`).
- Full English descriptors that accurately describe the variable, method, or class are used. For example, `enrollmentController`, `certificateService`, `resolveFileUrl` instead of abbreviated or cryptic names.
- Boolean variables are prefixed with `is`, `has`, or `can` for clarity (e.g., `isActive`, `isEnrolled`, `canAccess`, `hasBackgroundImage`).
- API route paths follow RESTful naming conventions using plural nouns (e.g., `/api/courses`, `/api/enrollments`, `/api/certificates`).

## 4.2. Development Environment

### Visual Studio Code

Visual Studio Code (VS Code) is the primary integrated development environment (IDE) used for developing SkillSphere. VS Code is a free, open-source code editor developed by Microsoft, available under the MIT License. It provides built-in support for JavaScript, TypeScript, and Node.js, along with a rich ecosystem of extensions.

The reason for using VS Code is that it provides an interactive and efficient interface for full-stack JavaScript development. Features such as IntelliSense (auto-completion), integrated terminal, Git integration, and React Native debugging tools significantly improved development productivity. Extensions such as ESLint, Prettier, React Native Tools, and REST Client were utilized during development. Few alternatives for VS Code are WebStorm, Sublime Text, Atom, and Android Studio.

### React Native CLI

React Native is an open-source framework created by Meta (Facebook) for building cross-platform mobile applications using JavaScript and React. SkillSphere uses React Native CLI (not Expo) for the mobile frontend, allowing native module access and custom build configurations. The application supports both Android and Web platforms.

### Node.js with Express.js

The backend server is built using Node.js (JavaScript runtime) with Express.js (web framework). Express.js provides a minimal and flexible set of features for building RESTful APIs. The backend handles authentication, course management, enrollment processing, certificate generation, AI chat integration, and file uploads.

### HeidiSQL

HeidiSQL is a free, open-source database management tool for Windows, developed by Ansgar Becker and released under the GNU General Public License (GPL). It provides a graphical interface for managing MySQL, MariaDB, PostgreSQL, Microsoft SQL Server, and SQLite databases.

The reason for using HeidiSQL is that it offers an intuitive and lightweight interface for database Instructoristration and development tasks. Features such as table browsing, query execution, data export/import, database structure visualization, and SQL syntax highlighting significantly streamlined database operations during SkillSphere development.

## 4.3. Database Management System

### MySQL

MySQL is an open-source relational database management system (RDBMS) developed by Oracle Corporation. It is the world's most popular open-source database, known for its reliability, performance, and ease of use. SkillSphere uses MySQL as its primary database to store all application data including user accounts, courses, topics, materials, enrollments, progress tracking, quiz results, certificates, notifications, and AI chat sessions.

The database is accessed through Sequelize ORM (Object-Relational Mapping), which provides an abstraction layer for database operations, model definitions, associations, and migrations.For production deployment, the MySQL database is hosted on Railway's managed MySQL service, accessible via a connection URL. For local development, a local MySQL server running on port 3306 is used.

Alternatives for MySQL include PostgreSQL, MongoDB, MariaDB, and Microsoft SQL Server.Cloudinary Cloudinary is a cloud-based media management platform used for persistent file storage. All uploaded files including course thumbnails, PDF materials, certificate templates, Instructor signatures, and generated certificate PDFs are stored on Cloudinary. This ensures files persist across server restarts and redeployments on Railway's ephemeral filesystem.

## 4.4. Software Description

Main modules of our project are:

- User Authentication and Authorization Module
- Course Management Module
- Enrollment and progress tracking
- Quiz and Assessment Module
- Certificate Generation Module
- AI Chat Assistant Module
- File Upload and Storage Module

### User Authentication and Authorization Module

**Input:**
User provides email and password credentials through the login screen. Alternatively, users can authenticate via Google OAuth using Firebase.

**Output:**
Upon successful authentication, a JSON Web Token (JWT) is generated and returned to the client for subsequent authenticated API requests.

```
javascript
const jwt = require('jsonwebtoken');
const { User } = require('../models');

const generateToken = (user) => {
  return jwt.sign(
    { id: user.id, email: user.email, role: user.role },
    process.env.JWT\_SECRET,
    { expiresIn: '7d' }
  );
};

exports.login = async (req, res) => {
  try {
    const { email, password } = req.body;
```

```
    if (!email || !password) {
      return res.status(400).json({ error: 'Email and password are required' });
    }

    const user = await User.findOne({ where: { email } });

    if (!user) {
      return res.status(401).json({ error: 'Invalid credentials' });
    }

    const isValidPassword = await user.comparePassword(password);
    if (!isValidPassword) {
      return res.status(401).json({ error: 'Invalid credentials' });
    }

    const token = generateToken(user);
    res.json({ success: true, token, user });
  } catch (error) {
    console.error('Login error:', error);
    res.status(500).json({ error: 'Internal server error' });
  }
};
```

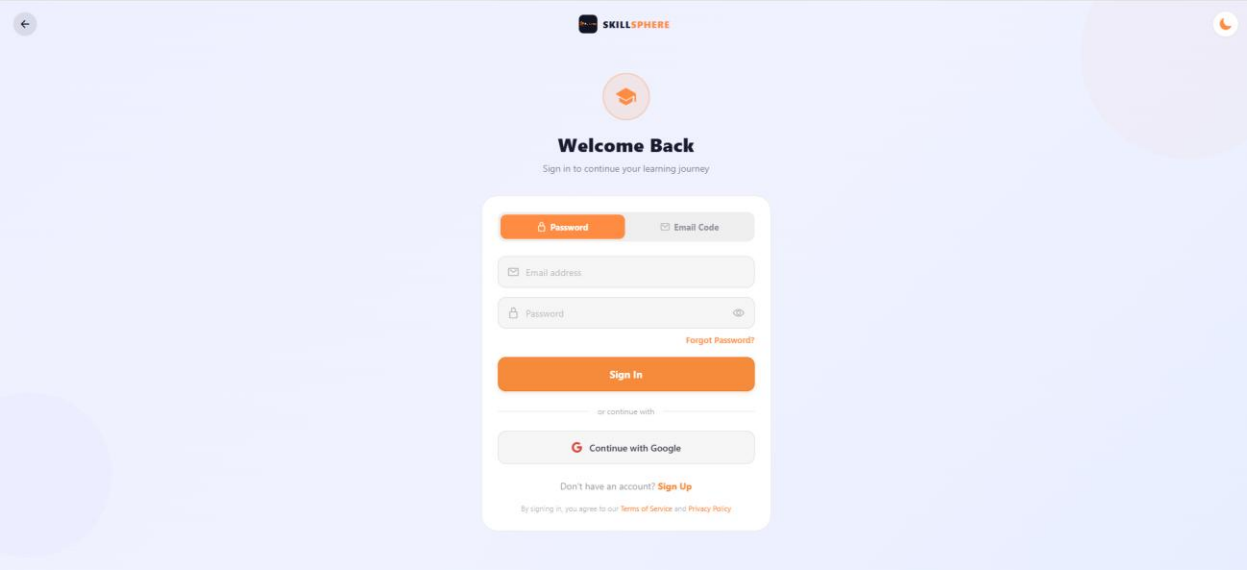



*Figure 29:User Authentication Module*

In this code, the login function handles user authentication. First, the email and password are

extracted from the request body. The system validates that both fields are provided. It then queries the database to find a user with the matching email. If found, the password is compared using bcrypt hashing. Upon successful authentication, a JWT token is generated containing the user's ID, email, and role, with a 7-day expiration. The token is returned to the client for use in subsequent API requests.

## JWT Authentication Middleware

**Input:**
Every authenticated API request includes a Bearer token in the Authorization header.

**Output:**
The middleware verifies the token and attaches the user object to the request for downstream handlers.

```
javascript
const jwt = require('jsonwebtoken');
const { User } = require('../models');

const authenticateToken = async (req, res, next) => {
  try {
    const authHeader = req.headers\['authorization'];
    const token = authHeader \&\& authHeader.split(' ')\[1];

    if (!token) {
      return res.status(401).json({ error: 'Access token required' });
    }

    const decoded = jwt.verify(token, process.env.JWT\_SECRET);
    const user = await User.findByPk(decoded.id);

    if (!user) {
      return res.status(401).json({ error: 'User not found' });
    }

    if (!user.isActive) {
      return res.status(401).json({ error: 'Account is inactive' });
    }
```

```
    req.user = user;
    next();
  } catch (error) {
    return res.status(403).json({ error: 'Invalid or expired token' });
  }
};

const requireInstructor = (req, res, next) => {
  if (!\['Instructor', 'Admin'].includes(req.user.role)) {
    return res.status(403).json({ error: 'Instructor access required' });
  }
  next();
};
```

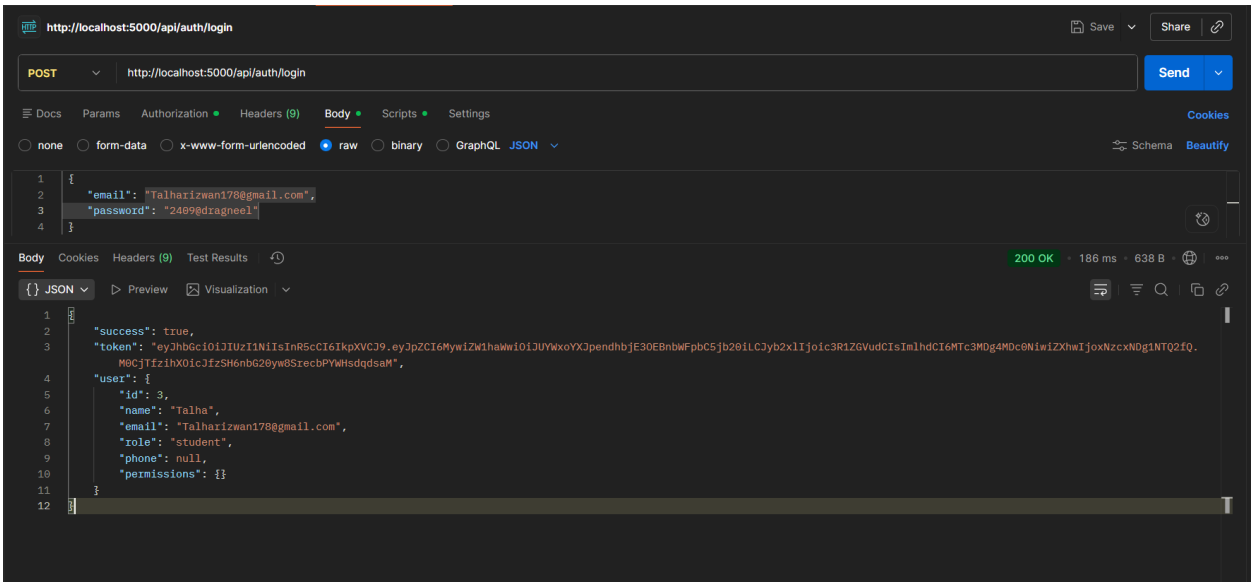



*Figure 30: JWT Token Response from Login API in Postman*

In this code, the authentication middleware intercepts every protected API request. It extracts the JWT token from the Authorization header, verifies it using the secret key, and fetches the corresponding user from the database. Role-based access control is implemented through additional middleware functions like `requireInstructor`, `requireExpert`, and `requireStudent`, which check the user's role before allowing access to specific endpoints.

## Course Management Module

**Input:**
Instructor or expert provides course details including name, description, level, language, category, duration, and optional thumbnail image through the Create Course screen.

**Output:**
A new course record is created in the database with all provided details and associated materials.

```
javascript
const { Course, Category, Material } = require('../models');

exports.createCourse = async (req, res) => {
  try {
    const { name, description, level, language, categoryId,
          duration, materials, thumbnailImage } = req.body;


    if (!name || !description || !level || !language ||
        !categoryId || !duration) {
      return res.status(400).json({
        error: 'All required fields must be provided'
      });
    }

    const category = await Category.findByPk(categoryId);
    if (!category) {
      return res.status(404).json({ error: 'Category not found' });
    }

    const course = await Course.create({
      name, description, level, language, categoryId,
      duration, status: 'draft', userId: req.user.id,
      thumbnailImage: thumbnailImage || null
    });

    if (materials \&\& Array.isArray(materials) \&\& materials.length > 0) {
      const courseMaterials = materials.map(material => ({
        ...material,
```

```
      courseId: course.id
    }));
    await Material.bulkCreate(courseMaterials);
  }

  const createdCourse = await Course.findByPk(course.id, {
    include: \[
      { model: Category, as: 'category' },
      { model: Material, as: 'materials' }
    ]
  });

  res.status(201).json({ success: true, course: createdCourse });
 } catch (error) {
  console.error('Create course error:', error);
  res.status(500).json({ error: 'Internal server error' });
 }
};
```

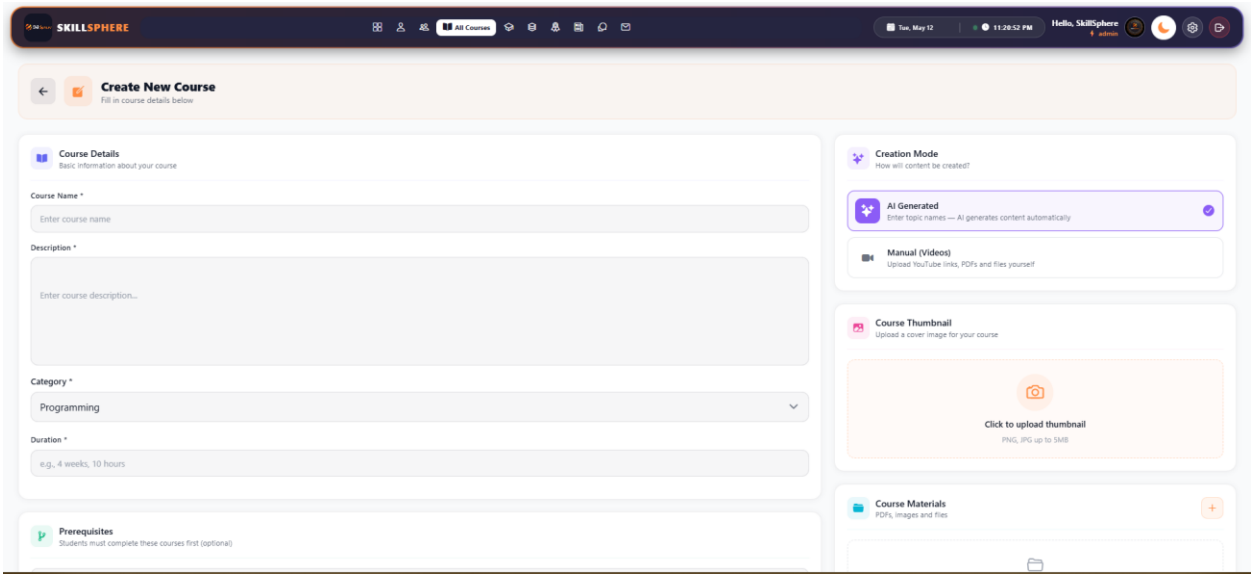



*Figure 31: Course Management Module*

In this code, the course creation function validates all required fields, verifies that the specified category exists in the database, creates a new course record with an initial status of 'draft', and optionally bulk-creates associated materials. The function uses Sequelize's eager loading to return the complete course object with its category and materials associations.

## Enrollment Module

**Input:**
A student clicks the "Enroll" button on a course detail page, sending the course ID to the backend.

**Output:**
An enrollment record is created linking the student to the course, enabling access to course materials and progress tracking.

```
javascript
const { Enrollment, Course, Progress } = require('../models');

exports.enrollInCourse = async (req, res) => {
  try {
    const { courseId } = req.body;

    if (!courseId) {
      return res.status(400).json({ error: 'Course ID is required' });
    }

    const course = await Course.findByPk(courseId);
    if (!course) {
      return res.status(404).json({ error: 'Course not found' });
    }

    const existing = await Enrollment.findOne({
      where: { userId: req.user.id, courseId }
    });




    if (existing) {
      return res.status(400).json({
        error: 'Already enrolled in this course'
      });
    }
```

```
    const enrollment = await Enrollment.create({
      userId: req.user.id,
      courseId,
      status: 'enrolled'
    });

    res.status(201).json({ success: true, enrollment });
  } catch (error) {
    console.error('Enroll in course error:', error);
    res.status(500).json({ error: 'Internal server error' });
  }
};

exports.unenrollFromCourse = async (req, res) => {
  try {
    const { courseId } = req.params;

    const enrollment = await Enrollment.findOne({
      where: { userId: req.user.id, courseId }
    });

    if (!enrollment) {
      return res.status(404).json({ error: 'Enrollment not found' });
    }

    await Progress.destroy({
      where: { userId: req.user.id, courseId }
    });

    await enrollment.destroy();
    res.json({ success: true, message: 'Unenrolled successfully' });


  } catch (error) {
    console.error('Unenroll from course error:', error);
    res.status(500).json({ error: 'Internal server error' });
  }
};
```

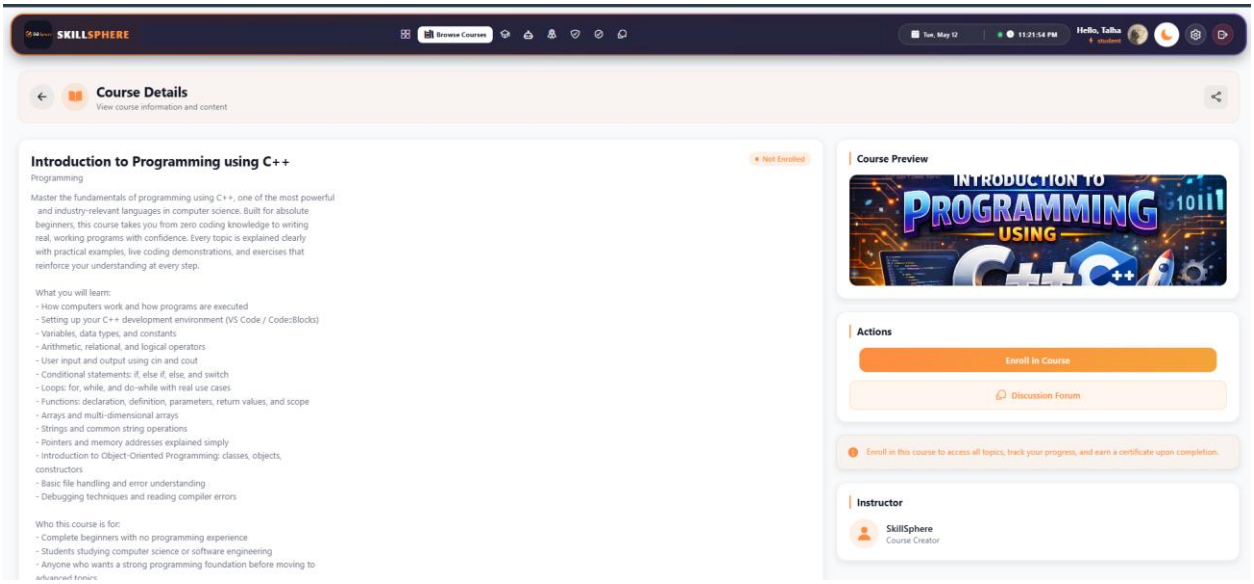



*Figure 32: Enrollment Module*

In this code, the enrollment function first validates the course existence, then checks for duplicate enrollment to prevent a student from enrolling in the same course twice. Upon successful enrollment, a record is created linking the student's user ID to the course ID. The unenrollment function removes both the enrollment record and all associated progress records, ensuring a clean state.

## File Upload and Cloud Storage Module

**Input:**
User uploads a file (PDF or image) through the material upload modal or course creation form.

**Output:**
The file is uploaded to Cloudinary cloud storage and a persistent URL is returned and stored in the database.

```
javascript
const cloudinary = require('cloudinary').v2;
const { CloudinaryStorage } = require('multer-storage-cloudinary');
const path = require('path');

cloudinary.config({
  cloud\_name: process.env.CLOUDINARY\_CLOUD\_NAME,
  api\_key: process.env.CLOUDINARY\_API\_KEY,
```

```
  api\_secret: process.env.CLOUDINARY\_API\_SECRET,
});

const generalStorage = new CloudinaryStorage({
  cloudinary,
  params: async (req, file) => {
    const isPdf = file.mimetype === 'application/pdf';
    const ext = path.extname(file.originalname);


    const nameWithoutExt = path.basename(file.originalname, ext);
    const uniqueName = `${Date.now()}-${nameWithoutExt}${ext}`;

    return {
      folder: 'skillsphere/uploads',
      resource\_type: isPdf ? 'raw' : 'image',
      public\_id: isPdf ? uniqueName : undefined,
      allowed\_formats: \['pdf', 'jpg', 'jpeg', 'png'],
    };
  },
});
```

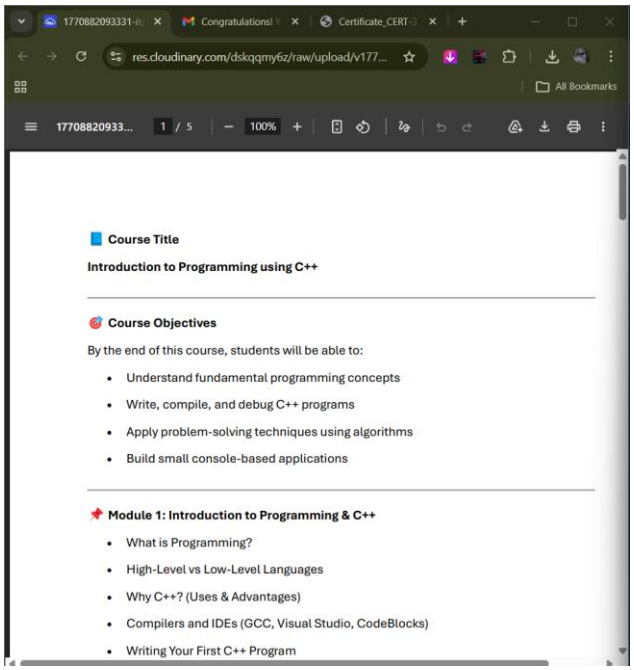



*Figure 33: File Upload and Cloud Storage Module*

In this code, the Cloudinary SDK is configured with environment credentials. A custom storage engine is created using `multer-storage-cloudinary` that dynamically determines the resource type

based on the uploaded file's MIME type. PDF files are uploaded as 'raw' resources (preserving the original file format for download) while images are uploaded as 'image' resources (enabling Cloudinary's image optimization). The original filename with extension is preserved for PDFs to ensure proper download behavior in browsers.

## AI Chat Assistant Module

**Input:**
Student types a message in the AI chat interface, which is sent along with conversation history to the backend.

**Output:**
An AI-generated response is returned using Google Gemini API, with context from previous messages in the session.

```
javascript
const GEMINI\_API\_URL = 'https://generativelanguage.googleapis.com/' +
  'v1beta/models/gemini-1.5-flash:generateContent';


async function generateResponse(userMessage, chatHistory = \[]) {
  const apiKey = process.env.GEMINI\_API\_KEY;


  if (!apiKey) {
    return getFallbackResponse();
  }

  try {
    const contents = \[];

    contents.push({
      role: 'user',
      parts: \[{ text: SYSTEM\_PROMPT }]
    });
    contents.push({
      role: 'model',
      parts: \[{ text: 'Understood! I am SkillSphere AI, ' +
        'your versatile assistant.' }]
    });
```

```
  const recentHistory = chatHistory.slice(-10);
  for (const msg of recentHistory) {
    contents.push({
      role: msg.sender === 'user' ? 'user' : 'model',
      parts: \[{ text: msg.content }]
    });
  }

  contents.push({
    role: 'user',
    parts: \[{ text: userMessage }]
  });

  const response = await fetch(
    `${GEMINI\_API\_URL}?key=${apiKey}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      contents,
      generationConfig: {
        temperature: 0.7,
        topK: 40,
        topP: 0.95,
        maxOutputTokens: 1024,
      },
    })
  });

  const data = await response.json();

  if (data.candidates \&\& data.candidates\[0]
      ?.content?.parts?.\[0]?.text) {
    return data.candidates\[0].content.parts\[0].text;
  }

  return getFallbackResponse();
} catch (error) {
  return getFallbackResponse();
}
```

}

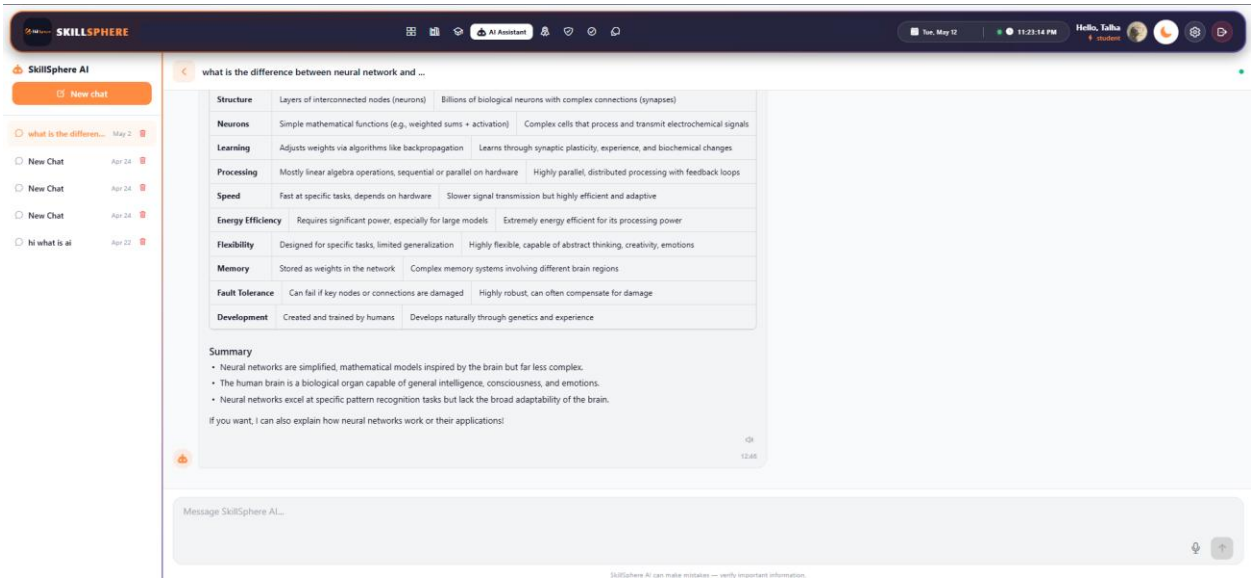



*Figure 34: AI Chat Assistant Module*

In this code, the Gemini AI service handles intelligent conversation. The function builds a conversation context by combining a system prompt (defining the AI's personality and capabilities), the last 10 messages from chat history, and the current user message. This context is sent to Google's Gemini 1.5 Flash API, which generates a contextually relevant response. The temperature parameter (0.7) balances creativity and accuracy. Fallback responses are provided when the API is unavailable or rate-limited.

## Certificate Generation Module

**Input:**
When a student completes all topics in a course, the system triggers certificate generation with student name, course name, certificate number, and issue date.

**Output:**
A professionally formatted PDF certificate is generated using React PDF renderer and uploaded to Cloudinary for persistent storage.

```
javascript
const { cloudinary } = require('../config/cloudinary');

const saveCertificatePDF = async (pdfBuffer, certificateNumber) => {
  try {
    const filename = `Certificate\_${certificateNumber}`;

    const result = await new Promise((resolve, reject) => {
      const uploadStream = cloudinary.uploader.upload\_stream(
        {
          resource\_type: 'raw',
          folder: 'skillsphere/certificates',
          public\_id: filename,
        },
        (error, result) => {
          if (error) reject(error);
          else resolve(result);
        }
      );
      uploadStream.end(pdfBuffer);
    });

    return result.secure\_url;
  } catch (error)
{
    console.error('Error saving certificate PDF:', error);
```

```
    throw error;

  }
};

const generateCertificateNumber = (userId, courseId) => {
  const timestamp = Date.now();

  const random = crypto.randomBytes(4).toString('hex').toUpperCase();

  return `CERT-${userId}-${courseId}-${timestamp}-${random}`;
};
```

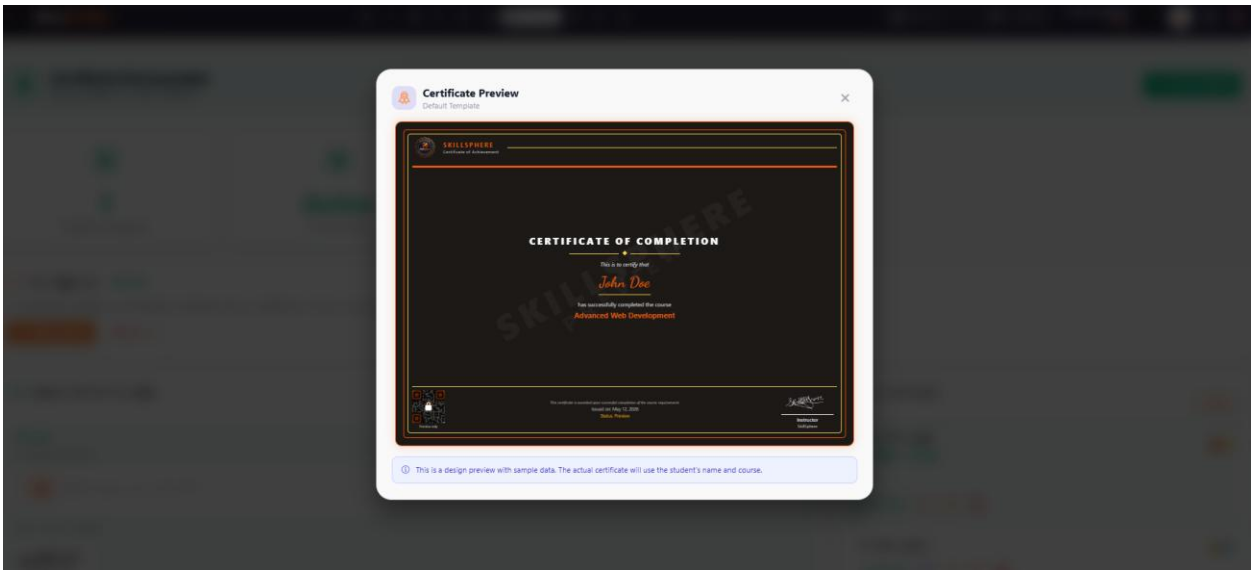



*Figure 35:Certificate Generation Module*

In this code, the certificate PDF buffer (generated by @react-pdf/renderer) is uploaded to Cloudinary using the stream upload API with `resource\_type: 'raw'` to preserve the PDF format. A unique certificate number is generated using a combination of user ID, course ID, timestamp, and cryptographically random bytes to ensure uniqueness. The Cloudinary secure URL is returned and stored in the database for future access.

# *Chapter 5: Software Testing*

This chapter provides a description about the adopted testing procedure. This includes the selected testing methodology, test suite, and the test results of the developed software.

## 5.1. Testing Methodology

We have used black box testing in our testing phase. Black box testing is a method of software testing that examines the functionality of an application without peering into its internal structures or workings. This method of test was applied at multiple levels of software testing: unit, integration, and system testing.

Additionally, API endpoint testing was performed using Postman to verify all RESTful API endpoints. Frontend component testing was conducted manually across multiple screen sizes (mobile, tablet, and desktop) to ensure responsive design compatibility.

Benefits of the adopted testing approach:

- Black box tests are reproducible and can be run repeatedly.
- Tests are performed from a user's perspective, ensuring usability.
- API testing with Postman validates request/response contracts.
- Cross-platform testing ensures compatibility on Android and Web.
- Tests help expose ambiguities or inconsistencies in the specifications.

## 5.2. Testing Environment

### Postman

Postman is an API development and testing platform used for testing all backend RESTful API endpoints. Each endpoint was tested with valid inputs, invalid inputs, missing authentication tokens, and edge cases to ensure robust error handling.

### Web Browser (Chrome DevTools)

Google Chrome with Developer Tools was used for testing the React Native Web version of the application. The responsive design mode was used to simulate various screen sizes (mobile, tablet, desktop) for cross-platform compatibility testing.

### HeidiSQL

HeidiSQL is a database management tool used for inspecting the MySQL/MariaDB database. It was used to verify that database tables were created correctly, foreign key relationships were maintained, and data was being stored as expected after API operations.

## 5.3. API Testing with Postman

### 5.3.1. Test Case: User Login

*Table 31: Test Case: User Login*

| | |
|---|---|
| **Date** | Febuary 2026 |
| **System** | SkillSphere LMS |
| **Objective** | User Login Authentication |
| **Version** | 1.0 |
| **Test Type** | Unit Testing |
| **Method** | POST |
| **Endpoint** | /api/auth/login |
| **Full URL** | http://localhost:5000/api/auth/login |
| **Auth Required** | No |
| **Input** | {"email": "skillspherefyp@gmail.com", "password": "skillsphere@123"} |
| **Expected Result** | User is authenticated and JWT token is returned with user data. |
| **Actual Result** | **Passed** |

**Description:**

The login API endpoint /api/auth/login was tested using Postman. A POST request was sent with valid email and password in the request body (raw JSON). The API returned a 200 status code with a JWT token and user object containing id, name, email, and role fields. The token was then used

in subsequent requests to verify authentication middleware. Testing with invalid credentials returned 401 Unauthorized status.

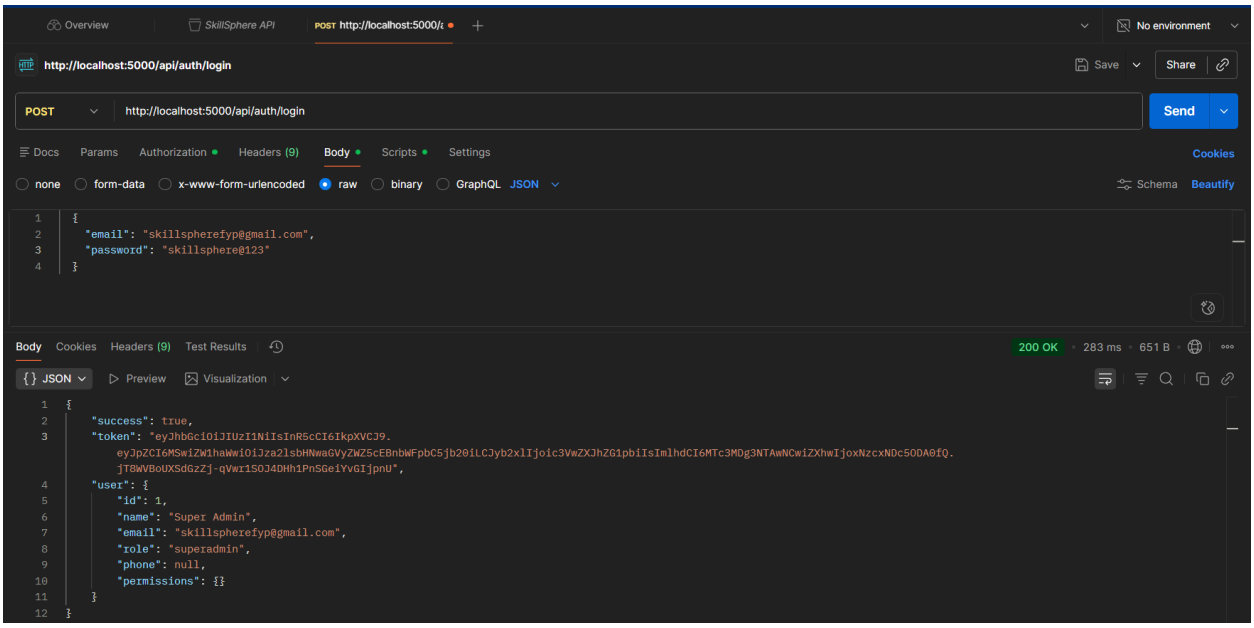



*Figure 36: Test login Result Postman*

## 5.3.2. Test Case: User Registration with OTP

*Table 32: Test Case: User Registration with OTP*

| | |
|---|---|
| **Date** | Febuary 2026 |
| **System** | SkillSphere LMS |
| **Objective** | New User Registration |
| **Version** | 1.0 |
| **Test Type** | Integration Testing |
| **Method** | POST |
| **Endpoint** | /api/auth/send-otp -> /api/auth/verify-signup-otp |
| **Full URL** | http://localhost:5000/api/auth/send-otp |
| **Auth Required** | No |
| **Input** | {"email": "Talharizwan178@gmail.com", "name": "Talha"} |
| **Expected Result** | OTP is sent to email, and upon verification, user account is created. |
| **Actual Result** | **Passed** |

**Description:**

The registration flow was tested as a two-step process. First, /api/auth/send-otp was called with name and email, which triggered an OTP email. Then /api/auth/verify-signup-otp was called with the email and OTP code. Upon successful verification, a student account was automatically created and a JWT token was returned.

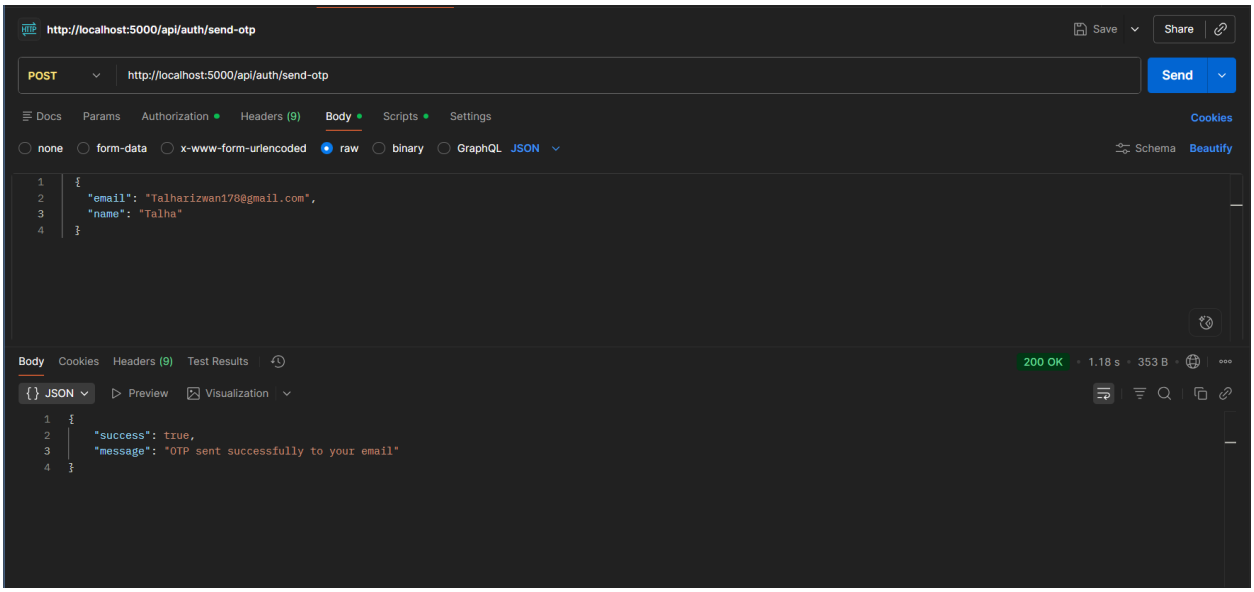



*Figure 37: Test Registration with OTP Result Postman*

### 5.3.3. Test Case: Get All Categories

*Table 33: Test Case: Get All Categories*

| | |
|---|---|
| **Date** | Febuary 2026 |
| **System** | SkillSphere LMS |
| **Objective** | Retrieve Course Categories |
| **Version** | 1.0 |
| **Test Type** | Unit Testing |
| **Method** | GET |
| **Endpoint** | /api/categories |
| **Full URL** | http://localhost:5000/api/categories |
| **Auth Required** | No |
| **Input** | None |
| **Expected Result** | Array of all categories with id, name, and description fields. |
| **Actual Result** | **Passed** |

**Description:**

The categories endpoint was tested without authentication as it is a public endpoint. A GET request returned a 200 status code with an array of category objects. Each category contained id, name,

description, and createdAt fields. The response was verified to contain the expected categories that were created.

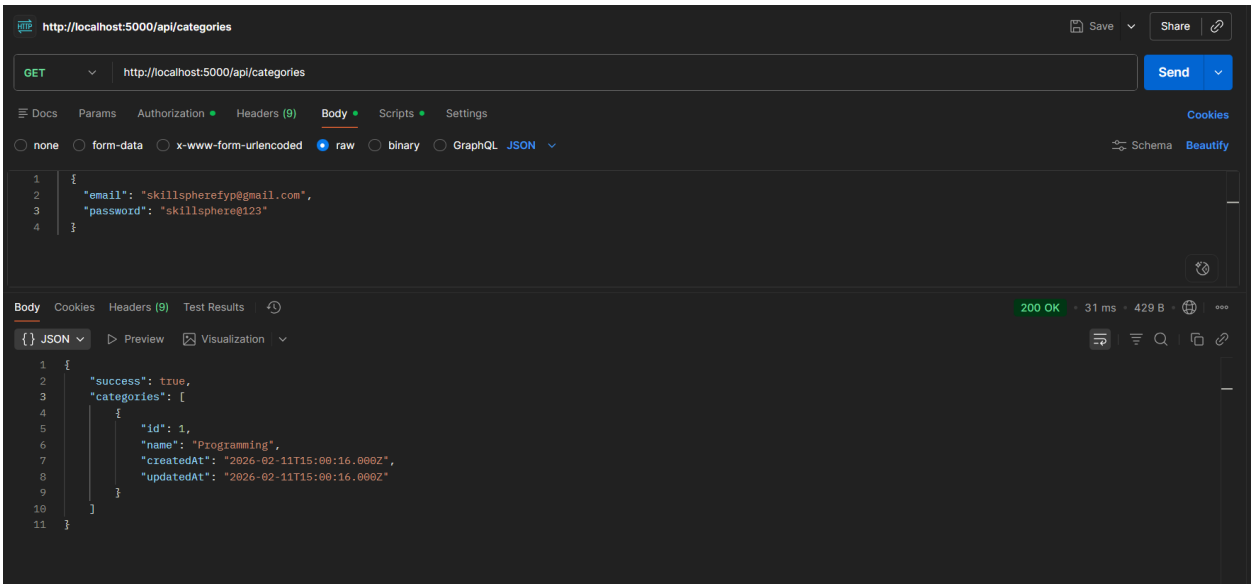



*Figure 38: Test Get All Categories Result Postman*

### 5.3.4. Test Case: Get Topics by Course

*Table 34: Test Case: Get Topics by Course*

| | |
|---|---|
| **Date** | Febuary 2026 |
| **System** | SkillSphere LMS |
| **Objective** | Retrieve Course Topics |
| **Version** | 1.0 |
| **Test Type** | Unit Testing |
| **Method** | GET |
| **Endpoint** | /api/topics/course/:courseId |
| **Full URL** | http://localhost:5000/api/topics/course/1 |
| **Auth Required** | Yes (Bearer Token) |
| **Input** | courseId: 1 (URL parameter) |
| **Expected Result** | Array of topics belonging to the specified course, ordered by sequence. |
| **Actual Result** | **Passed** |

**Description:**

The topics endpoint was tested with a valid JWT token in the Authorization header. A GET request to /api/topics/course/1 returned all topics for course ID 1. Each topic object contained id, name, description, courseId, order, and associated materials. Topics were returned in the correct order as specified in the order field.

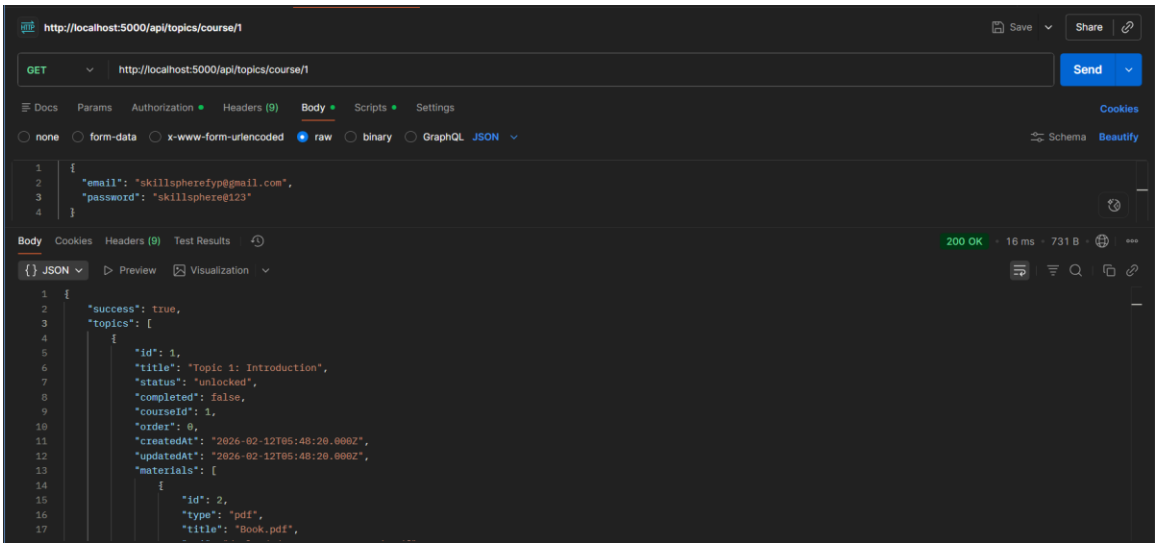



*Figure 39: Test Get Topics by Course id Result Postman*

## 5.3.5. Test Case: Get All Courses

*Table 35: Test Case: Get All Courses*

| | |
|---|---|
| **Date** | Febuary 2026 |
| **System** | SkillSphere LMS |
| **Objective** | Retrieve All Courses |
| **Version** | 1.0 |
| **Test Type** | Unit Testing |
| **Method** | GET |
| **Endpoint** | /api/courses |
| **Full URL** | http://localhost:5000/api/courses |
| **Auth Required** | No |
| **Input** | None |
| **Expected Result** | Array of all published courses with course details. |
| **Actual Result** | **Passed** |

**Description:**

The courses listing endpoint was tested without authentication. The API returned a 200 status code with an array of course objects including id, name, description, categoryId, isPublished, and instructor details. Only published courses were returned to unauthenticated users.

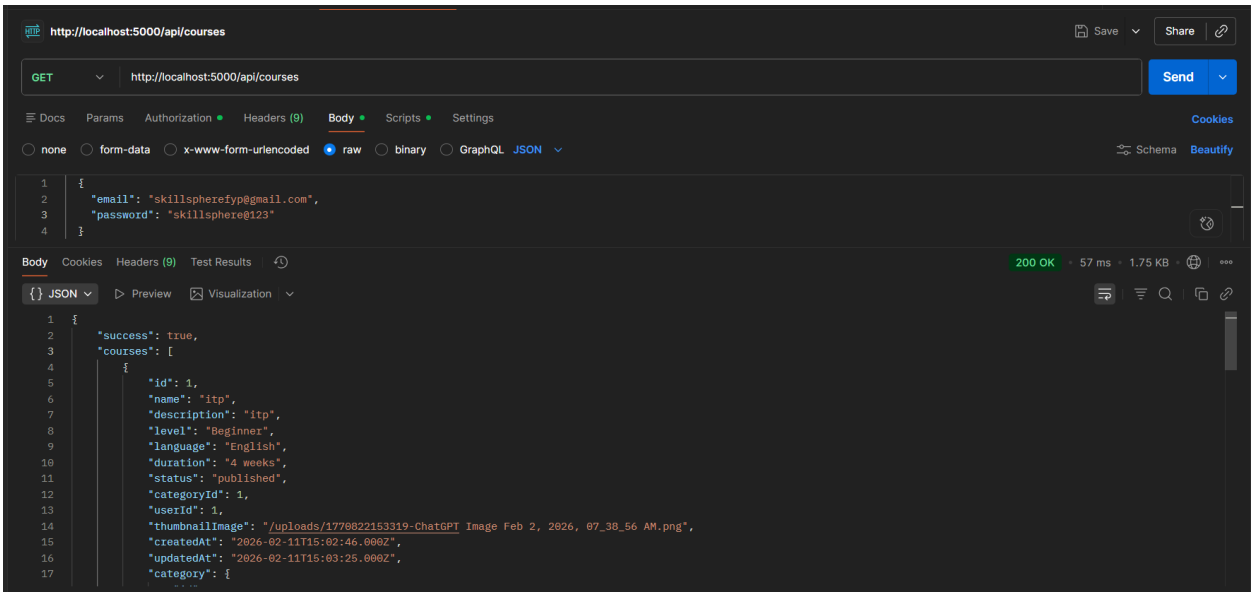



*Figure 40: Test Get All Courses Result Postman*

### 5.3.6. Test Case: Course Enrollment

*Table 36: Test Case: Course Enrollment*

| | |
|---|---|
| **Date** | Febuary 2026 |
| **System** | SkillSphere LMS |
| **Objective** | Enroll Student in Course |
| **Version** | 1.0 |
| **Test Type** | Unit Testing |
| **Method** | POST |
| **Endpoint** | /api/enrollments |
| **Full URL** | http://localhost:5000/api/enrollments |
| **Auth Required** | Yes (Bearer Token) |
| **Input** | {"courseId": 1} |
| **Expected Result** | Enrollment record is created and returned with status 201. |
| **Actual Result** | **Passed** |

**Description:**

The enrollment API endpoint was tested with a valid student JWT token. A POST request with the course ID was sent in the request body. The API created the enrollment record and returned it with status 201. A duplicate enrollment attempt returned status 400 with error message "Already enrolled in this course", confirming the duplicate prevention logic.

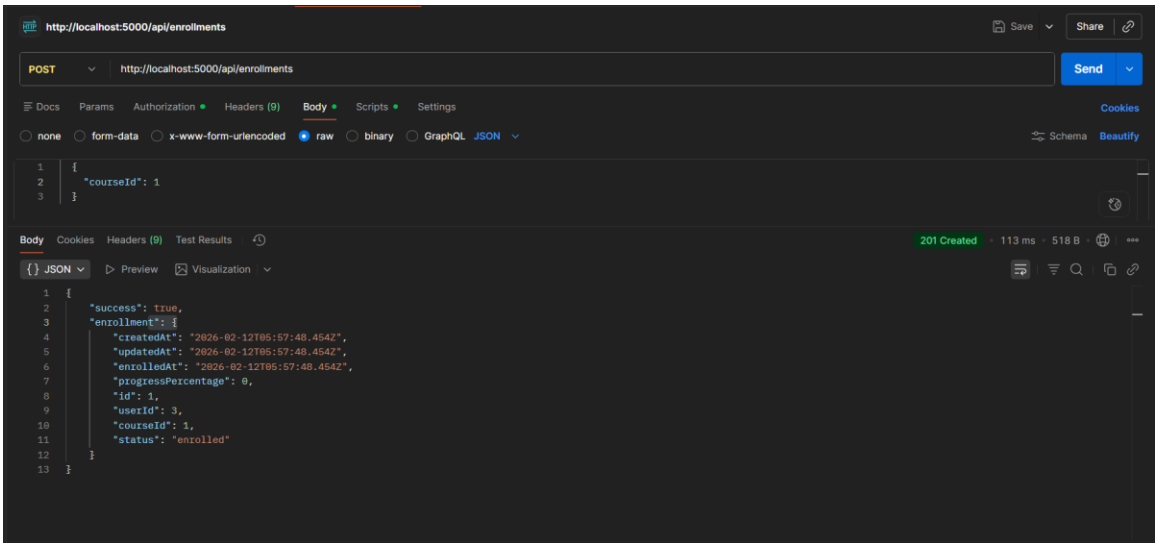



*Figure 41: Test Course Enrollment Result Postman*

## 5.3.7. Test Case: Check Enrollment Status

*Table 37: Test Case: Check Enrollment Status*

| | |
|---|---|
| **Date** | Febuary 2026 |
| **System** | SkillSphere LMS |
| **Objective** | Verify Enrollment Status |
| **Version** | 1.0 |
| **Test Type** | Unit Testing |
| **Method** | GET |
| **Endpoint** | /api/enrollments/check/:courseId |
| **Full URL** | http://localhost:5000/api/enrollments/check/1 |
| **Auth Required** | Yes (Bearer Token) |
| **Input** | courseId: 1 (URL parameter) |
| **Expected Result** | Boolean indicating whether user is enrolled in the course. |
| **Actual Result** | **Passed** |

**Description:**

The enrollment check endpoint was tested with a student JWT token. For an enrolled course, the API returned {"enrolled": true}. For a non-enrolled course, it returned {"enrolled": false}. This endpoint is used by the frontend to show appropriate enrollment buttons.

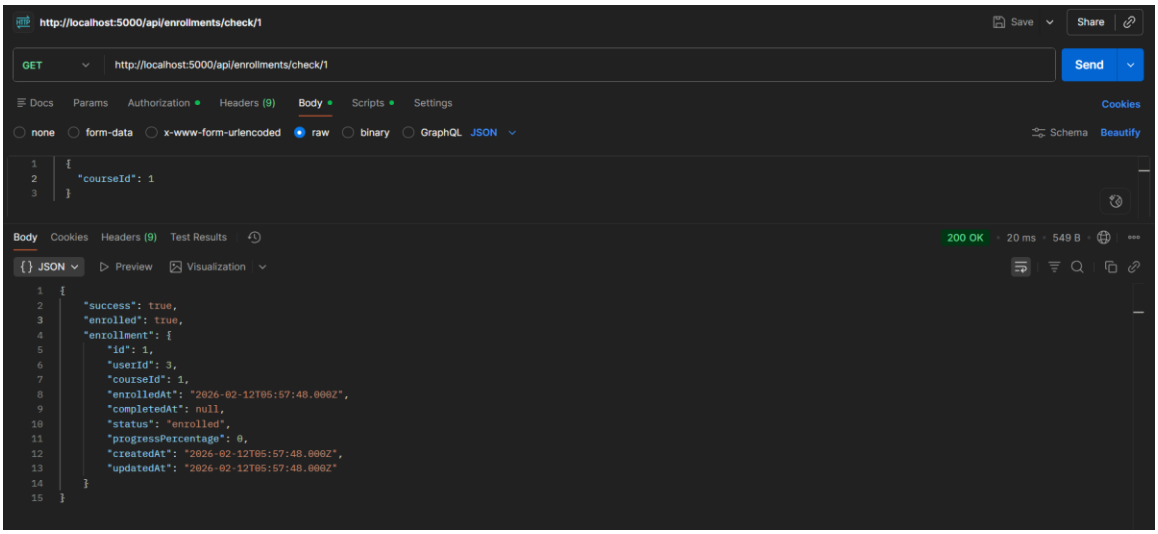



*Figure 42: Test Enrollment Status Result Postman*

## 5.3.8. Test Case: File Upload (PDF Material)

*Table 38: Test Case: File Upload (PDF Material)*

| | |
|---|---|
| **Date** | Febuary 2026 |
| **System** | SkillSphere LMS |
| **Objective** | Upload PDF Learning Material |
| **Version** | 1.0 |
| **Test Type** | Integration Testing |
| **Method** | POST |
| **Endpoint** | /api/upload/file |
| **Full URL** | http://localhost:5000/api/upload/file |
| **Auth Required** | No |
| **Input** | PDF file (Computer_Forensice_ Full_Study_Guide (1).pdf , 466 KB) via form-data with key "file" |
| **Expected Result** | File is uploaded to uploads folder and full URL is returned. |
| **Actual Result** | **Passed** |

**Description:**

The file upload endpoint was tested by uploading a PDF file via multipart form-data in Postman. The Body tab was set to "form-data" with key "file" and type changed to "File". The API uploaded the file to Cloudinary with resource_type: 'raw', preserved the original filename with .pdf extension, and returned the full Cloudinary URL. The returned URL was verified to be accessible and downloadable in a web browser. Files exceeding 10MB were rejected with a 400 status code.

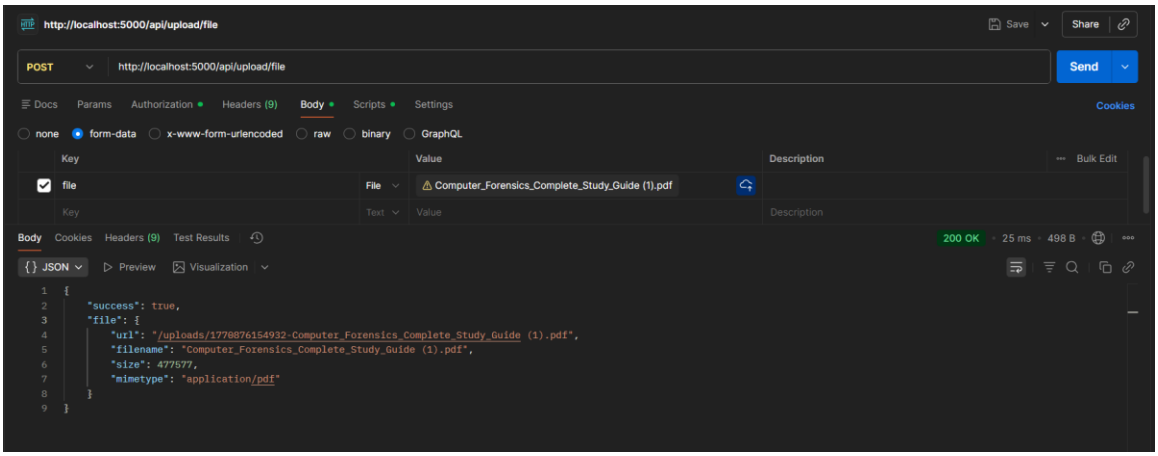



*Figure 43: Test File Upload Result Postman*

## 5.3.9. Test Case: Route Protection

*Table 39: Test Case: Route Protection*

| | |
|---|---|
| **Date** | Febuary 2026 |
| **System** | SkillSphere LMS |
| **Objective** | Verify Route Protection |
| **Version** | 1.0 |
| **Test Type** | Unit Testing |
| **Method** | GET |
| **Endpoint** | api/Instructors |
| **Full URL** | http://localhost:5000/api/Instructors |
| **Auth Required** | Yes (Bearer Token) |
| **Input** | {"email": "skillspherefyp@gmail.com", "password": "skillsphere@123"} |
| **Expected Result** | Access denied with 403 status code and error message. |
| **Actual Result** | **Passed** |

**Description:**

Route protection was tested in Postman by attempting to access the Admin-only endpoint /api/Instructors using a Student JWT token. The requireAdmin middleware correctly rejected the request with a 403 Forbidden status and returned the error message "Admin access required". This confirms that role-based access control is properly implemented and prevents unauthorized users from accessing protected routes.

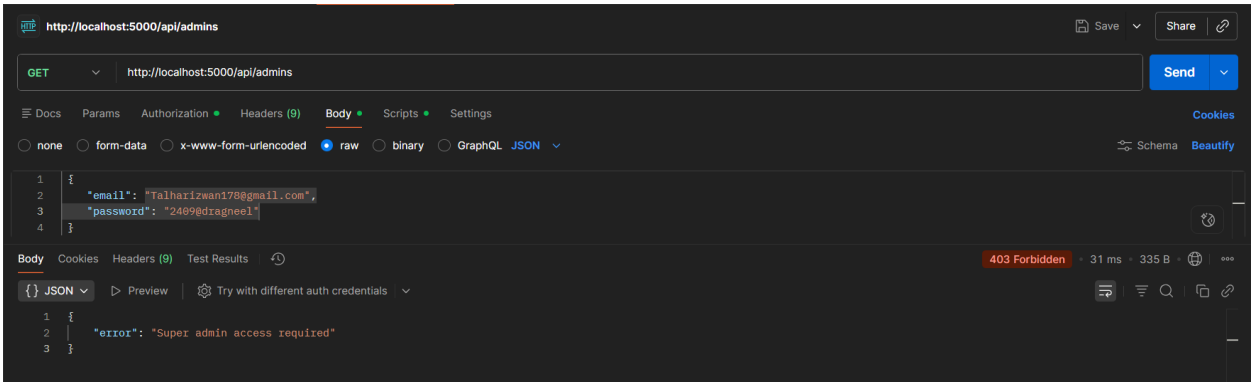



*Figure 44: Test Route Protection Result Postman*

## 5.4. Manual Testing with Chrome DevTools

### 5.4.1. Test Case: AI Chat Response Interface

*Table 40: Test Case: AI Chat Response Interface*

| | |
|---|---|
| **Date** | February 2026 |
| **System** | SkillSphere LMS |
| **Objective** | AI Chat User Interface Functionality |
| **Version** | 1.0 |
| **Test Type** | System Testing |
| **Input** | User types "What are your Capabilities?" in chat interface |
| **Expected Result** | Loading indicator appears, AI response displays with proper formatting. |
| **Actual Result** | **Passed** |

**Description:**

The AI chat interface was manually tested in Chrome browser. When a user submitted a question, a loading indicator appeared immediately. The AI response was displayed with proper markdown formatting including code blocks, bullet points, and headers. The chat history persisted across page navigation. Message timestamps were displayed correctly. The interface handled long responses with proper scrolling.

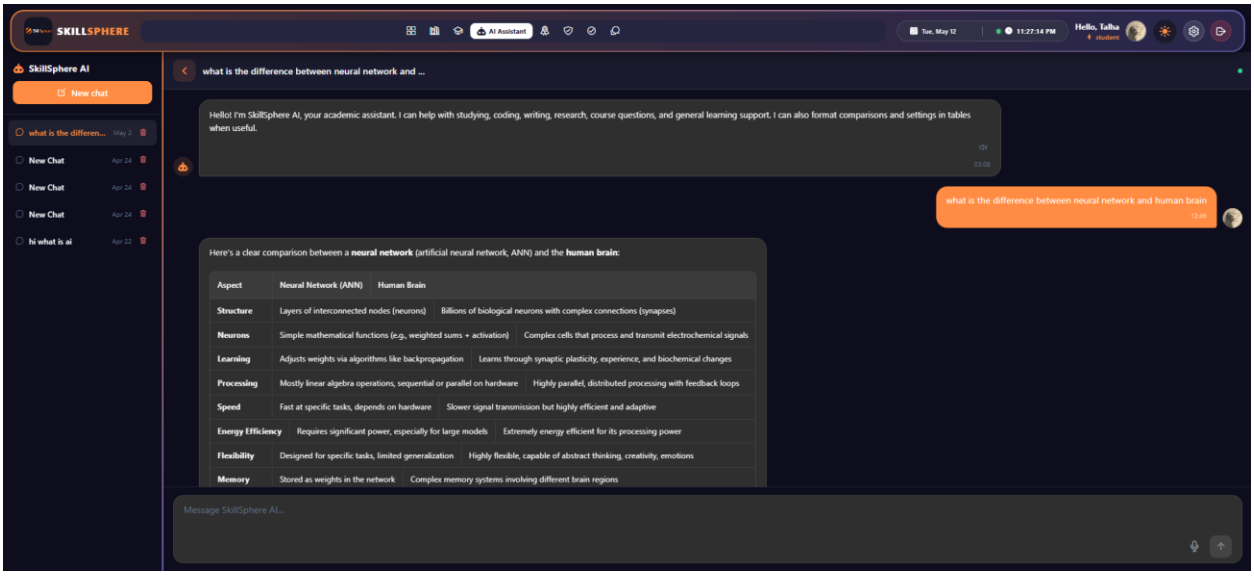



*Figure 45: Test AI Chat Response Result Chrome DevTools*

### 5.4.2. Test Case: Responsive Layout - Mobile (375px)

*Table 41: Test Case: Responsive Layout - Mobile (375px)*

| | |
|---|---|
| **Date** | February 2026 |
| **System** | SkillSphere LMS |
| **Objective** | Mobile Screen Responsiveness |
| **Version** | 1.0 |
| **Test Type** | System Testing |
| **Input** | Access application using Chrome DevTools device toolbar at 375px width |
| **Expected Result** | Layout adapts for mobile with collapsible sidebar and stacked elements. |
| **Actual Result** | **Passed** |

**Description:**

The application was tested at 375px width using Chrome DevTools responsive mode. The sidebar collapsed into a hamburger menu icon. Navigation items were accessible via the hamburger menu. Course cards displayed in a single-column layout. Stat cards on the dashboard displayed in a 2-column grid. Charts rendered at reduced height to fit the screen. Touch targets were appropriately sized for mobile interaction. Font sizes remained readable.

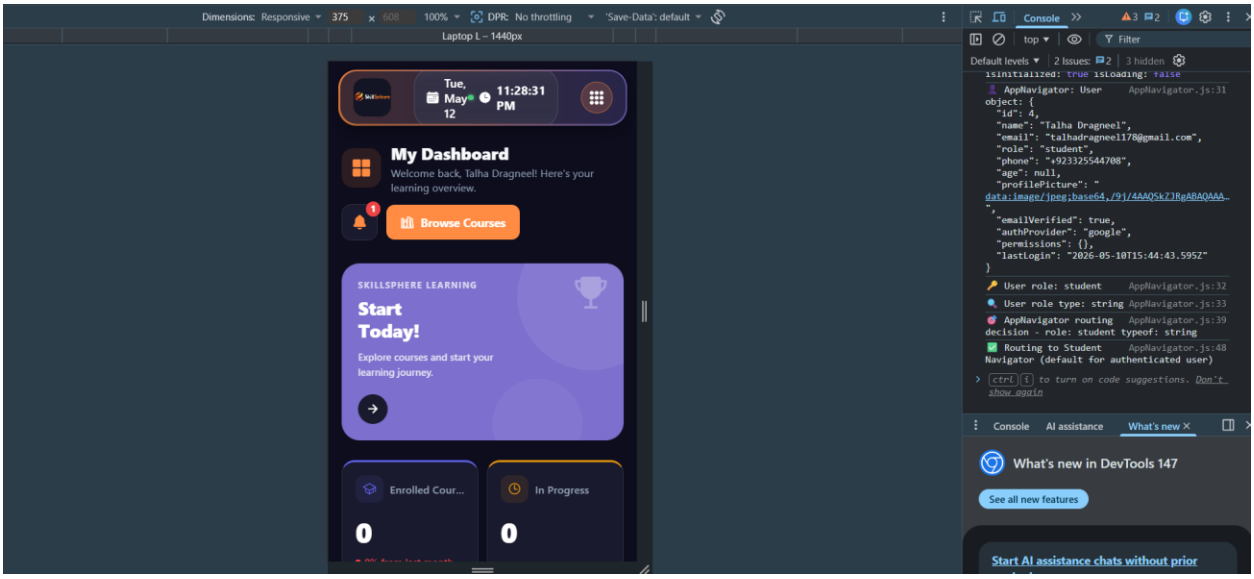



*Figure 46: Test Responsive Layout - Mobile (375px) Result Chrome DevTools*

### 5.4.3. Test Case: Responsive Layout - Tablet (768px)

*Table 42: Test Case: Responsive Layout - Tablet (768px)*

| | |
|---|---|
| **Date** | February 2026 |
| **System** | SkillSphere LMS |
| **Objective** | Tablet Screen Responsiveness |
| **Version** | 1.0 |
| **Test Type** | System Testing |
| **Input** | Access application using Chrome DevTools device toolbar at 768px width |
| **Expected Result** | Layout adapts with compact sidebar and two-column grids. |
| **Actual Result** | **Passed** |

**Description:**

The application was tested at 768px width simulating tablet devices. The sidebar displayed with compact icon-only navigation. Course cards arranged in a 2-column grid layout. Dashboard statistics displayed in a 2x2 grid. Charts displayed side by side at medium size. The PDF viewer was functional with pinch-to-zoom support simulation.

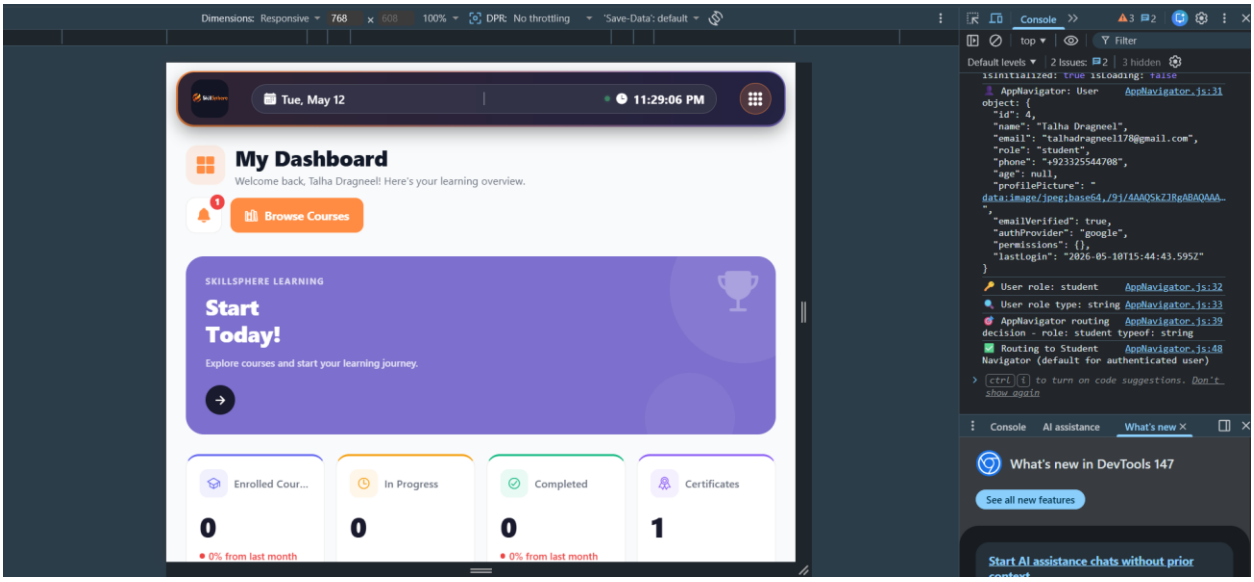



*Figure 47: Test Responsive Layout - Tablet (768px) Result Chrome DevTools*

### 5.4.4. Test Case: Responsive Layout - Desktop (1440px)

*Table 43: Test Case: Responsive Layout - Desktop (1440px)*

| | |
|---|---|
| **Date** | February 2026 |
| **System** | SkillSphere LMS |
| **Objective** | Desktop Screen Responsiveness |
| **Version** | 1.0 |
| **Test Type** | System Testing |
| **Input** | Access application using Chrome DevTools device toolbar at 1440px width |
| **Expected Result** | Full desktop layout with expanded sidebar and multi-column grids. |
| **Actual Result** | **Passed** |

**Description:**

The application was tested at 1440px width for desktop displays. The full sidebar with labels and icons was displayed. Course cards arranged in a 3-4 column grid. Dashboard displayed stat cards in a 4-column layout. Charts rendered at full size with all data points visible. All hover effects and tooltips functioned correctly.

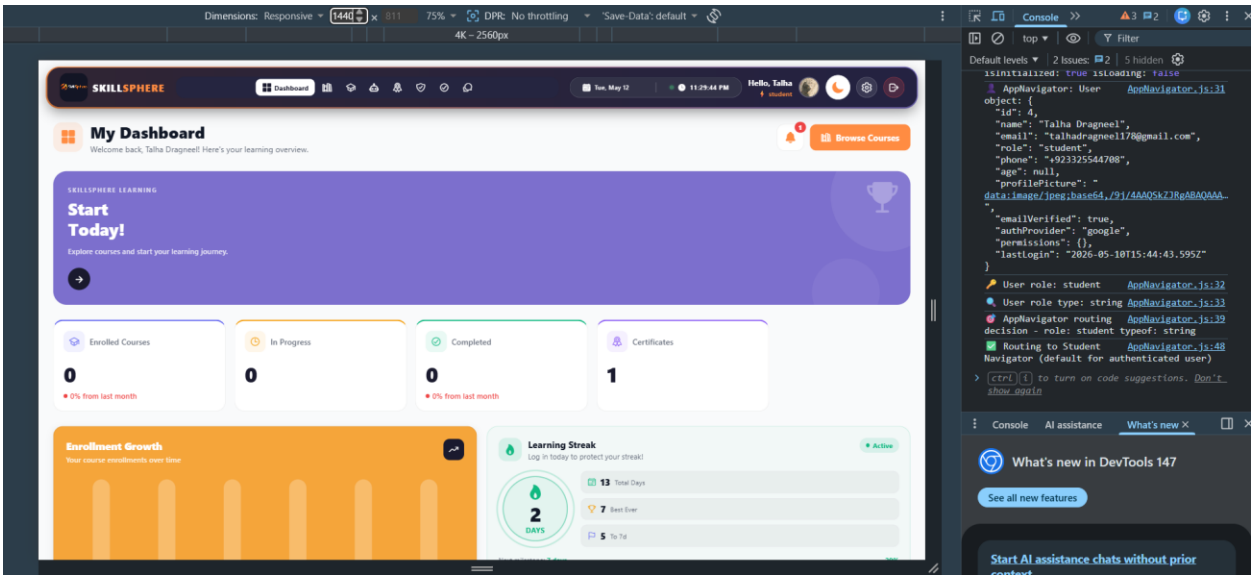



*Figure 48: Test Responsive Layout - Desktop (1440px) Result Chrome DevTools*

### 5.4.5. Test Case: Role-Based Access Control - User Roles and Dashboard Navigation

*Table 44: Test Case: Role-Based Access Control - User Roles and Dashboard Navigation*

| | |
|---|---|
| **Date** | Feburary 2026 |
| **System** | SkillSphere LMS |
| **Objective** | Verify Role Management and Dashboard Navigation |
| **Version** | 1.0 |
| **Test Type** | System Testing |
| **Input** | Login attempts with different user roles (Student, Expert, Instructor, Admin) |
| **Expected Result** | Users are redirected to their role-specific dashboard after login. |
| **Actual Result** | **Passed** |

**Description:**

*User Role Management:*

The SkillSphere LMS implements a role-based access control system where all users are stored in a single Users table in the database. The system supports four roles: Student, Expert, Instructor, and Admin. Each user record contains a "role" field that determines their access level and dashboard.

*Role Creation Process:*

1. Student Role: When a new user signs up through the public registration page (/api/auth/register or /api/auth/verify-signup-otp), their account is automatically created with the role set to "student" in the Users table. This is the only role that can be created through self-registration.
2. Expert Role: Expert accounts are created by the Admin through the Instructor panel. The Admin uses the /api/Instructors endpoint or user management interface to create expert accounts. The role is stored as "expert" in the Users table.
3. Instructor Role: Instructor accounts can only be created by the Admin through the /api/Instructors endpoint. This endpoint requires Admin authentication. The role is stored as "Instructor" in the Users table along with specific permissions.
4. Admin Role: The Admin account is pre-seeded in the database during initial setup and cannot be created through any API endpoint for security purposes.

*Login and Dashboard Navigation:*

When a user logs in through the /api/auth/login endpoint:

- The system checks the provided email against the Users table in the database.
- If the email exists and password matches, the user's role is retrieved from the database.
- A JWT token is generated containing the user's id, email, and role.
- The frontend receives the token and user data, then navigates to the appropriate dashboard based on the role:

### *Dashboard Navigation by Role:*

*Table 45: Dashboard Navigation by Role*

| Role | Dashboard | Access Level |
|---|---|---|
| Student | Student Dashboard | View courses, enroll, track progress, earn certificates |
| Expert | Expert Dashboard | View course material, topics and provides the feedback |
| Instructor | Instructor Dashboard | Manage users, courses, categories, view analytics. Create and manage courses, view enrolled students |
| Admin | Admin Dashboard | Full system access, create Instructors and experts |

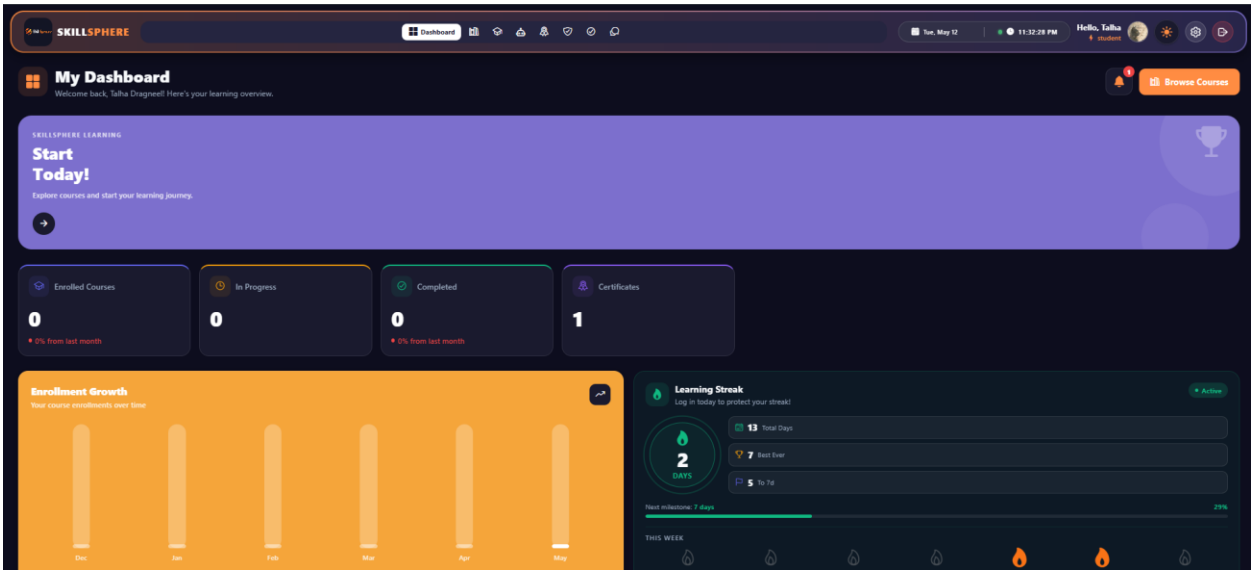



*Figure 49: Student Dashboard*

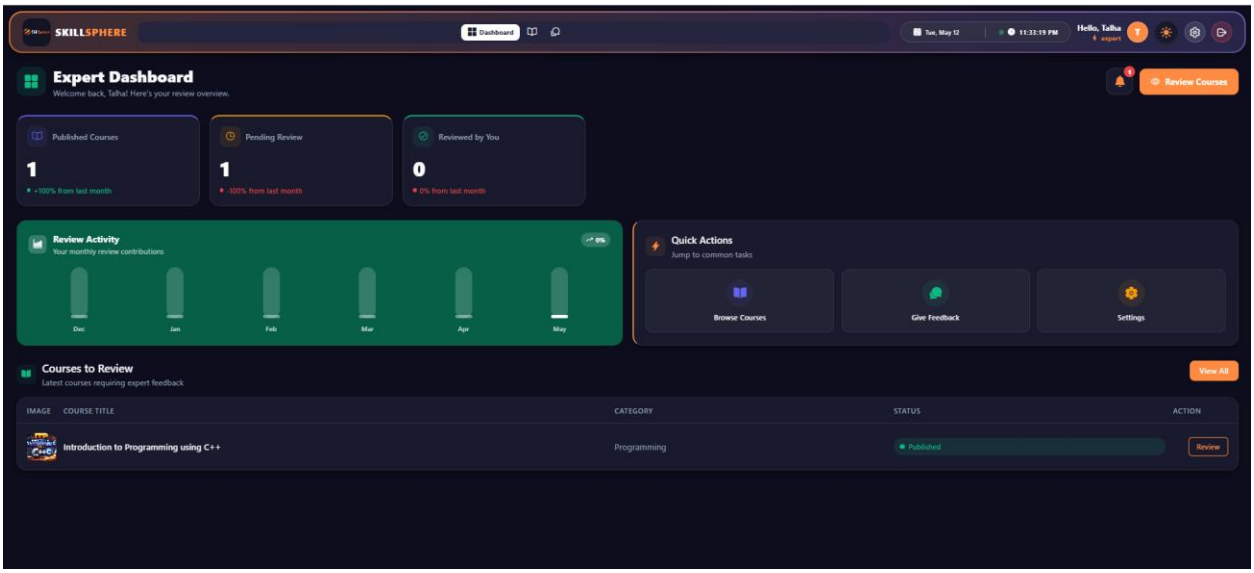



*Figure 50: Expert Dashboard*

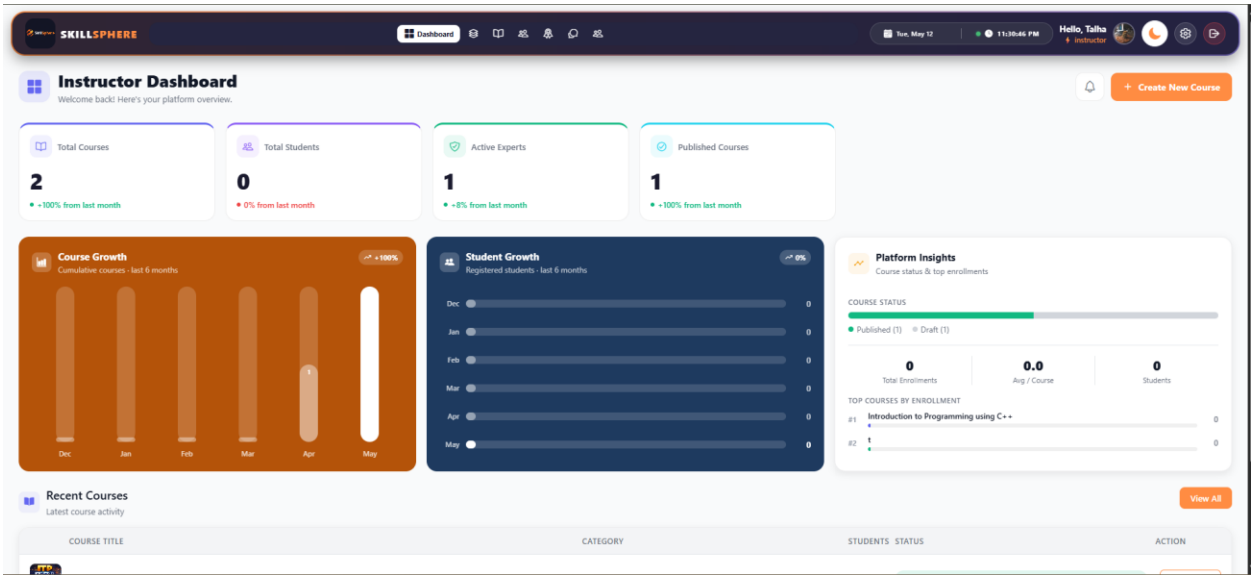



*Figure 51: Instructor Dashboard*

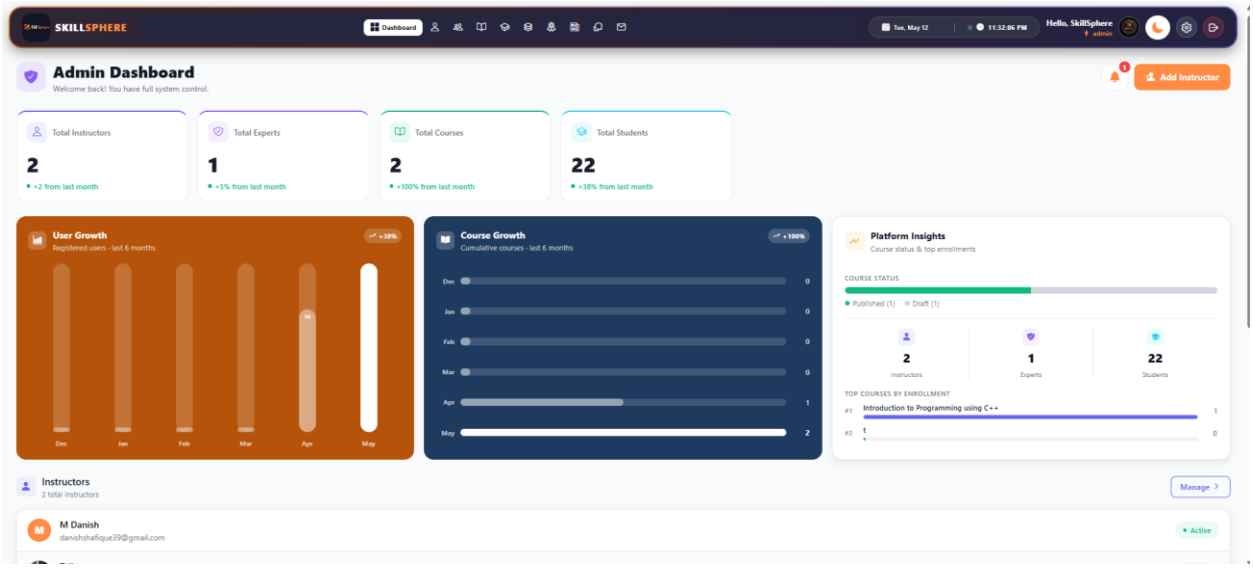



*Figure 52: Admin Dashboard*

## 5.4.6. Test Case: Google OAuth Login

*Table 46: Test Case: Google OAuth Login*

| | |
|---|---|
| **Date** | January 2025 |
| **System** | SkillSphere LMS |
| **Objective** | Google Sign-In Integration |
| **Version** | 1.0 |
| **Test Type** | Integration Testing |
| **Input** | Google OAuth ID token from Firebase |
| **Expected Result** | User is created/found and JWT token is returned. |
| **Actual Result** | **Passed** |

**Description:**

Google OAuth authentication was tested using the Firebase Instructor SDK integration. Clicking "Sign in with Google" opened the Google authentication popup. After successful Google authentication, the Firebase ID token was sent to /api/auth/google-auth. The backend verified the token with Firebase, created a new user account (or found the existing one by email), and returned a JWT token. The authProvider field was correctly set to 'google' for OAuth users, and password field was null for these accounts.

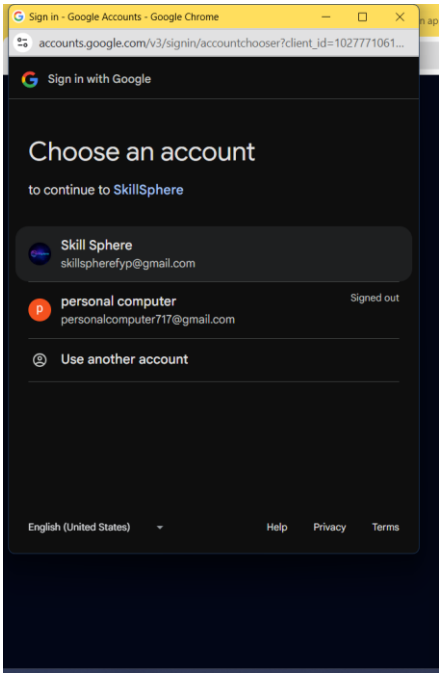



*Figure 53: Test Google OAuth Login Result*

## 5.5. Email Notification Testing

### 5.5.1. Test Case: OTP Email Delivery

*Table 47: Test Case: OTP Email Delivery*

| | |
|---|---|
| **Date** | January 2025 |
| **System** | SkillSphere LMS |
| **Objective** | Verify OTP Email Delivery |
| **Version** | 1.0 |
| **Test Type** | Integration Testing |
| **Method** | POST |
| **Endpoint** | /api/auth/send-otp |
| **Input** | {"email": "test@example.com", "name": "Test User"} |
| **Expected Result** | OTP email is delivered to user's inbox within 30 seconds. |
| **Actual Result** | **Passed** |

**Description:**

The OTP email delivery was tested by triggering the send-otp endpoint. The email was delivered within 30 seconds using the configured SMTP service. The email contained:

- ✓ SkillSphere branding and logo
- ✓ Personalized greeting with user's name
- ✓ 6-digit OTP code prominently displayed
- ✓ OTP expiration time (10 minutes)
- ✓ Security notice advising not to share the code

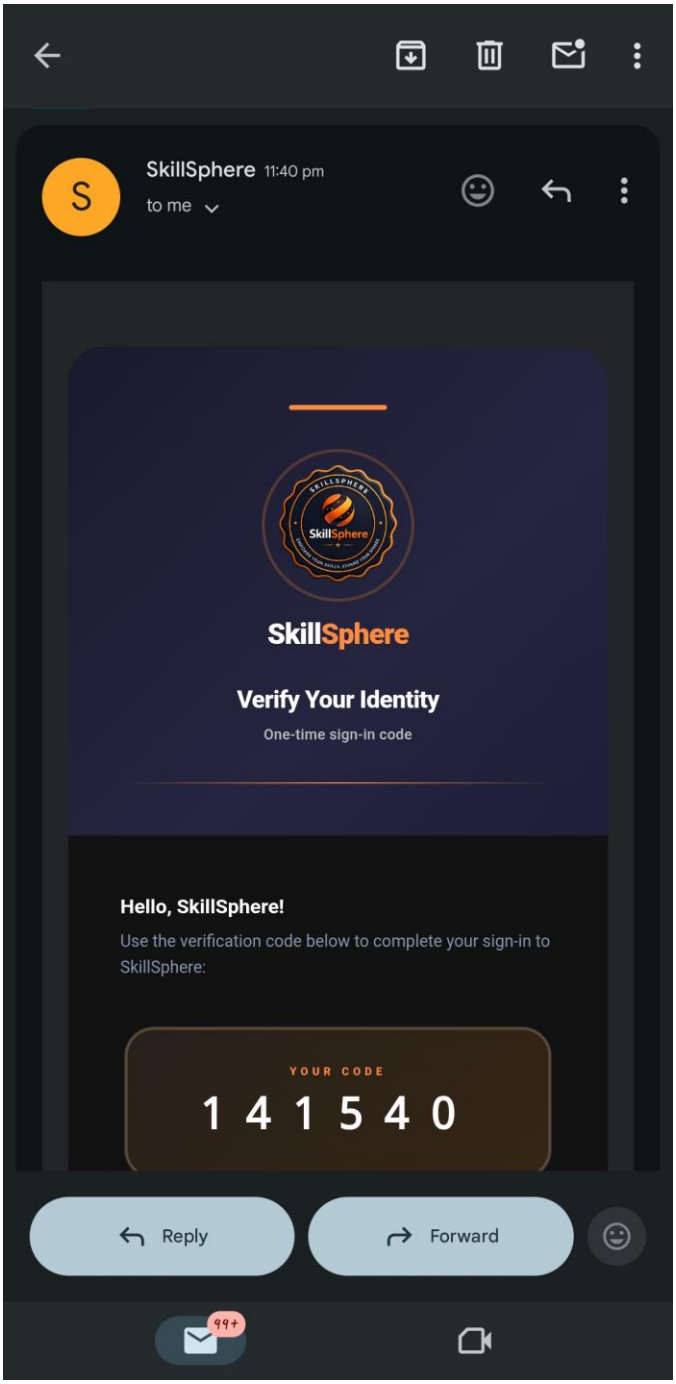



*Figure 54: Test OTP Email Delivery Result*

### 5.5.2. Test Case: Certificate Delivery Email

*Table 48: Test Case: Article Delivery Email*

| | |
|---|---|
| **Date** | February 2025 |
| **System** | SkillSphere LMS |
| **Objective** | Article Delivery Notification |
| **Version** | 1.0 |
| **Test Type** | Integration Testing |
| **Trigger** | Admin Publish Article in a blog |
| **Expected Result** | Email with title and Description of Article Send to Newsletter Subscribers. |
| **Actual Result** | **Passed** |

**Description:**

When Admin Publish Article in a blog, an automatic email notification was sent. The email contained:

- ✓ SkillSphere branding
- ✓ Article Title
- ✓ Article Description

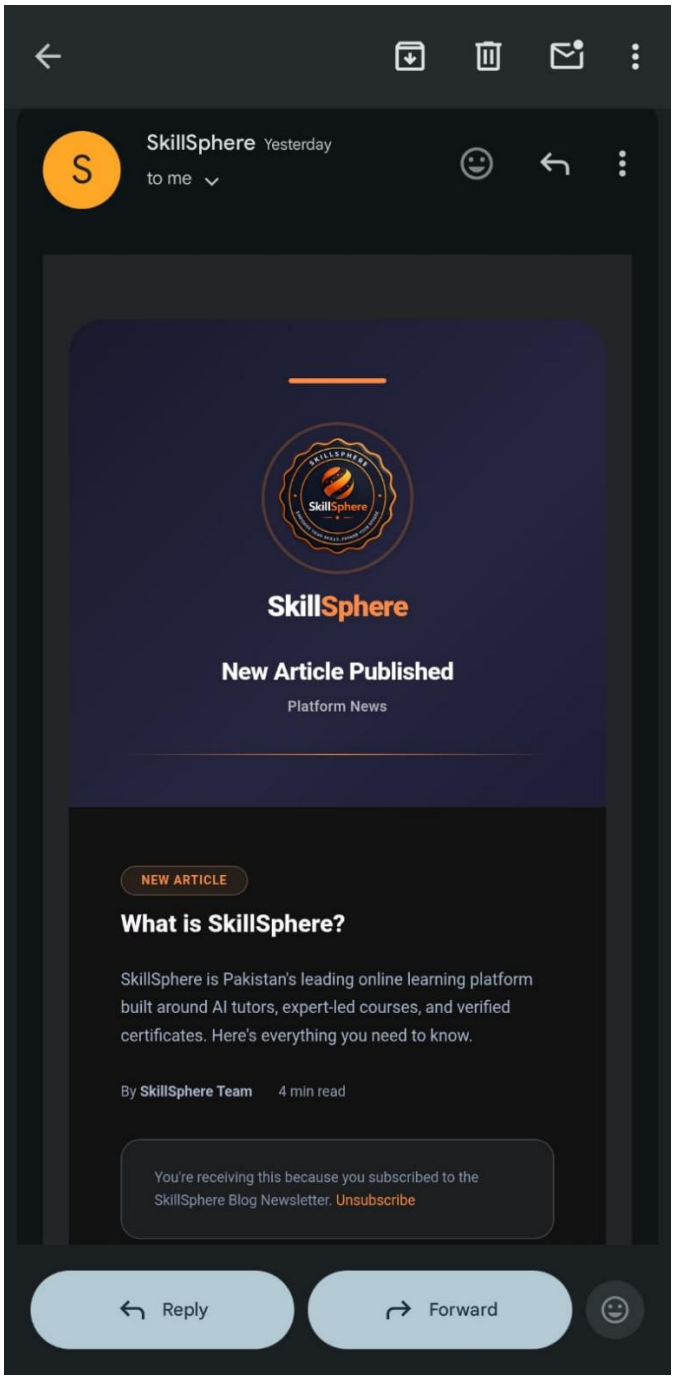



*Figure 55: Test Article Delivery Email Result*

## 5.6. Database Testing with HeidiSQL

HeidiSQL was used to verify the database schema, relationships, and data integrity after API operations. The following database tables were inspected and validated:

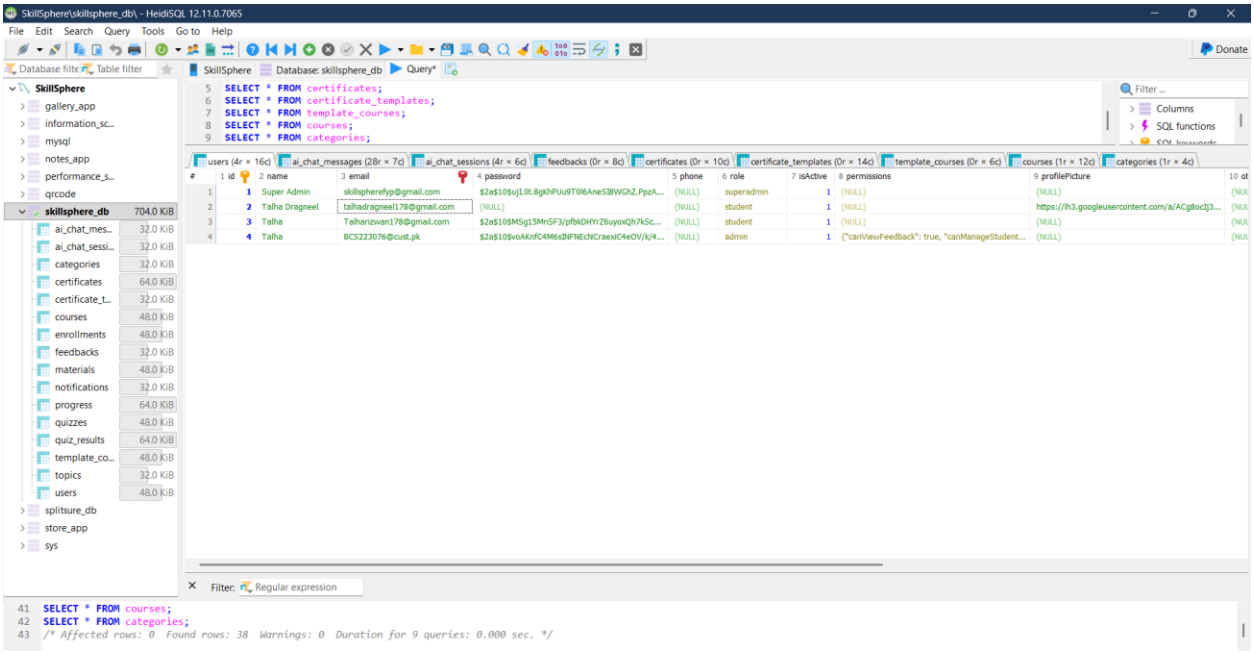



*Figure 56: Database Testing with HeidiSQL*

# *Chapter 6: Software Deployment*

## 6.1. Installation / Deployment Process Description

SkillSphere follows a cloud-based deployment architecture with the backend and frontend hosted on cPanel shared hosting, and media/file storage managed through Cloudinary. The application is accessible at **skillsphere.com.pk**.

### Backend Deployment

1. The backend source code is deployed to the cPanel server by cloning the GitHub repository directly using the cPanel Terminal:
2. Node.js is configured in cPanel and the Express.js application is managed using **PM2** (Process Manager 2). PM2 keeps the server running in the background, automatically restarts it on crashes, and ensures the process survives server reboots.
3. cPanel's Nginx acts as a reverse proxy in front of the Node.js process; the server is configured with `app.set('trust proxy', 1)` to correctly handle forwarded client IPs and HTTPS headers behind Nginx.
4. The server listens on the port provided by the `PORT` environment variable (defaulting to `5000` if not set).
5. All environment variables (database credentials, JWT secret, Cloudinary keys, OpenAI API key, Firebase credentials, email service keys) are configured in cPanel's environment variable manager or a `.env` file on the server.
6. The backend API is served at `https://skillsphere.com.pk/api` (or a subdomain such as `api.skillsphere.com.pk` depending on cPanel virtual host configuration).

The application uses Sequelize ORM with sync({ alter: { drop: false } }), which automatically creates or updates database tables based on model definitions without dropping any existing columns or data. Before the main sync runs, two compatibility scripts — syncCourseColumns and syncAITutorColumns — execute to ensure AI tutor schema columns are aligned. No manual migration scripts are needed for standard deployments.

On first deployment:

- All database tables are automatically created.
- An Admin account is automatically created using credentials from environment variables.
- A welcome email is sent to the Admin.

### File Storage (Cloudinary)

1. A free Cloudinary account is created (25GB storage, 25GB bandwidth/month).

2. Cloud Name, API Key, and API Secret are configured as environment variables.

3. In Cloudinary dashboard, "Allow delivery of PDF and ZIP files" is enabled under Security settings.
4. All uploaded files are stored in organized folders: `skillsphere/uploads/`, `skillsphere/templates/`, `skillsphere/certificates/`.

## Frontend Deployment

1. The React Native Web frontend (`AppAndroidSS/`) is built for production using Webpack:

**npm run build:web**

This produces a static build output in the `web-build/` directory.

2. The contents of `web-build/` are uploaded to the `public_html/` directory (or a subdirectory) on the cPanel server.
3. The `REACT_APP_API_URL` environment variable is set at build time to point to the production backend URL (e.g., `https://api.skillsphere.com.pk`), so the frontend communicates with the correct API endpoint.
4. cPanel's Apache or Nginx serves the static files. An `.htaccess` rewrite rule is configured to redirect all routes to `index.html` to support React's client-side routing (Single Page Application behaviour).
5. The application is accessible to users at **https://skillsphere.com.pk**.

## Environment Variables Required

*Table 49: .env variables Required*

| Variable | Description |
|---|---|
| MYSQL_URL | Railway MySQL connection string |
| JWT_SECRET | Secret key for JWT token signing |
| CLOUDINARY_CLOUD_NAME | Cloudinary account cloud name |
| CLOUDINARY_API_KEY | Cloudinary API key |
| CLOUDINARY_API_SECRET | Cloudinary API secret |
| GEMINI_API_KEY | Google Gemini AI API key |
| FIREBASE_PROJECT_ID | Firebase project ID for Google OAuth |
| FIREBASE_CLIENT_EMAIL | Firebase service account email |
| FIREBASE_PRIVATE_KEY | Firebase service account private key |
| BREVO_API_KEY | Brevo email service API key |
| SMTP_FROM_EMAIL | Sender email address |
| ADMIN_EMAIL | Initial Admin email |
| ADMIN_PASSWORD | Initial Admin password |